

# Error Correcting Codes in Optical Communication Systems

A Master Thesis

Submitted to the School of Electrical Engineering

Department of Signals and Systems

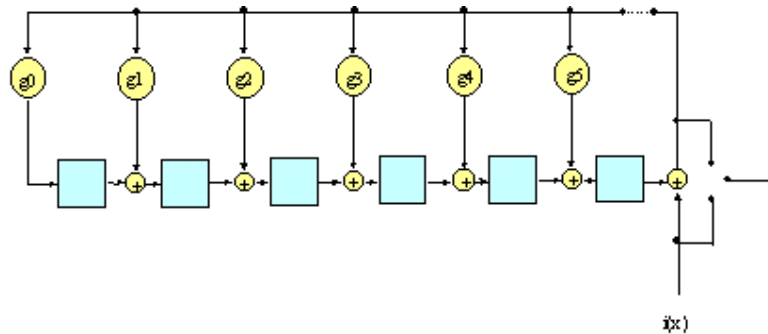
Chalmers University of Technology

For the Degree of

Master of Science (Civilingenjör)

By

Mangesh Abhimanyu Ingale



**CHALMERS**

Examiner: Erik Agrell  
Associate Professor  
Department of Signals and Systems  
Chalmers University of Technology  
Gothenburg, SWEDEN



Technical Supervisor: Tume Römer  
Administrative Supervisor: Dr. Björn Rudberg

January 2003

Dedicated to my parents  
And family members

## Abstract

Error correcting codes have been successfully implemented in wire-line and wireless communication to offer error-free transmission with high spectral efficiency. The drive for increased transmission capacity in fiber-optic links has drawn the attention of coding experts to implement forward error correction (FEC) for optical communication systems in the recent past. Particularly, the ITU-T G.975 recommended Reed-Solomon RS (255, 239) code, is now commonly used in most long-haul optical communication systems. It was shown that the code offers a net coding gain of around 4.3 dB for an output bit-error rate of  $10^{-8}$  after correction. The Monte-Carlo simulation and theoretical performance analysis for the RS (255,239) code with 6.7% redundancy were presented for a completely random distribution of errors over an additive white Gaussian noise (AWGN) channel with BPSK signaling and hard decision decoding. In addition, net coding gain comparison was done between the ITU-T G.975 standard and that offered by the RS (255, 247) and RS (255, 223) codes with 3.2% and 14.3% redundancy respectively.

An attractive solution comprising the serial concatenation of RS codes proposed in [16] was evaluated. The RS (255, 239) + RS (255, 239) and RS (255, 239) + RS (255, 223) concatenated codes with 13.8% and 22.0% redundancy offered a net coding gain of around 5.3 and 5.7 dB respectively with hard decision decoding of the component codes for an output bit-error rate of  $10^{-7}$ . Further improvement in coding gain was achieved by soft decision decoding. The coding gain performance of the Block Turbo Codes with BCH codes as component codes was evaluated through iterative decoding of the component codes with soft-input/soft-output decoder. The BTC's  $(63, 57)^2$  and  $(127, 113)^2$  with 22.16% and 26.13% redundancy offered a net coding gain of 5.25 and 6 dB respectively with soft decision iterative decoding of the component codes for an output bit-error rate of  $10^{-5}$  after correction. It was shown that the redundancy in the code and error correcting capability are crucial code parameters, which define the computational complexity of the encoder and decoder respectively. Algebraic and Transform decoding techniques were studied and analyzed in depth. Issues related to feasible hardware implementation were also addressed.

**Keywords:** Block codes, Maximum likelihood decoding, Concatenated codes, Block Turbo Codes, Iterative decoding.



# Contents

|                                                                  |    |
|------------------------------------------------------------------|----|
| Contents .....                                                   | i  |
| Table of Figures .....                                           | iv |
| Preface .....                                                    | vi |
| 1 Introduction .....                                             | 1  |
| 1.1 Distance-Capacity Metric .....                               | 2  |
| 1.2 Optical Communication System Model .....                     | 2  |
| 1.3 Maximum Likelihood Decoding .....                            | 3  |
| 1.4 Soft-Decision and Hard-Decision Decoding .....               | 4  |
| 2 The Optical Fiber Channel .....                                | 5  |
| 2.1 Channel Impairments.....                                     | 5  |
| 2.1.1 Noise .....                                                | 5  |
| 2.1.2 Dispersion .....                                           | 6  |
| 2.1.3 Fiber Loss and Attenuation .....                           | 7  |
| 2.1.4 NonLinear Effects .....                                    | 7  |
| 2.1.5 Inter-channel Cross talk.....                              | 8  |
| 2.2 Techniques to Compensate for Channel Impairment .....        | 8  |
| 3 Forward Error Correction.....                                  | 10 |
| 3.1 Advantages of Forward Error Correction.....                  | 11 |
| 3.2 Introduction to Error Correcting Codes.....                  | 12 |
| 3.2.1 Error Correcting Codes in Optical Communication Systems .. | 12 |
| 3.3 Galois Fields.....                                           | 13 |
| 3.3.1 Vector Spaces .....                                        | 14 |
| 3.3.2 Properties of Galois Field.....                            | 15 |
| 3.3.3 Construction of Extension Field.....                       | 16 |
| 4 Linear Block Codes.....                                        | 18 |
| 4.1 Systematic Encoding .....                                    | 18 |
| 4.1.1 Properties of Block Codes .....                            | 19 |
| 4.1.2 Generator and Parity Check Matrices .....                  | 19 |
| 4.2 Error Detection and Correction .....                         | 20 |
| 4.2.1 Error Detection .....                                      | 21 |
| 4.2.2 Error Correction.....                                      | 21 |
| 4.3 Standard Array Decoding.....                                 | 22 |

|                                                                 |    |
|-----------------------------------------------------------------|----|
| 4.4 Types of Decoders.....                                      | 23 |
| 4.5 Weight Distribution of a Block Code.....                    | 23 |
| 5 BCH and Reed-Solomon Codes.....                               | 24 |
| 5.1 Linear Cyclic Codes.....                                    | 24 |
| 5.2 Description of BCH and RS Codes.....                        | 25 |
| 5.2.1 Time Domain Description using the Generator Polynomial... | 26 |
| 5.2.2 Systematic Encoding using the Polynomial Division Method  | 27 |
| 5.3 The Galois Field Fourier Transform.....                     | 28 |
| 5.3.1 Systematic Encoding in Frequency Domain.....              | 29 |
| 5.4 Algebraic Decoding Algorithms for BCH and RS Codes          | 30 |
| 5.4.1 Berlekamp's Algorithm for BCH Codes.....                  | 33 |
| 5.4.2 Berlekamp-Massey Algorithm for RS Codes.....              | 34 |
| 5.4.3 Euclid's Algorithm for BCH and RS Codes.....              | 36 |
| 5.5 Transform Decoding of RS Code.....                          | 37 |
| 6 Performance Analysis for BCH and RS Codes.....                | 40 |
| 6.1 Error Detection Performance.....                            | 40 |
| 6.2 Error Correction Performance.....                           | 43 |
| 6.3 Optical Receiver Sensitivity.....                           | 47 |
| 6.3.1 Relation between BER and Q & Q and SNR.....               | 48 |
| 6.4 Hardware Implementation of Galois Field Arithmetic....      | 50 |
| 6.4.1 Encoder Architecture.....                                 | 50 |
| 6.4.2 Decoder Architecture.....                                 | 51 |
| 7 Concatenated Coding.....                                      | 53 |
| 7.1 Concatenated Coding Strategies.....                         | 53 |
| 7.2 Interleaver.....                                            | 54 |
| 7.3 Block Turbo Codes.....                                      | 55 |
| 7.4 Product Codes.....                                          | 56 |
| 7.4.1 RS Product Codes.....                                     | 57 |
| 7.4.2 BCH Product Codes.....                                    | 59 |
| 7.4.3 Soft-Decoding of Linear Block Codes.....                  | 60 |
| 7.4.4 Soft-Input Soft-Output Decoder.....                       | 62 |
| 7.4.5 Turbo Decoding of Product Code.....                       | 64 |
| 8 Theoretical and Simulated Results.....                        | 66 |
| 8.1 Performance of RS Codes.....                                | 66 |

|                                                         |    |
|---------------------------------------------------------|----|
| 8.2 Performance of Serially Concatenated RS Codes ..... | 74 |
| 8.3 Performance of Block Turbo Codes .....              | 77 |
| 8.4 Remarks and Conclusion .....                        | 82 |
| 9 APPENDICES .....                                      | 83 |
| 9.1 Appendix A .....                                    | 83 |
| 9.2 Appendix B .....                                    | 87 |
| 10 BIBLIOGRAPHY .....                                   | 88 |

# Table of Figures

|                                                                                           |    |
|-------------------------------------------------------------------------------------------|----|
| Figure 1.1 Optical Communication System Model .....                                       | 2  |
| Figure 2.1 Optical Channel Impairments .....                                              | 5  |
| Figure 2.2 Optical Channel Noise Classification .....                                     | 6  |
| Figure 2.3 Attenuation Profile of Single-mode Fiber .....                                 | 7  |
| Figure 2.4 Block Diagram of an Equalizer .....                                            | 9  |
| Figure 3.1 Classification of Error Correcting Codes .....                                 | 12 |
| Figure 4.1 Systematic Format of a Codeword .....                                          | 18 |
| Figure 4.2 Additive White Gaussian Noise Channel .....                                    | 20 |
| Figure 4.3 Binary Symmetric Channel (BSC) .....                                           | 20 |
| Figure 5.1 BCH (255, 239) Encoder .....                                                   | 27 |
| Figure 5.2 RS (255, 239) Encoder .....                                                    | 28 |
| Figure 5.3 GFFT Encoder for Reed-Solomon Code .....                                       | 30 |
| Figure 5.4 General Working of a BCH/RS Decoder .....                                      | 33 |
| Figure 5.5 Frequency Domain Decoding .....                                                | 38 |
| Figure 5.6 Transform Decoding of RS (255, 239) Code .....                                 | 39 |
| Figure 6.1 Word Error Detection Performance of BCH (31, 21) Code .....                    | 41 |
| Figure 6.2 Word Error Detection Performance of RS (31, 25) Code .....                     | 43 |
| Figure 6.3 Word Error Correction Performance of BCH (31, 21) Code .....                   | 44 |
| Figure 6.4 Bit-Error Correction Performance of BCH (31, 21) Code .....                    | 45 |
| Figure 6.5 Decoder Word Error Performance of RS (31, 25) Code .....                       | 46 |
| Figure 6.6 Approximate Word Error and Bit-Error Performance of RS (31, 25) Code .....     | 47 |
| Figure 6.7 Fluctuating Signal at the Receiver .....                                       | 49 |
| Figure 6.8 Encoder Computational Complexity vs. Redundancy .....                          | 51 |
| Figure 6.9 Decoder Computational Complexity vs. Error Correcting Capability .....         | 52 |
| Figure 7.1 Serial Concatenated Coding Scheme .....                                        | 54 |
| Figure 7.2 Row–Column Interleaver .....                                                   | 54 |
| Figure 7.3 BCH (255, 239) + BCH (255, 239) Concatenated Code .....                        | 55 |
| Figure 7.4 Construction of Product Code .....                                             | 56 |
| Figure 7.5 Serial Concatenation of RS Codes .....                                         | 57 |
| Figure 7.6 RS Product Code .....                                                          | 58 |
| Figure 7.7 BTC (255, 239) <sup>2</sup> .....                                              | 59 |
| Figure 7.8 Chase Decoder .....                                                            | 60 |
| Figure 7.9 PDF of Extrinsic Information .....                                             | 63 |
| Figure 7.10 Block Diagram of Elementary Block Turbo Decoder .....                         | 64 |
| Figure 7.11 Flow Chart for Turbo Decoding .....                                           | 65 |
| Figure 8.1 Theoretical Performance of the RS Codes .....                                  | 67 |
| Figure 8.2 Simulated Performance of the RS Codes .....                                    | 67 |
| Figure 8.3 Simulated Performance of the RS Codes (Extrapolated to $\leq 10^{-12}$ ) ..... | 68 |
| Figure 8.4 Theoretical Output BER vs. Input BER Performance .....                         | 69 |
| Figure 8.5 Simulated Output BER vs. Input BER Performance .....                           | 69 |
| Figure 8.6 Symbol Error Rate Performance (Theoretical) .....                              | 70 |
| Figure 8.7 Symbol Error Rate Performance (Simulated) .....                                | 70 |
| Figure 8.8 RS (255, 247) Codec .....                                                      | 71 |
| Figure 8.9 RS (255, 239) Codec .....                                                      | 72 |

|                                                                                                |           |
|------------------------------------------------------------------------------------------------|-----------|
| <b>Figure 8.10 RS (255, 223) Codec .....</b>                                                   | <b>73</b> |
| <b>Figure 8.11 Approximate Output BER Performance of RS Product Codes .....</b>                | <b>75</b> |
| <b>Figure 8.12 Simulated Output BER Performance of RS Product Codes.....</b>                   | <b>75</b> |
| <b>Figure 8.13 Simulated Performance of RS Product Codes after 2 Iterations .....</b>          | <b>76</b> |
| <b>Figure 8.14 Simulated Performance of BTC (127, 113)<sup>2</sup> .....</b>                   | <b>77</b> |
| <b>Figure 8.15 Simulated Performance of BTC (63, 57)<sup>2</sup> after 3 Iterations.....</b>   | <b>78</b> |
| <b>Figure 8.16 Simulated Performance of BTC (127, 113)<sup>2</sup> after 3 Iterations.....</b> | <b>78</b> |
| <b>Figure 8.17 PDF of Extrinsic Information BTC (127, 113)<sup>2</sup> .....</b>               | <b>79</b> |
| <b>Figure 8.18 PDF of Extrinsic Information BTC (63, 57)<sup>2</sup> .....</b>                 | <b>79</b> |
| <b>Figure 8.19 Simulated Performance of BTC (255, 239)<sup>2</sup> after 2 Iterations.....</b> | <b>80</b> |
| <b>Figure 8.20 Performance of all BTC's (Extrapolated).....</b>                                | <b>80</b> |

# Preface

The Master thesis was conducted during July 2002-January 2003 at Optillion AB, Stockholm, Sweden, towards the completion of Master of Science "Civilingenjör" degree at the Department of Signals and Systems, School of Electrical Engineering, Chalmers University of Technology.

The pioneering work lead by Claude Shannon in 1948 and the significant and breakthrough contribution from coding theorist like Hamming, Peterson, Bose-Ray Chaudhari, Hocquenghem, Reed and Solomon, E.Berlekamp, Massey, G.D.Forney Jr and many more was the source of inspiration and motivation to develop an interest in the field of Information theory and Coding theory. The opportunity knocked the door in the form of Master thesis to be carried out at Optillion AB.

## Thesis Outline

This thesis investigates the class of error control codes to be used to perform forward error correction in optical communication systems. After a comprehensive literature survey, it was observed that block codes are most suitable candidates codes. The BCH code is a binary code belonging to the sub-class of linear cyclic block codes having multiple random error correcting capability while the RS code is a nonbinary code also belonging to the sub-class of linear cyclic block codes having multiple random as well as burst error correcting capability. One important attribute of BCH and RS codes is that they offer error correction at high code rate, which make them very attractive for optical communication applications.

Chapter 1 introduces the system model used in the thesis and important concepts like maximum-likelihood decoding, followed by a brief description of soft and hard decision decoding. In chapter 2, we present the different channel impairments that degrade the received signal quality and the techniques used to compensate them.

Chapter 3 gives a brief overview of the different error correcting codes available in communication theory. It describes the advantages of forward error correction and a short overview of Galois fields, which forms the mathematical base for the BCH/RS codes. Chapter 4 describes in short the properties of linear block codes and error event handling.

In Chapter 5, we have introduced the BCH and RS codes. A comprehensive treatment is given to the encoding and decoding techniques for the BCH/RS codes. The Berlekamp, Berlekamp-Massey and Euclid's algorithm used to decode BCH and RS codes is presented. Encoding/decoding of the RS codes in the frequency domain using the transform technique is also discussed in detail.

In Chapter 6, we have evaluated the upper bound on the code word error detection and code word error correction performance of the BCH/RS codes. Upper bound, exact and approximate decoder word error probabilities for the BCH/RS codes are also evaluated. The computational complexity of the encoder/decoder is evaluated and a short overview of the hardware implementation of the BCH/RS encoder/decoder is given towards the end of the chapter.

Chapter 7 introduces the principle behind concatenated coding technique. Encoding of Block Turbo codes is discussed in detail. The mathematical formulation for iterated decoding of Block Turbo codes using soft-input/soft-output component decoder based on Chase algorithm is presented. In chapter 8, we present the analytical and simulated output bit-error performance for the different RS codes. The simulated output bit-error performance for the serially concatenated RS code and Block Turbo code with BCH codes as component codes is also presented. Comparative analysis of the coding gain performance of the different codes

based on redundancy and code rate is performed. We conclude the thesis report with some suggestions and direction for future work.

## **Methodology**

The methodologies applied throughout this thesis were analytical analysis and computer simulations. The simulations were performed with custom coded MATLAB functions running on a Pentium 4 Windows 2000 workstation. The results of the analytical analysis and computer simulations were presented using bit-error rate vs.  $E_b/N_o$  or Q plots. Tables were also used to present word enumerators of some codes and parameters of potential codes to be used in optical communication systems.

## **Acknowledgements**

Initially there was uncertainty whether the offered project would be continued due to unavoidable circumstances at the Gothenburg office. I am grateful to Dr. Thomas Swahn and Dr. Joakim Hallin for their efforts to reschedule the work to be continued at the Stockholm facility. I appreciate and thank Dr. Björn Rudberg and Mr. Tume Römer for their positive attitude shown for supervising the thesis work. Their suggestions and encouragement was helpful during the course of the work. I would like to express my gratitude to Professor Erik Agrell and Professor Erik Ström for the valuable advice and academic counseling throughout my Master studies at the Chalmers University of Technology. Professor Erik Agrell has played an instrumental role in motivating me and contributing critical suggestions throughout the progress of the thesis project. He has always been willing to take time out of his hectic schedule to review the work and make corrections and improvements in the interim and final draft of the thesis report. I thank Johan Sjölander for participating enthusiastically in the discussions. I would also like to appreciate the amicability and co-operation from the employees at Optillion, particularly the members of Optics and Electronic Design Group with whom I had colloquy on interesting topics related to India and Sweden during lunch break. I would like to specially thank Mr. Aravind Sanadi (Ericsson AB) and his family for extending moral support during my stay in Stockholm. Last but not the least, I thank my parents for taking all the hardships throughout my upbringing and giving me world-class education.

*Mangesh Abhimanyu Ingale  
Stockholm, Sweden*



# Chapter 1

## 1 Introduction

The noisy channel-coding theorem states that, the basic limitation that noise causes in a communication channel is not on the reliability of communication, but on the speed of communication [49]-[50]. The capacity of an additive white Gaussian noise (AWGN) channel is given by

$$C = W \log \left( 1 + \frac{P}{N_o W} \right) \text{ bits/sec} \quad 1.1$$

where

W: channel bandwidth in Hz.

P: signal power in watts

$N_o$ : noise power spectral density in watts/Hz.

Channel capacity depends on two parameters namely, the signal power P and the channel bandwidth W. Increase in channel capacity as a function of power is logarithmic [9]. By increasing the channel bandwidth infinitely, we obtain

$$\lim_{W \rightarrow \infty} C = 1.44 \frac{P}{N_o} \quad 1.2$$

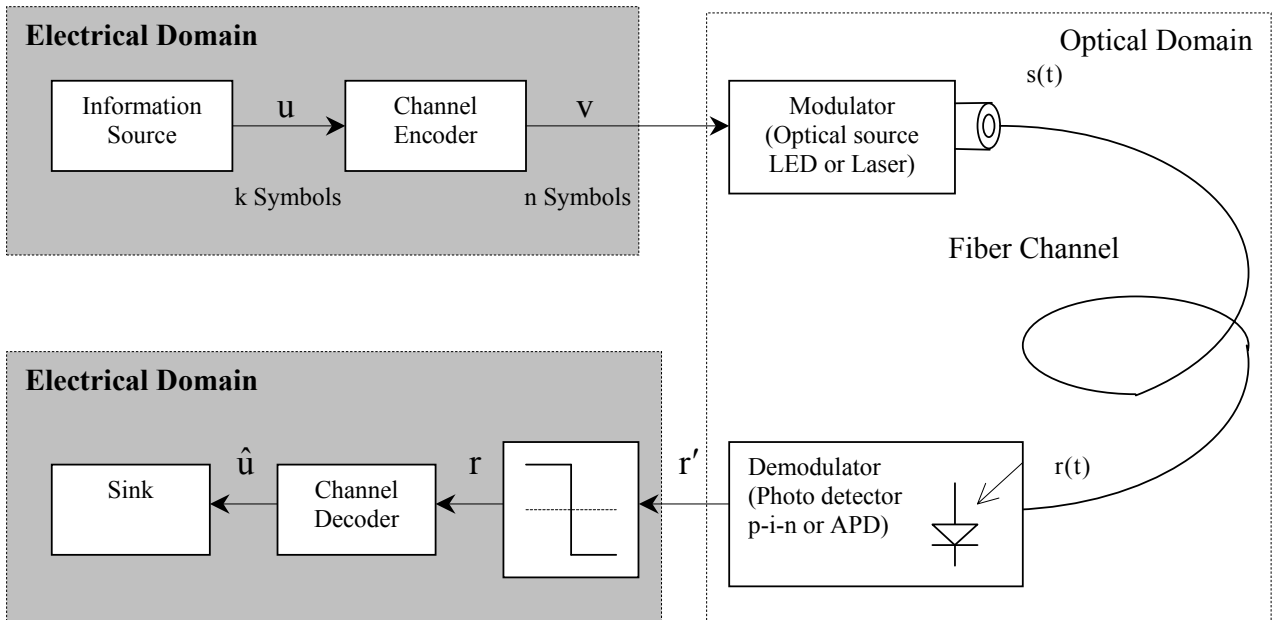
Equation 1.2 means that channel capacity cannot be increased to any desired value by increasing W thus imposing a fundamental limitation on the maximum achievable channel capacity. Shannon [49]-[50], stated that there exist error control codes such that information can be transmitted across the channel at the transmission rate R below the channel capacity C with error probability close to zero. Thus, *error-correction (control) coding* (ECC), essentially a signal processing technique, in which controlled redundancy is added to the transmitted symbols to improve the reliability of communication over noisy channels can achieve transmission rate R close to the channel capacity C. The channel bandwidth of an optical communication system is larger (~100 THz) by a factor of nearly 10,000 times than that of microwave systems (~10 GHz). However, the channel capacity is not necessarily increased by the same factor because of the fundamental limitation stated above. The channel capacity given by 1.1 is the theoretical upper limit for a given optical fiber and depends upon the type of fiber. The present semiconductor and high speed optics technology is also a limitation to achieve excessively high data rates to take complete advantage of the enormous bandwidth offered by the fiber cable. Current light wave systems using single-mode fibers operate below the theoretical channel capacity, with bit rates 10 Gbits/s and above.

## 1.1 Distance-Capacity Metric

Just as speed-power product is a popular measure for gauging IC performance, distance-capacity product provides a useful baseline for comparing optical communication systems. It is important to understand the channel impairments (refer chapter 2) that limit the practically feasible transmission rates. However, increase in transmission rates can be achieved through multiplexing of multiple channels over the same fiber by the use of Time Division (TDM) or Frequency Division Multiplexing (FDM) techniques. In the optical domain, FDM is referred as Wavelength Division Multiplexing (WDM). TDM increases the data rate of the system by sending more data in the same amount of time by allocating less time for each individual bit that is sent. But we have to pay a price for it not only in terms of increased component complexity associated with the transmission, but also the properties of the fiber and the signal degrade at higher data rates. TDM is not considered any further, while treatment to WDM is given where necessary. By using, WDM different wavelengths (frequencies)<sup>1</sup> are used to transmit independent channels of information. Again, there are limiting factors to WDM transmission that involve the degradation of signal transmission quality. The distance-capacity metric is a function of two parameters, viz., the number of wavelengths that can be transmitted via WDM and the rate at which data is transmitted on these wavelengths. We will see in section 2.1.3 by increasing the capacity by adding wavelengths or increasing the data rate will decrease the distance the link can span with unchanged optical parameters. Improving one factor – distance or capacity will result in a reduction of the other, allowing the distance-capacity product to remain constant.

## 1.2 Optical Communication System Model

The optical communication system model is depicted in figure 1.1. The *information source* generates a sequence of binary bits. For the Reed-Solomon encoder these binary bits are grouped to form q-ary symbols from GF ( $q=2^m$ ) (refer sections 3.3, 4.1 and 5.2). These symbols<sup>2</sup> (bits) in turn are grouped into blocks of k symbols denoted by  $u = (u_1, u_2, \dots, u_k)$  the *message block*. The *channel encoder* adds controlled amount of redundant symbols to each of the k symbol message blocks to form *code word* blocks of n symbols denoted by  $v = (v_1, v_2, \dots, v_n)$ .



**Figure 1.1 Optical Communication System Model**

<sup>1</sup> The terms wavelength and frequency are used interchangeably throughout the report

<sup>2</sup> symbols: Reed-Solomon codes and bits: BCH codes. The term symbols and bits are used interchangeably depending upon the code that is used.

In our work, we have used a digital transmission scheme in which an electrical bit stream modulates the intensity of the optical carrier (modulated signal). The modulated signal is detected directly by a photo detector to convert it to the original digital (modulating) signal in the electrical domain. It is referred to as *Intensity Modulation with Direct detection* (IM/DD) or *On-Off Keying* (OOK) or *Amplitude Shift Keying* (ASK). When the RS encoder is used, the q-ary symbols have to be translated into a sequence of  $\log_2(q)$  binary bits before driving the optical source. The output of the *modulator* is an optical pulse of duration T for bit 1 and no pulse for bit 0. Thus a signal waveform  $s_i(t)$  of duration T is transmitted over the fiber optic channel such that

$$\begin{aligned} s_i(t) &= A \quad ; i = 1 \\ &= 0 \quad ; i = 0 \end{aligned} \quad 1.3$$

where A is the amplitude of the transmitted optical pulse

A mathematical model for the optical channel is the AWGN channel which models Gaussian (thermal) noise present at the receiver's front-end electronic circuitry. In the model, a Gaussian random noise process  $n(t)$  is added to the transmitted signal waveforms  $s(t)$  [14]. We have introduced the AWGN channel model in section 4.2, which is a vector representation where  $\mathbf{v}$  represents the transmitted code word,  $\mathbf{e}$  represents the white noise process and the corrupted received word is represented as  $\mathbf{r}'$ . At the receiver, the *demodulator* is a photo detector (p-i-n or APD), which converts the received optical signal  $r(t)$  into electrical current  $I(t)$  (refer section 6.3). The vector  $\mathbf{r}'$  in the model is obtained as the output of the demodulator. The vector  $\mathbf{r}'$  contains sufficient statistics for the detection of the transmitted symbols [14]. A sequence of vector  $\mathbf{r}'$  is then fed to the *decoder*, which attempts to reconstruct the original message block  $\hat{\mathbf{u}}$ , using the redundant symbols. In many situations, the vector  $\mathbf{r}'$  is passed through the threshold *detector*, which provides the decoder a vector  $\mathbf{r}$  with only binary zeros and ones. In such a case the decoder is said to perform hard decision decoding and the resulting channel consisting of the modulator, AWGN channel, demodulator and detector is called the *Binary Symmetric Channel* (refer section 4.2) [14]. The AWGN and BSC channel models are used throughout the report for our analysis.

### 1.3 Maximum Likelihood Decoding

Assuming that the decoder has received a vector  $\mathbf{r}'$ , which is unquantized the optimum decoder that minimizes the probability of error will then select the vector  $\hat{\mathbf{v}} = \mathbf{v}_j$  iff [14]

$$\Pr(\mathbf{v}_j | \mathbf{r}') > \Pr(\mathbf{v}_i | \mathbf{r}') \quad ; \forall i \neq j \quad 1.4$$

This is known as the *maximum a posteriori probability* (MAP) criterion. Using Bayes rule

$$\Pr(\mathbf{v}_i | \mathbf{r}') = \frac{\Pr(\mathbf{v}_i) p(\mathbf{r}' | \mathbf{v} = \mathbf{v}_i)}{p(\mathbf{r}')} \quad 1.5$$

where  $\Pr(\mathbf{v}_i | \mathbf{r}')$  for  $i = 1, 2, \dots, q^k$  are the posterior probabilities

$p(\mathbf{r}' | \mathbf{v} = \mathbf{v}_i)$  is the conditional pdf of  $\mathbf{r}'$  given  $\mathbf{v}_i$  is transmitted & called the likelihood function  
 $\Pr(\mathbf{v}_i)$  is the a priori probability of the  $i^{\text{th}}$  vector being transmitted and

$$p(\mathbf{r}') = \sum_{i=1}^{q^k} p(\mathbf{r}' | \mathbf{v}_i) \Pr(\mathbf{v}_i) \quad 1.6$$

Computation of the posterior probabilities  $\Pr(\mathbf{v}_i | \mathbf{r}')$  is simplified when the  $\mathbf{q}^k$  vectors are equiprobable and  $p(\mathbf{r}')$  is independent of the transmitted vector. The decision rule based on finding the vector that maximizes  $\Pr(\mathbf{v}_i | \mathbf{r}')$  is equivalent to finding the signal that maximizes  $p(\mathbf{r}' | \mathbf{v} = \mathbf{v}_i)$ . Thus the MAP criterion simplifies to the *maximum-likelihood* (ML) criterion and the optimum decoder then sets  $\hat{\mathbf{v}} = \mathbf{v}_j$  iff

$$p(\mathbf{r}' | \mathbf{v} = \mathbf{v}_j) > p(\mathbf{r}' | \mathbf{v} = \mathbf{v}_i) \quad ; \forall i \neq j \quad 1.7$$

For the AWGN channel the likelihood function is given as

$$p(\mathbf{r}' | \mathbf{v} = \mathbf{v}_i) = p(\mathbf{r}' - \mathbf{v}_i | \mathbf{v} = \mathbf{v}_i) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-|\mathbf{r}' - \mathbf{v}_i|^2 / 2\sigma^2} \quad 1.8$$

Taking the natural logarithm, we have

$$\ln p(\mathbf{r}' - \mathbf{v}_i | \mathbf{v} = \mathbf{v}_i) = -\frac{1}{2} \ln(2\pi\sigma^2) - \frac{1}{2\sigma^2} |\mathbf{r}' - \mathbf{v}_i|^2 \quad 1.9$$

Consequently the optimal ML decoder will set  $\hat{\mathbf{v}} = \mathbf{v}_j$  iff

$$|\mathbf{r}' - \mathbf{v}_j|^2 < |\mathbf{r}' - \mathbf{v}_i|^2 \quad ; \forall i \neq j \quad 1.10$$

$$|\mathbf{r}' - \mathbf{v}_i|^2 = \sum_{l=1}^n (r_l - v_{il})^2 \quad ; \forall i = 1, 2, \dots, q^k \quad 1.11$$

is the squared Euclidean distance. Hence, for the AWGN channel, the decision rule based on ML criterion simplifies to finding the vector  $\mathbf{v}_i$  that is closest to the received vector  $\mathbf{r}'$  in the Euclidean distance sense. If hard decision is made on  $\mathbf{r}'$  prior to decoding by means of a threshold detector, then the BSC replaces the AWGN and the ML decoder will select  $\hat{\mathbf{v}} = \mathbf{v}_j$  iff

$$p(\mathbf{r} | \mathbf{v} = \mathbf{v}_j) > p(\mathbf{r} | \mathbf{v} = \mathbf{v}_i) \quad ; \forall i \neq j \quad 1.12$$

where  $\mathbf{r} = (r_1, r_2, \dots, r_n)$ , and  $r_l \in \{0, 1\}; \forall l = 1, 2, \dots, n$ . The BSC flips a binary '0' to a binary '1' with probability  $p$  called the *transition probability* (refer figure 4.3). The number of components in which  $\mathbf{r}$  and  $\mathbf{v}_i$  differ is called the Hamming distance and the ML decoding criterion for hard decision decoding simplifies to finding  $\mathbf{v}_i$  that is closest to the received vector  $\mathbf{r}$  in the Hamming distance sense. Our further discussion throughout the report regarding decoding is related only to hard decision decoding unless otherwise stated explicitly. We defer our discussion on decoding based on ML criterion to section 4.2.

## 1.4 Soft-Decision and Hard-Decision Decoding

We conclude the chapter stating that the received vector  $\mathbf{r}'$  (a point in the Euclidean space) is obtained by passing the received signal waveform through the demodulator and then the decoder chooses  $\mathbf{v}$  closest to  $\mathbf{r}'$  in the Euclidean distance sense from all possible  $\mathbf{q}^k$  code words. This type of decoding that involves finding the minimum Euclidean distance is called *soft-decision decoding* and involves computations on unquantized values. *Hard-decision decoding* involves quantizing the components of the received vector  $\mathbf{r}'$  to the discrete levels used at the transmitter to obtain  $\mathbf{r}$  and then to find the code word  $\mathbf{v}$  that is closest to the received word  $\mathbf{r}$  in the Hamming distance sense. Soft-decision decoding is the optimal detection method and achieves a lower probability of error [14]. For both cases, computation of the distances is a complex operation even for a small value of  $k$ . However, there exist algorithms that reduce the computational complexity to a considerable extent which are further discussed in the succeeding chapters.

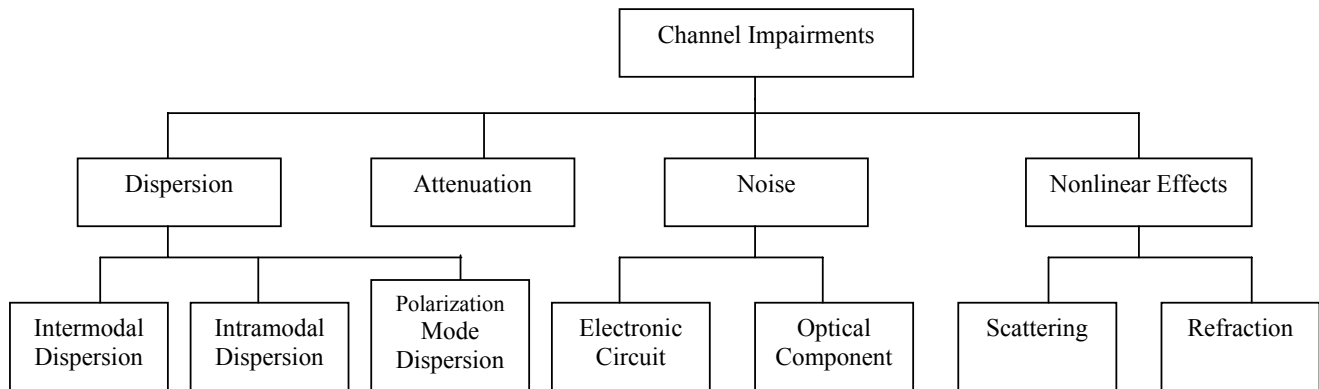
# Chapter 2

## 2 The Optical Fiber Channel

In order to get a broad and basic understanding of the principles and operation of optical communication systems, we review the different types of optical fiber; discuss the effects of channel impairments in section 2.1 and techniques to compensate for channel impairments in section 2.2. The fiber mode, in a crude way, can be described as one of the various possible patterns of propagating electromagnetic field through the fiber. A single-mode fiber supports long-distance signal transmission of a single ray or fundamental mode of light at a particular wavelength( $\lambda$ ). Multimode fibers, on the other hand, allow multiple light rays concurrently each at a slightly different reflection angle within the optical fiber core.

### 2.1 Channel Impairments

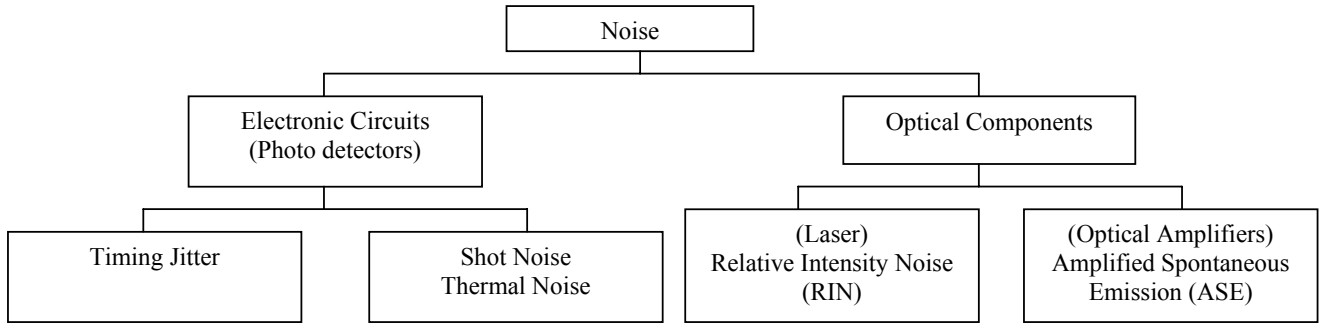
We will discuss fiber channel impairments in terms of dispersion, attenuation and noise in general, and nonlinear effects and inter-channel cross talk particularly in WDM systems.



**Figure 2.1 Optical Channel Impairments**

#### 2.1.1 Noise

Factors contributing to the degradation in signal-to-noise ratio (SNR) at the receiver in optical fiber are due to impairments introduced by a combination of electronic circuits, optical components such as add/drop multiplexers, optical cross-connects and fiber optics. Electronic interface circuits introduce timing jitter occurring due to fluctuations in sampling time from bit to bit, shot noise due to random fluctuations of charge carriers and thermal noise due to random thermal motion of electrons in the photo detector.



**Figure 2.2 Optical Channel Noise Classification**

Optical components– lasers introduce fluctuations in transmitted power and optical amplifiers introduce amplified spontaneous emission (ASE) noise, which has constant spectral density (white noise) [1]. Both shot and thermal noise have approximately Gaussian statistics [1] such that

$$I(t) = I_p + i_s(t) + i_T(t) \quad 2.1$$

where

$I(t)$  : photodiode current generated in response to an optical signal

$I_p$  : the average current

$i_s(t)$  : current fluctuation related to shot noise

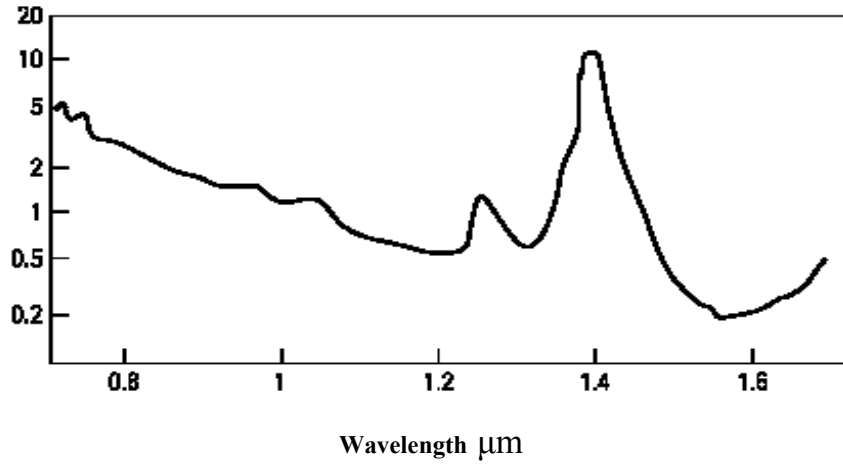
$i_T(t)$  : current fluctuation induced by thermal noise

### 2.1.2 Dispersion

Dispersion in the fiber-optic systems causes the optical pulses to broaden in time as they travel through the fiber, thus giving rise to intersymbol interference (ISI). The effect of ISI is severe at high transmission rates and increased link lengths [18]. Dispersion in optical fibers can be classified broadly as Intermodal (multipath or differential mode) dispersion, Intramodal (group-velocity or chromatic) dispersion and Polarization Mode dispersion (PMD). Intermodal dispersion (*between rays or modes*) occurs in multimode fibers, where different rays travel along paths of different lengths resulting in spread of the optical pulse at the output end of the fiber. Using the concept of fiber modes in context with the wave propagation theory, intermodal dispersion is due to different group velocities associated with different modes. Intermodal dispersion does not occur in single-mode fibers. Multimode fibers are not suitable for long-distance communication [1]. Multimode fibers and Intermodal dispersion are not discussed any further in the report.

Group velocity dispersion (GVD) causes pulse broadening in single-mode fibers within the fundamental mode, since the group velocity of photons associated with the fundamental mode is frequency dependent. The optical source does not emit just a single frequency but a band of frequencies centered around a desired frequency such that the different spectral components of the transmitted signal have different propagation delays. Consequently, the different spectral components travel at slightly different speeds resulting in dispersion. GVD depends upon the transmitted pulse shape, spectral width of the light source and chirp [52]. Broad spectral width of the light source results in a range of wavelengths separated infinitesimally. Thus, signals with high data rate and broad spectral width are distorted by GVD. A transmitted pulse is said to be chirped if its carrier frequency changes with time [1]. Lasers generally generate pulses that are chirped. The spectrum of a chirped pulse is broader than that of the unchirped pulse. In short, broadening of the spectrum accentuate GVD. A single-mode fiber supports two orthogonal states of polarization for the same fundamental mode (*polarization-division multiplexing PDM*). With PDM, two channels at the same wavelength are transmitted. The orthogonal polarized components of the fundamental mode undergo birefringence (*splitting of a light wave into two unequally reflected waves*) due to

#### Attenuation dB/km



**Figure 2.3 Attenuation Profile of Single-mode Fiber**

irregularities in the cylindrical symmetry of the core [1]. The resulting pulse broadening is due to difference in speed of the light waves and called Polarization Mode dispersion (PMD). Since the birefringence changes with temperature, pressure, stress and other physical conditions, the impairments due to PMD are time-variant. PMD is proportional to the square root of the fiber length hence PMD is significant in long-haul communication systems. Dispersion is typically measured in picoseconds per kilometer. After GVD, PMD is the next critical bottleneck for higher bit rate transmission systems (10 Gbits/s and above).

#### 2.1.3 Fiber Loss and Attenuation

Fiber loss reduces the average received power at the receiver. The transmission distance is inherently limited by fiber loss since minimum threshold power must be available at the receiver to recover the transmitted signal. If  $P_{in}$  is the input power to a fiber of length  $L$  &  $P_{out}$  is the output power at the other end of the fiber, then the attenuation coefficient  $\alpha$  [1] is given by

$$\alpha = -\frac{10}{L} \log_{10} \left( \frac{P_{out}}{P_{in}} \right) \text{ dB/km} \quad 2.2$$

Fiber loss depends on the wavelength of transmitted light. First minimum is observed at around 850 nm, second at 1310 nm and third in the wavelength region near 1550 nm. Multimode fibers operate in the 850 nm region while 1310 nm wavelength is used both for single and multimode fibers. Short single-mode fiber links operate at 1310 nm while long-distance communication is at 1550 nm where the attenuation is the minimum. The 10 Gbits/s Ethernet operates at both 1310 nm and 1550 nm. WDM divides the optical power among multiple channels, attenuating them further. Therefore, increasing capacity by adding WDM channels leads to increased attenuation of optical signal per wavelength and hence decreases transmission distance. Material absorption and scattering also contribute to fiber losses.

#### 2.1.4 Nonlinear Effects

The fiber channel also introduces nonlinear effects like scattering and Kerr effects, which are dependent on the intensity of optical power. The refractive index of the fiber core depends upon the intensity of light at high power levels [1]. The Kerr effects are due to intensity dependence of the refractive index, such that signals with different intensity levels travel at different speeds in the fiber.

The effects are explained below

- ◆ Self phase modulation (SPM) in which the phase of the signal gets modulated such that a wavelength can spread out onto an adjacent wavelength.
- ◆ Cross phase modulation (XPM), whereby several wavelengths in a WDM system can cause each other to spread out.
- ◆ Four-wave mixing (FWM) observed in WDM systems in which three signals at different wavelengths interact with each other to create a fourth signal at a new wavelength.

The Kerr Effects account to spectral broadening. In WDM systems, nonlinear effects cause interference by spreading energy from one wavelength channel into another channel. These effects can be reduced by transmitting signals at low power levels at the expense of signal-to-noise ratio and reduced link lengths.

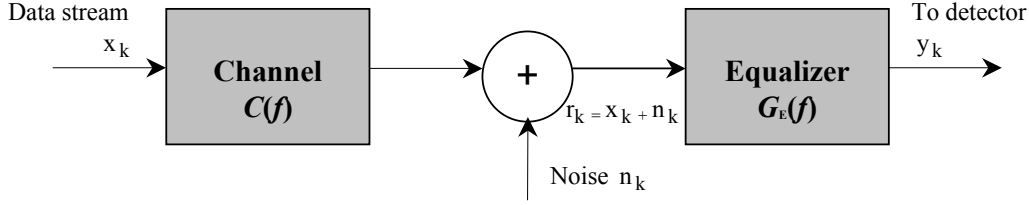
### ***2.1.5 Inter-channel Cross talk***

Linear cross talk occurs when optical filters and multiplexers often leak a fraction of signal power from neighboring channels to the photo detector [1]. Such a cross talk is out-of-band and is less severe because of its incoherent nature. In-band cross talk, also linear, occurs when a WDM signal is routed by an  $N \times N$  wave guide-grating router (WGR) [1]. The routing is based solely on the wavelengths of incoming channels. In-band cross talk is coherent in nature. In multichannel systems, transfer of power from one channel to another takes place due to Kerr Effects (refer section 2.1.4). Such a cross talk is nonlinear. Inter-channel cross talk may be one of the reasons behind burst errors in multichannel fiber-optic communication. Scattering of light is a loss mechanism in the optical fiber and occurs due to fluctuation in silica density in the core during fabrication [1]. Two types of scattering effects occur, viz., linear and nonlinear scattering. Rayleigh scattering is linear and occurs when the fluctuations in silica density are smaller than the wavelength of light. Stimulated Raman (SRS) and Brillouin scattering (SBS) are nonlinear scattering effects due to intensity-dependent refractive index, which occur only at high optical power levels in single-mode fibers. Nonlinear scattering alters the frequency of scattered light thus contributing to attenuation for light transmission. Thus, scattering effects induce loss in transmitted optical power.

We conclude this section stating that, dispersion introduces memory in the signal and limits the data rate of the signal, which is transmitted over the fiber. Fiber loss imposes a limitation on the transmission distance. Impairments induced by GVD and attenuation is linear since both are independent of light intensity. It is interesting to note that, though GVD and attenuation are linear in nature but they make the fiber channel nonlinear. PMD is time varying phenomenon and occurs in multichannel transmission using single-mode fibers over long distances. The optical point-to-point link in Gigabit and 10 Gbits/s Ethernet is relatively short (less than 80 km) in which case PMD can be ignored and the channel can be treated as time-invariant. The Kerr effects, SRS and SBS are nonlinear in nature since they are intensity dependent, occurring at higher power levels in WDM systems.

## ***2.2 Techniques to Compensate for Channel Impairment***

In the previous section, we have noticed that in an optical link SNR degradation is largely due to two effects— optical attenuation and dispersion. Regenerators mitigate attenuation, which carry out the optical-electrical-optical (OEO) domain conversions to enable long-haul transmission. Regenerator has two distinct disadvantages, viz., it is limited by the sensitivity of the receiver and it adds to the cost of the system. With the advent of Erbium-doped fiber amplifier (EDFA), the OEO conversions are avoided since amplification of the signal is done in the optical domain. Although EDFA allow the elimination of costly regenerators, they are not ideal and generate ASE noise. Dispersion compensating fiber (DCF) and optical Polarization Mode Dispersion (PMD) compensator can be used to compensate for dispersion



**Figure 2.4 Block Diagram of an Equalizer**

optically [18]. DCF allow fiber with various dispersion characteristics to be spliced together, reversing the effects of dispersion. Unfortunately, DCF causes much more attenuation than normal fiber and its use will require addition optical amplification. More EDFA results in more ASE in the link. Therefore compensating for one factor inevitably leads to increasing the effects of other. The solution is expensive and lacks flexibility [18]. Electronic compensation using *Digital Equalization* and high-speed analog techniques integrated in electronic circuits may be a better choice. The required SNR at the receiver for achieving the desired bit error probability is high which can be relaxed by employing *Forward Error Correction* (FEC), which is discussed in detail in chapter 3.

The ISI introduced by dispersion affects a finite number of symbols. To compensate for ISI introduced by the channel, the equalizer is a finite impulse response filter (FIR) whose frequency response is the inverse of the channel response (refer figure 2.4)

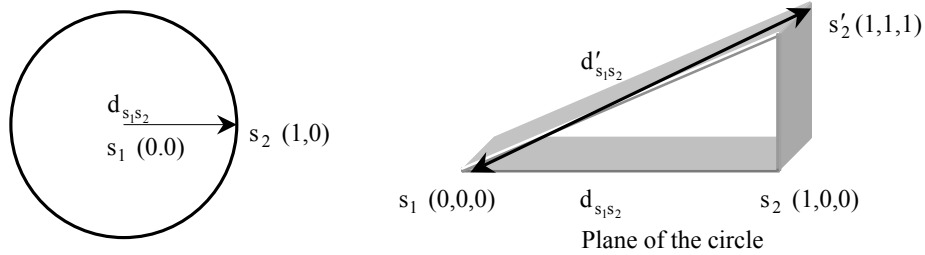
$$|G_E(f)| = \frac{1}{|C(f)|}; \quad |f| \leq W \quad 2.3$$

The optimum detector for the data stream  $x_k$  based on the observation of  $y_k$  is a *maximum-likelihood sequence detector* (MLSD) [9]. The computational complexity of the MLSD increases exponentially  $M^L$  with the span (L) of the ISI where, M is size of signal constellation.[9]. When M and L are large, the MLSD becomes impracticable. Suboptimum methods, viz., Linear and Nonlinear equalizers are discussed next. The frequency response of the channel is unknown, but time-invariant (PMD is ignored). If the channel is time-variant, (due to PMD) the equalizer coefficients have to be updated on a periodic basis during the transmission of data. Such equalizers are called *adaptive equalizers* [9]. In presence of noise, the noise variance at the output of the linear equalizer is higher than that at its input. The equalizer coefficients are estimated using a stochastic (random) gradient algorithm called as the *least mean square* (LMS) algorithm [9]. The severity of the ISI is directly related to the frequency response of the channel and not necessarily to the time span of the ISI. The linear equalizer introduces a large gain in its frequency response for spectral nulls in the channel response. This imposes a limitation on the performance of linear equalizers on channels having spectral nulls. A decision-feedback equalizer (DFE) is a nonlinear equalizer that uses previous decisions to eliminate the ISI caused by the preceding symbol on the current symbol to be detected [9]. It should be noted that even though the DFE outperforms a linear equalizer, the MLSD is the optimum [9]. We conclude this chapter stating that, in fiber-optic systems operating at high data rate of 10 Gbits/s and above the main challenge in the implementation of digital equalization techniques discussed above resides in the design of analog to digital converter [18]. Therefore, analog equalization can be a more practical alternative to digital equalization. The analog equalizer is a FFE, implemented using analog delay lines, digital programmable multipliers and LMS algorithm or eye monitoring techniques to adapt the filter taps[18]. For our analysis on FEC from chapter 3 onwards we model the optical channel as a memory less AWGN channel taking the different noise phenomenon into account (section 2.1.1) and neglecting dispersion or assuming that the equalizer compensates the effect of ISI.

# Chapter 3

## 3 Forward Error Correction

The Euclidean distance between the transmitted signals is increased through coding [9]. Consider two points  $s_1$  and  $s_2$  in a two dimension plane. The Euclidean distance between  $s_1$  and  $s_2$  is given by  $d_{s_1s_2} = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$ . In a two-dimensional plane, these two points can be viewed as  $s_1$  at the center of the circle and  $s_2$  on the circumference of the circle. The Euclidean distance is the radius of the circle. By moving the point  $s_1$  along the diameter of the circle in the opposite direction onto the circumference will increase the distance between  $s_1$  and  $s_2$  but this increase in distance is translated in terms of increasing the transmitter power. There is an inherent limitation on the transmitted power. The Euclidean distance between  $s_1$  and  $s_2$  can be increased by adding one more dimension and viewing the two points in three dimension.



From geometry it is evident that  $d'_{s_1s_2} > d_{s_1s_2}$  i.e.  $\sqrt{3} > 1$ . The error probability is a function of the distance between the points  $s_1$  and  $s_2$  [9], which are points in the BPSK signal constellation. The probability of bit-error for BPSK signaling is given by

$$P_b = Q\left(\sqrt{\frac{2E_b}{N_o}}\right) = Q\left(\frac{d_{s_1s_2}}{\sqrt{2N_o}}\right) \quad 3.1$$

where

$E_b$  : the energy per transmitted bit in Joules

$N_o$  : the noise power spectral density in watts/Hz

$\frac{E_b}{N_o}$  : the SNR per bit and  $Q(x) = \frac{1}{\sqrt{2\pi}} \int_x^\infty e^{-u^2/2} du$

The  $Q(x)$  function is a decreasing function of its argument  $x$  i.e. the bit-error probability decreases as the Euclidean distance increases. The reduction in error probability is not obtained for free but with an increase in bandwidth. Here, it is shown that the Euclidean distance is increased through adding one more dimension, which in essence is adding redundancy and the principle behind channel coding. In FEC, the algebraic structure of the code is used to determine which of the valid code word is most likely to have been transmitted, given the erroneous received word. We will discuss error detection and correction in detail in section 4.2. A scheme in which the decoder requests a retransmission of the code word upon detection of an error in the received word is referred to as *automatic repeat request* (ARQ). The focus of our discussion is on FEC so ARQ will not be discussed any further in the report.

There are two primary system parameters, viz., BER and SNR per bit that determine the performance of modern optical communication systems. Specifically, data is transmitted by a sequence of pulses, and the system must ensure that these pulses are received with a sufficiently low probability of error. Given a particular receiver (photo detector), a minimum received power is required for achieving a specified BER. An optical fiber introduces attenuation and dispersion in the system such that, attenuation tends to increase the transmitted power requirement to meet the desired SNR at the receiver, whereas dispersion imposes a limitation on the data transmission rate over the fiber. We state the advantages of FEC in section 3.1 introduce ECC in section 3.2 and give a justification for the use of block codes for error correction in fiber-optic systems in section 3.3.

### ***3.1 Advantages of Forward Error Correction***

The span of an optical link is determined by the optical power budget [1] (refer section 6.3). To create links with large spans, EDFA or repeaters are used which add to the noise floor of the system (refer sections 2.1.1 and 2.2). The span can be increased without the use of EDFA such that using high quality, high cost optical components, increases the transmitted power. This increases the overall system cost. With the use of FEC, following benefits can be achieved for a desired link span [46]

- ◆ Significant gain in the overall optical power budget is achieved.
- ◆ FEC implementation reduces the transmitted optical power requirement, thus the intensity dependent impairments (refer section 2.1.4) are reduced automatically.
- ◆ Relaxation on the high-end specification of the optical components reduces the cost.
- ◆ Correction of burst errors introduced by inter-channel cross talk in WDM systems.

OR for a specified power budget

- ◆ The power gain margin can be used to increase the span of the optical link, which accounts for less number of repeaters and amplifiers.
- ◆ Use of few repeaters and amplifiers reduces the overall noise floor and improves the SNR, which pays in terms of lower BER.
- ◆ In systems implementing ARQ, retransmission results in wastage of bandwidth, which can be avoided by implementing FEC.

It is quite natural there exist some inherent disadvantages for the advantages gained from a particular technology. FEC is no exception, since redundancy is introduced in the transmitted data stream. This imposes a requirement on increased signaling rate, which accounts for increase in bandwidth requirement. Also, at data rates of 10 Gbits/s and above the computational complexity and power consumption involved in implementing FEC plays an important role in system design [18]. There is a trade-off in power efficiency and spectral efficiency when implementing FEC.

### 3.2 Introduction to Error Correcting Codes

We open the discussion by introducing the types of error correcting codes available in communication theory. Error correcting codes are broadly classified in two categories, viz., *Block Codes* and *Convolutional Codes*. The encoder for block codes takes a message block of  $k$  information symbols represented by a  $k$ -tuple  $u = (u_1, u_2, \dots, u_k)$  and transforms each message  $u$  independently into an  $n$ -tuple  $v = (v_1, v_2, \dots, v_n)$  of discrete symbols called a *code word*, where  $(k < n)$ . There are a total of  $q^k$  different possible messages and accordingly the encoder generates  $q^k$  possible code words<sup>1</sup>. This set of  $q^k$  code words of length  $n$  is called a  $C(n, k)$  block code. The encoder for *convolutional codes* also accepts  $k$ -tuple of information symbols  $u$  and generates an  $n$ -tuple code word  $v$ , however the generated code word  $v$  at the time of encoding depends not only on the current  $k$  symbol message, but also on  $m$  previous message blocks. The fundamental difference between block codes and convolutional codes is that in block coding a finite length of output code word is generated for all input message words of finite length whereas input and output symbol sequences are infinite in convolutional coding. Another important aspect is the introduction of memory element in convolutional codes.

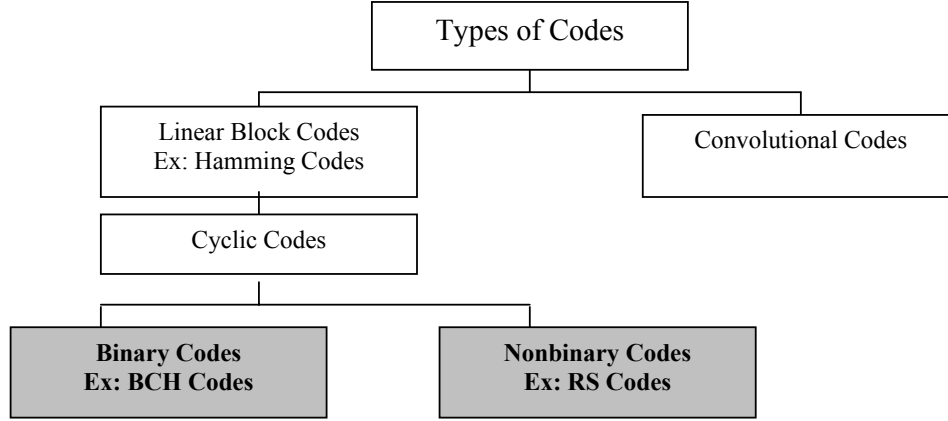


Figure 3.1 Classification of Error Correcting Codes

#### 3.2.1 Error Correcting Codes in Optical Communication Systems

FEC is widely used in wired and mobile communication, deep space communication as well as data storage systems. In the recent past, it has begun to find applications in optical links [45]. In this section, we outline the reasons for using block codes for error correction in optical systems. In optical communication systems that operate at very high data rate, the challenge is to find low overhead codes that are capable of correcting random errors due to noise and burst errors due to dispersion and inter-channel cross talk with special emphasis on complexity and cost. It is difficult to implement convolutional codec that operates at high code rates required for fiber-optic systems [36]. Algebraic block codes, such as **Bose-Chaudhuri-Hocqueaghem** (BCH) and **Reed-Solomon** (RS) codes (refer chapter 5) are capable of correcting multiple bit-errors with the low overhead constraint. As mentioned earlier the introduced redundancy of  $(n-k)$  symbols increases the bandwidth requirement. If  $T$  is the time duration required to transmit  $k$  symbols without coding, then  $T/k$  is the time required to transmit one symbol. After encoding the  $k$  symbols into a code word of  $n$  symbols, we transmit  $n$  symbols in time duration  $T$  and hence the symbol period is  $T/n$ , which is less than  $T/k$ . Thus, the width of each symbol after encoding is reduced by a factor  $k/n$  and the bandwidth required to transmit them is increased by a factor  $n/k$ , which is called the

<sup>1</sup> For the binary case  $q = 2$  while for nonbinary case  $q = p^m$  where  $p \geq 2$ . In the work presented in the report  $q = 2^m$

**Bandwidth expansion ratio.** The ratio  $k/n$  is called the *code rate*  $R_c$ . In case of fiber-optic communication systems operating at very high data rate ( $R_c > 0.8$ ), while selecting an error correcting code one should take into account the practical limitation imposed by the hardware to make it feasible to introduce an overhead of  $(n-k)$  symbols. Thus, low overhead constraint becomes an important parameter while selecting FEC for optical communication application. In our work, we have done a comparative analysis of the performance of different block codes (refer chapter 8) taking into account the low overhead requirement as a prime design criterion. In the next section, we will present elementary algebra, knowledge of which is necessary to understand the underlying principle of error control coding.

### 3.3 Galois Fields<sup>1</sup>

**Definition 3.3.1:** A *field*  $F$  is a set of elements in which we can perform addition, subtraction, multiplication, and division without leaving the set. A field with finite number of elements is called a *Finite field* or *Galois field*<sup>2</sup>. Addition and multiplication satisfy the commutative, associative and distributive property.

**Definition 3.3.2:** The *order of the field* is the number of elements in the field or in other words the order of the field is defined to be the cardinality of the field. Refer [12], for properties of fields. By definition, a field consisting of two elements  $\{0,1\}$  is called the binary field and denoted by GF (2). The zero-element is the additive identity and the unit element is the multiplicative identity. The properties stated above are shown below for the binary field

| + | 0 | 1 |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 1 | 0 |

Modulo-2 addition

| . | 0 | 1 |
|---|---|---|
| 0 | 0 | 0 |
| 1 | 0 | 1 |

Modulo-2 multiplication

**Definition 3.3.3:** When the field is constructed from a prime  $p$ , it is called a *prime field* and denoted by GF ( $p$ ) whereas, an *extension field* is formed from a power  $m$  of prime  $p$ , where  $m$  is a positive integer and denoted by GF ( $q = p^m$ ).

Galois field do not exist for any arbitrary number of elements, they exist only when the number of elements in the field is a prime or is a power of a prime number. Finite field arithmetic is very similar to ordinary arithmetic; techniques of algebra are used in the computations over finite fields. Construction of extension field is explained in section 3.3.3. Since GF ( $q$ ) =  $\{0, 1, 2, \dots, q-1\}$  has a finite number of elements, for any nonzero element  $\alpha$  of GF ( $q$ ), all the powers of  $\alpha$  cannot be distinct and at some point there is a repetition i.e. for  $m > k$ ,  $\alpha^k = \alpha^m$ .

**Definition 3.3.4:** The *order of field element*  $\alpha$  is defined as a smallest positive integer  $n$  such that  $\alpha^{m-k} = \alpha^n = 1$ . The sequence  $\alpha^1, \alpha^2, \alpha^3, \dots$  repeats itself after  $\alpha^n = 1$ .

**Definition 3.3.5:** A nonzero element  $\alpha$  is said to be *primitive* if the order of  $\alpha$  is  $q-1$ . The primitive element is also called the *generator* element. The  $(q-1)$  consecutive powers of a primitive element  $\alpha$  generate all the other nonzero elements of GF ( $q$ ). Consider the prime field GF (5) =  $\{0, 1, 2, 3, 4\}$ , 2 has order 4 and hence is a primitive element, i.e.  $2^1 = 2, 2^2 = 4, 2^3 = 3, 2^4 = 1$ . It should be noted that 1 to 4 consecutive powers of 2 results in the other nonzero elements of GF (5). For every finite field there exist *at least* one primitive element.

<sup>1</sup> The definitions in section 3.3 are reproduced from [12 Chapter 2] for understanding the mathematical background of error control codes

<sup>2</sup> The term Finite field and Galois field (GF) is used interchangeably throughout the report

In our example field GF (5), the element 3 is also a primitive element. In coding theory, the codes are constructed with elements from any Galois field GF (q), where q is either a prime p or a power of p. Since digital communication systems work with binary data, the codes are constructed with elements from binary field GF (2) or the extension field GF (2<sup>m</sup>). The BCH codes used in the report are constructed from elements of GF (2) and the nonbinary RS codes from the elements of GF (2<sup>8</sup>) = GF (256). The construction and properties of these codes are discussed in detail in chapter 5. We continue the remaining section with a few other definitions and properties of the Galois fields and vector spaces.

### 3.3.1 Vector Spaces<sup>1</sup>

$C$  is a set of elements called *vectors* and  $F$  is a field of elements called *scalars*, i.e. the field elements of GF (q). The set  $C$  forms a set of code words (vectors)<sup>2</sup> where each code word is constructed from elements (scalars) of GF (q). To make it clear we explicitly mention that, the code words form a set of elements called code vectors in a  $C(n, k)$  code. The “+” binary additive operation is referred as vector addition when two code vectors  $v$  and  $w$  are added. The “.” binary multiplicative operation is referred as scalar multiplication when a scalar  $\alpha \in F$  and a vector  $v \in C$  are multiplied. Our discussion on vector spaces is confined over GF (2) but is valid over any GF (q).

**Definition 3.3.6:**  $C$  forms a *vector space* over  $F$  if the following conditions are satisfied [7]

- ◆  $C$  forms a commutative group under addition
- ◆ For any element  $\alpha \in F$  and  $v \in C$ ,  $\alpha \cdot v = w \in C$
- ◆ For any two elements  $v$  and  $w$  in  $C$  and any two elements  $\alpha$  and  $\beta$  in  $F$ 

$$\alpha \cdot (v + w) = \alpha \cdot v + \alpha \cdot w \quad \text{and} \quad (\alpha + \beta) \cdot v = \alpha \cdot v + \beta \cdot v$$
- ◆ For any  $v$  in  $C$  and any  $\alpha$  and  $\beta$  in  $F$   $(\alpha \cdot \beta) \cdot v = \alpha \cdot (\beta \cdot v)$
- ◆ The multiplicative identity 1 in  $F$  acts as a multiplicative identity in scalar multiplication i.e. for any  $v \in C$   $1 \cdot v = v$

**Definition 3.3.7:** If a subset  $S$  of  $C$  is a vector space over  $F$ , then  $S$  is called a *subspace* of  $C$ . Let  $S$  be a nonempty subset of a vector space  $C$  over a field  $F$ , then  $S$  is a subspace of  $C$  if the following conditions are satisfied [7]

- ◆ For any vectors  $v$  and  $w$  in  $S$ ,  $v + w$  is also a vector in  $S$
- ◆ For any element  $\alpha$  in  $F$  and any vector  $v$  in  $S$ ,  $\alpha \cdot v$  is also in  $S$

A subspace  $S$  of  $C$  is formed by linear combination of any  $k$  elements of the vector space  $C$ . Thus a  $C(n, k)$  code with a set of  $2^k$  binary code words forms a subspace  $S$  of vector space  $C_n$  having  $2^n$  binary vectors. The order or cardinality of the  $C_n$  is  $2^n$ . Analogous, to the additive identity 0 of fields, the all-zero  $n$ -tuple  $\mathbf{0}$  is the additive identity in  $C_n$ .

**Definition 3.3.8:** If  $S$  is a  $k$  dimensional subspace of a vector space  $C$  then  $S^\perp$  forms a set of all vectors  $w$  in  $C$  such that for all  $v \in S$  and for all  $w \in S^\perp$ ,  $v \cdot w = 0$ .  $S^\perp$  is said to be the *dual space* of  $S$ . The vectors  $\{[0 \ 0 \ 0], [1 \ 0 \ 1], [0 \ 1 \ 1], [1 \ 1 \ 0]\}$  form a 2-dimensional subspace  $S$  of  $C_3$ . The dual space  $S^\perp$  of  $S$  comprises of vectors  $\{[0 \ 0 \ 0], [1 \ 1 \ 1]\}$  and has dimension 1.

**Definition 3.3.9:** A set of vectors  $P$ , the linear combination of which results in all the vectors in a vector space  $C$  is called a *spanning set* for  $C$ . The set  $P$  is said to span  $C$ . The set  $P = \{[0 \ 0 \ 1], [1 \ 0 \ 1], [0 \ 1 \ 1], [1 \ 1 \ 0]\}$  spans the vector space  $C_3$ . The elements of  $P$  are linearly dependent. Thus, a  $C(n, k)$  code is said to span the vector space  $C_n$ .

<sup>1</sup> The definitions in subsection 3.3.1 are reproduced from [7 Chapter 2] for understanding the mathematical background of error control codes

<sup>2</sup> The terms code words and code vectors are used interchangeably

**Definition 3.3.10:** A spanning set for  $C$  that has minimal order is called a *basis* for  $C$ .

By definition, the elements of a basis must be linearly independent. A vector space may have several possible bases, but all of the bases will have the same order. The generator matrix (refer section 4.1.2) forms the basis for a  $C(n, k)$  code.

**Definition 3.3.11:** A vector space  $C$  is said to have a *dimension*  $k$  if the basis  $B$  for the vector space  $C$  has  $k$  elements and is denoted by  $\dim(C) = k$ .

In our example,  $C_3$  has a dimension 3. It should be noted that  $\dim(S) + \dim(S^\perp) = \dim(C)$

### 3.3.2 Properties of Galois Field<sup>1</sup>

In this section, we will introduce polynomials whose coefficients are elements of  $GF(q)^2$ . The BCH and RS codes presented in chapter 8 are constructed using polynomial called *generator polynomial* whose coefficients are elements of  $GF(2)$  and  $GF(2^8)$  respectively. The polynomials over  $GF(q)$  satisfy the commutative, associative and distributive laws. Modulo-2 addition and multiplication govern the addition and multiplication operations between the polynomials over  $GF(2)$ . Subtraction is same as addition over  $GF(2)$ . Principles of algebra applied to carry computations on polynomials with real coefficients also apply to polynomials over  $GF(q)$ .

**Definition 3.3.12:** A polynomial  $f(X)$  of degree  $m$  is said to be *irreducible* iff  $f(X)$  is not divisible by any polynomial  $g(X)$  of degree less than  $m$  but greater than zero. Consider the polynomials of degree 2  $X^2, X^2 + 1, X^2 + X, X^2 + X + 1$ . Here  $f(X) = X^2 + X + 1$  is an irreducible polynomial since  $f(0) \neq 0$ ,  $f(1) \neq 0$  and it is not divisible by any polynomial of degree 1.

**Definition 3.3.13:** An *irreducible*<sup>3</sup> polynomial  $f(X)$  of degree  $m$  is said to be *primitive* if the smallest positive integer  $n$  for which  $f(X)$  divides  $X^n + 1$  is  $n = 2^m - 1$ .

It should be noted that

- ◆ Every *primitive*<sup>4</sup> polynomial  $p(X)$  is *irreducible*, but every other  $f(X)$  may not necessarily be *primitive*.
- ◆ If  $f(X)$  has a root  $\alpha$  in the extension field  $GF(2^m)$  and  $\alpha$  is a primitive element of  $GF(2^m)$  then  $f(X)$  is a primitive polynomial and denoted by  $p(X)$ .

$f(X)$  cannot be factored using elements from  $GF(2)$ , but otherwise will always have roots in the extension field  $GF(2^m)$ . Our example polynomial  $f(X) = X^2 + X + 1$  is irreducible over  $f(X)$   $GF(2)$  but not in  $GF(2^2 = 4) = \{0, 1, \beta, \beta^2\}$ .  $f(X)$  is primitive over  $GF(2)$  since  $n = 2^2 - 1 = 3$ , is the smallest positive integer, such that  $f(X)$  divides  $X^3 + 1$ .

$$\begin{array}{r}
 \phantom{X^2 + X + 1} \overline{X + 1} \\
 X^2 + X + 1 \overline{) X^3 \phantom{+ X^2 + X} + 1} \\
 \underline{+ X^3 + X^2 + X} \phantom{+ 1} \\
 \phantom{X^2 + X + 1} X^2 + X + 1 \\
 \underline{+ X^2 + X + 1} \\
 \phantom{X^2 + X + 1} 0
 \end{array}$$

<sup>1</sup> The definitions in subsection 3.3.2 are reproduced from [7 Chapter 2] for understanding the mathematical background of polynomial codes

<sup>2</sup>  $GF(q)$ : it is either the binary field  $GF(2)$  or the extension field  $GF(q = p^m)$  where  $p = 2$  and  $m > 1$

<sup>3</sup> An irreducible polynomial will be denoted by  $f(X)$

<sup>4</sup> A primitive polynomial will be denoted by  $p(X)$

Therefore,  $p(X)=f(X)$  is primitive over GF (2) but may not necessarily be irreducible over GF (4). Let  $\alpha$  be an element of GF (4) and root of  $f(X)$  such that  $f(\alpha)=\alpha^2+\alpha+1=0 \Rightarrow \alpha^2=\alpha+1$ . Thus, we can express the nonzero elements of GF (4) as  $\beta^2=\beta+1$ ,  $\beta^3=\beta^2+\beta$  and  $\beta^4=\beta^3+\beta^2=\beta$

Next, we will show that  $\beta^2$  is a root of  $f(X)$  and  $\beta^2$  is a primitive element of GF (4).

$$\begin{aligned} f(\beta^2) &= (\beta^2)^2 + \beta^2 + 1 \\ &= \beta^4 + \beta^2 + 1 \\ &= \beta + \beta + 1 + 1 = 0 \end{aligned}$$

The order of GF (4) is  $q=4$ , by definition,  $\beta^2$  will be primitive element of GF (4) if  $(\beta^2)^n = 1$ , such that  $n = q-1 = 3$ . Let us check this

$$\begin{aligned} (\beta^2)^3 &= \beta^6 \\ &= \beta^3 = \beta^2 + \beta \\ &= \beta + 1 + \beta = 1 \end{aligned}$$

Hence,  $\beta^2$  is a primitive element of GF (4) and since it is a root of  $f(X)=X^2+X+1$ ,  $f(X)$  is primitive over GF (2).

We state without proof that, the roots  $\alpha_i$  of an  $m^{\text{th}}$  degree primitive polynomial over GF (q) has order  $q^m-1$  [12]. We will show this holds for  $p(X)=X^2+X+1$ . It was shown earlier that  $\beta^2$  is a root of  $p(X)$  and has order 3. It is trivial to show that for  $q=2$ ,  $m=2$ , and hence the order of  $\beta^2$  is  $2^2-1=3$ . It could also be verified that  $\beta^2$  is a root of  $X^{2^{m-1}}+1$  since  $f(X)$  is a factor of  $X^{2^{m-1}}+1$ . The generator polynomial (refer section 5.2) for the BCH (127, 113) code considered in the report is constructed from the irreducible polynomial  $f(X)=X^7+X^3+1$ .

### 3.3.3 Construction of Extension Field<sup>1</sup>

Following the properties of Galois field, in this section we show the construction of the extension field GF ( $q=2^m$ ) in general and GF ( $q=2^4$ ) in particular. Given that  $\alpha$  is a root of an  $m^{\text{th}}$  degree primitive polynomial  $p(X)$  over GF (p) has order  $(p^m-1)$ . The  $(p^m-1)$  consecutive powers of  $\alpha$  form a *multiplicative group* of order  $(p^m-1)$ .

If  $p(X)=p_0+p_1X+\dots+p_{m-1}X^{m-1}+X^m$ , is the  $m^{\text{th}}$  degree primitive polynomial, then

$p(\alpha)=p_0+p_1\alpha+\dots+p_{m-1}\alpha^{m-1}+\alpha^m=0$ , where the  $p_i$  are elements of GF (p) for  $0 \leq i \leq m$ .

$$\therefore \alpha^m = -p_0 - p_1\alpha - \dots - p_{m-1}\alpha^{m-1}.$$

The powers of  $\alpha$  with degree greater than or equal to  $m$  can be expressed as polynomials in  $\alpha$  of degree  $(m-1)$  or less. Thus, there will be  $(p^m-1)$  distinct nonzero polynomials in  $\alpha$  of degree  $(m-1)$  of the form  $p_0+p_1\alpha+p_2\alpha^2+\dots+p_{m-1}\alpha^{m-1}$ . These  $(p^m-1)$  polynomials and zero form an additive group. It can be shown that the  $(p^m-1)$  consecutive powers of  $\alpha$  form the nonzero elements of the field GF ( $p^m$ ).

**Example 3.3.1:**  $p(X)=X^4+X+1$  is a primitive polynomial of degree 4. If  $\alpha$  is a primitive element in GF ( $q=2^4$ ) is a root of  $p(X)$ , then  $\alpha^4=\alpha+1$  and has order  $(2^4-1)=15$ . These 15 consecutive powers of  $\alpha$  form the nonzero elements of GF ( $2^4$ ) and are expressed in power (exponential) representation, polynomial representation as well as binary  $m$ -tuple format

<sup>1</sup> The definitions in subsection 3.3.3 are reproduced from [7 Chapter 2] for understanding the mathematical background of polynomial codes

below. The coefficients of the polynomial representation of the elements in GF ( $q = p^m$ ) are from the “base or ground field” GF ( $p$ ) i.e. from GF (2). The power representation with only the exponent and the binary m-tuple format of the field elements of GF ( $q = p^m$ ) for  $3 \leq m \leq 8$  is shown in Appendix A.

| Power (Exponential)<br>representation | Polynomial<br>representation         | Binary m-tuple<br>format |
|---------------------------------------|--------------------------------------|--------------------------|
| 0                                     | 0                                    | 0 0 0 0                  |
| $\alpha^0 = 1$                        | 1                                    | 1 0 0 0                  |
| $\alpha^1$                            | $\alpha^1$                           | 0 1 0 0                  |
| $\alpha^2$                            | $\alpha^2$                           | 0 0 0 1                  |
| $\alpha^3$                            | $\alpha^3$                           | 0 0 0 1                  |
| $\alpha^4$                            | $1 + \alpha^1$                       | 1 1 0 0                  |
| $\alpha^5$                            | $\alpha^1 + \alpha^2$                | 0 1 1 0                  |
| $\alpha^6$                            | $\alpha^2 + \alpha^3$                | 0 0 1 1                  |
| $\alpha^7$                            | $1 + \alpha^1 + \alpha^3$            | 1 1 0 1                  |
| $\alpha^8$                            | $1 + \alpha^2$                       | 1 0 1 0                  |
| $\alpha^9$                            | $\alpha^1 + \alpha^3$                | 0 1 0 1                  |
| $\alpha^{10}$                         | $1 + \alpha^1 + \alpha^2$            | 1 1 1 0                  |
| $\alpha^{11}$                         | $\alpha^1 + \alpha^2 + \alpha^3$     | 0 1 1 1                  |
| $\alpha^{12}$                         | $1 + \alpha^1 + \alpha^2 + \alpha^3$ | 1 1 1 1                  |
| $\alpha^{13}$                         | $1 + \alpha^2 + \alpha^3$            | 1 0 1 1                  |
| $\alpha^{14}$                         | $1 + \alpha^3$                       | 1 0 0 1                  |

**Definition 3.3.14:** If  $\alpha$  is an element in the extension field GF ( $q = p^m$ ), the *conjugates* of  $\alpha$  with respect to the base field GF ( $p$ ) are the elements  $\alpha^p, \alpha^{p^2}, \alpha^{p^3}, \dots$ . The conjugates of the field elements of our example field GF ( $q = 2^4$ ) are shown below

| Field Elements | Conjugates                                               |
|----------------|----------------------------------------------------------|
| $\alpha^0 = 1$ | 1                                                        |
| $\alpha^1$     | $\alpha^2, \alpha^4, \alpha^8$                           |
| $\alpha^2$     | $\alpha^4, \alpha^8, \alpha^{16} = \alpha$               |
| $\alpha^3$     | $\alpha^6, \alpha^{12}, \alpha^{24} = \alpha^9$          |
| $\alpha^4$     | $\alpha^8, \alpha^{16} = \alpha, \alpha^{32} = \alpha^2$ |
| $\alpha^5$     | $\alpha^{10}$                                            |

Similarly, conjugates of rest of the field elements can be obtained.

**Definition 3.3.15:** If  $\alpha$  is an element in GF ( $q = p^m$ ), the *minimal polynomial* of  $\alpha$  with respect to the base field GF ( $p$ ) is the smallest degree nonzero polynomial  $\phi(X)$  in GF ( $p$ ) such that

- ◆  $\phi(\alpha) = 0$
- ◆ The degree of  $\phi(X)$  is less than or equal to  $m$
- ◆  $\phi(X)$  is irreducible in GF (2)
- ◆ A field element  $\alpha$  and its conjugates  $\alpha^p, \alpha^{p^2}, \alpha^{p^3}, \dots$  have the same minimal polynomial  $\phi(X)$

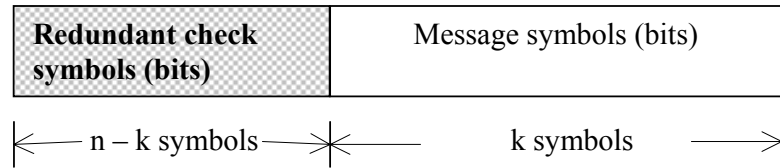
# Chapter 4

## 4 Linear Block Codes

A block code is *linear* if any linear combination of two code words is also a code word. If  $v$  and  $w$  are code words, then  $v \oplus w$  is also a code word, where  $\oplus$  denotes bit-wise modulo-2 addition. The necessary background concerning encoding and decoding is presented in this chapter. We will discuss systematic encoding in section 4.1, properties of block codes in section 4.1.1 and construction of linear block codes in terms of generator and parity-check matrices in section 4.1.2. Error detection and correction capability of block codes is presented in section 4.2. Decoding of block codes using standard array is discussed in section 4.3.

### 4.1 Systematic Encoding

At the receiver, the decoder has to recover the message block  $u$  of  $k$ -tuple from the code word  $v$  of  $n$ -tuple. If the structure of the code word is in the *systematic format* as shown in figure 4.1 the decoder does not have to perform any additional computations to recover the message block after decoding the received word  $r$  to the most likely transmitted code word  $v$ .



**Figure 4.1 Systematic Format of a Codeword**

A linear block code with this structure is referred to as a *linear systematic block code* such that the message part of the code word consists of the unaltered  $k$  message symbols and the redundant check symbols are linear sum of the information symbols. The code word could also have the systematic format with the  $k$  leftmost symbols as message symbols and  $n-k$  rightmost symbols as check symbols. Throughout the report, the transmitted code word is in the systematic format as shown in figure 4.1 unless stated explicitly. A block code of length  $n$  and  $2^k$  code words is called a linear  $C(n, k)$  code if and only if its  $2^k$  code words form a  $k$  dimensional *subspace* of the vector space of all the  $n$ -tuples over the field  $GF(2)$ . (Refer section 3.3 on Galois field and vector spaces)

#### 4.1.1 Properties of Block Codes

**Definition 4.1.1:** The *minimum distance* of a code is the minimum Hamming distance between any two different code words. Any two distinct code words of  $C(n, k)$  differ in at least  $d_{min}$  locations. The minimum Hamming distance  $d_{min}$  is a very important parameter when comparing the theoretical performance of different codes of same length  $n$  and dimension  $k$ .

**Definition 4.1.2:** The net electrical coding gain (NECG) is defined as the difference in the required SNR per information bit ( $E_b/N_0$ ) for coded and uncoded system to achieve a specified bit-error rate when operating over an ideal AWGN channel. It is expressed in (dB). This is another important parameter, which is used to compare the performance of different codes having comparable  $R_c$  from power budget point of view.

#### 4.1.2 Generator and Parity Check Matrices<sup>1</sup>

Each of the  $2^k$  code words in  $C(n, k)$  can be expressed as a linear combination of  $k$  linearly independent code words. The set of these  $k$  *linearly independent* code words form a *basis* of order  $k$ , which generate (or span) the  $2^k$  code words in  $C(n, k)$ , which, is a subset (or subspace) of the vector space of  $2^n$  vectors. Since the  $k$  linearly independent code words generate the  $C(n, k)$  code, these can be arranged as  $k$  rows of a matrix called the *generator matrix*  $\mathbf{G}$  for  $C(n, k)$  [7]. Let the  $k$  linearly independent code words be denoted by  $\mathbf{g}_1, \mathbf{g}_2, \mathbf{g}_3, \dots, \mathbf{g}_k$ . Using the notations introduced in section 3.2, a  $k$ -tuple message  $\mathbf{u}$  is encoded in an  $n$ -tuple code word  $\mathbf{v}$  by the dot product between  $\mathbf{u}$  and  $\mathbf{G}$ .

$$\mathbf{v} = \mathbf{u} \cdot \mathbf{G} \quad 4.1$$

$$\mathbf{v} = v_1 + v_2 + \dots + v_n$$

$$\mathbf{u} = u_1 + u_2 + \dots + u_k$$

$$\mathbf{G} = [\mathbf{g}_1 \ \mathbf{g}_2 \ \dots \ \mathbf{g}_k]'$$

The generator matrix  $\mathbf{G}$  is

$$\mathbf{G}_{(k,n)} = \begin{bmatrix} g_{1_1} & g_{1_2} & g_{1_3} & \dots & \dots & g_{1_n} \\ g_{2_1} & g_{2_2} & g_{2_3} & \dots & \dots & g_{2_n} \\ \dots & \dots & \dots & \dots & \dots & \dots \\ g_{k_1} & g_{k_2} & g_{k_3} & \dots & \dots & g_{k_n} \end{bmatrix} \quad 4.2$$

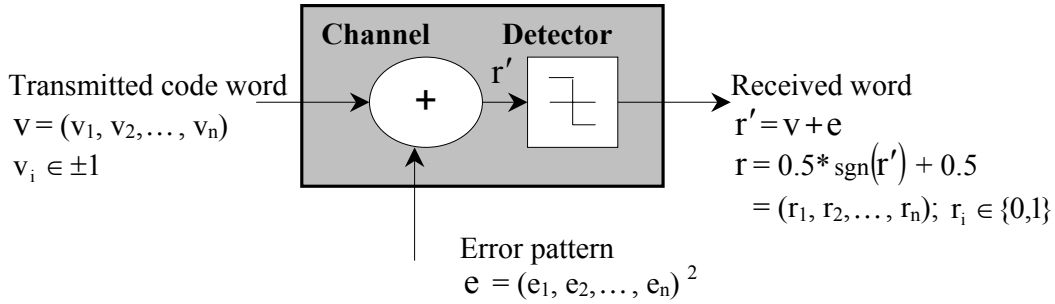
Thus, the  $C(n, k)$  linear code in the systematic format is completely specified by the  $k$  rows of a generator matrix  $\mathbf{G}$  of the form  $\mathbf{G} = [\mathbf{P} \ \mathbf{I}_{k \times k}]$  where  $\mathbf{I}$  is an  $(k \times k)$  identity matrix and  $\mathbf{P}$  is a  $(k \times n-k)$  parity matrix. For any  $(k \times n)$  matrix  $\mathbf{G}$  with  $k$  linearly independent rows, there exists a  $((n-k) \times n)$  matrix  $\mathbf{H}$  with  $n-k$  linearly independent rows such that any row vector of  $\mathbf{G}$  is orthogonal to the row vectors of  $\mathbf{H}$ . In addition, any vector that is orthogonal to the row vectors of  $\mathbf{H}$  is in the  $k$  rows of  $\mathbf{G}$ . i.e.  $\mathbf{G} \cdot \mathbf{H}^T = \mathbf{0}$ . Thus, alternatively it can be stated that an  $n$ -tuple  $\mathbf{v}$  is a code word in the code  $C(n, k)$  generated by  $\mathbf{G}$  if and only if  $\mathbf{v} \cdot \mathbf{H}^T = \mathbf{0}$ . The matrix  $\mathbf{H}$  is called the *parity-check matrix* of the code  $C(n, k)$  [7]. The  $2^{n-k}$  linear combinations of the rows of  $\mathbf{H}$  form a  $(n, n-k)$  linear code  $C^\perp$  that is a dual of the  $C(n, k)$  code. The parity-check matrix  $\mathbf{H}$  of  $C(n, k)$  is the generator matrix for the dual  $C^\perp(n, n-k)$  code. Given  $\mathbf{G}$  in the systematic form  $\mathbf{G} = [\mathbf{P} \ \mathbf{I}_k]$  for a  $C(n, k)$  code the parity-check matrix takes the form  $\mathbf{H} = [\mathbf{I}_{n-k} \ \mathbf{P}^T]$ . We list the form of  $\mathbf{G}$  and  $\mathbf{H}$  for the (7, 4) and (15, 11) codes.

| $C(n, k)$ Code | Generator Matrix                                                                       | Parity-check Matrix                                                                   |
|----------------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| $C(7, 4)$      | $\mathbf{G}_{(4 \times 7)} = [\mathbf{P}_{4 \times 3} \ \mathbf{I}_{4 \times 4}]$      | $\mathbf{H}_{(3 \times 7)} = [\mathbf{I}_{3 \times 3} \ \mathbf{P}_{4 \times 3}^T]$   |
| $C(15, 11)$    | $\mathbf{G}_{(11 \times 15)} = [\mathbf{P}_{11 \times 4} \ \mathbf{I}_{11 \times 11}]$ | $\mathbf{H}_{(4 \times 15)} = [\mathbf{I}_{4 \times 4} \ \mathbf{P}_{11 \times 4}^T]$ |

<sup>1</sup> The definition of generator matrix and parity-check matrix is reproduced from [7 chapter 4]

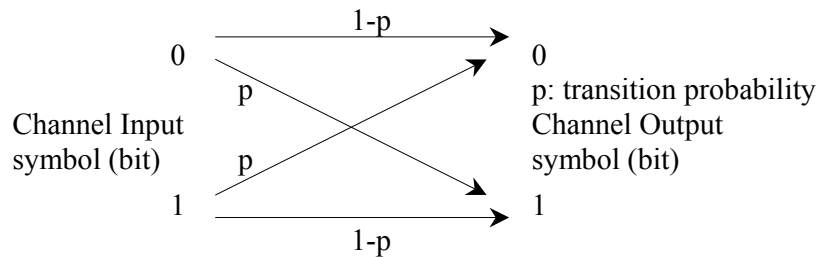
## 4.2 Error Detection and Correction

Given the parity check matrix  $\mathbf{H}$ , it is possible to check whether the received word  $\mathbf{r}$  is a valid code word or not. Consider the AWGN channel model shown below.



**Figure 4.2 Additive White Gaussian Noise Channel<sup>3</sup>**

The decoder computes  $\mathbf{s} = \mathbf{r} \cdot \mathbf{H}^T$  where  $\mathbf{s}$  is a  $(n-k)$  tuple and is called the *syndrome* of  $\mathbf{r}$ . The decoder declares absence of error event if the syndrome  $\mathbf{s} = \mathbf{0}$  and accepts  $\mathbf{r}$  as a valid transmitted code word  $\mathbf{v}$ . The only lazy action the decoder has to take in such a scenario is to extract the rightmost  $k$  symbols of the code word  $\mathbf{v}$  and deliver it to the sink as transmitted message  $\mathbf{u}$ . On the other hand, if  $\mathbf{s} \neq \mathbf{0}$ , the decoder declares an error event and in such a case the decoder needs to stimulate its gray cells and perform some smart computations to locate the errors and correct them. There is a possibility that even if  $\mathbf{s} = \mathbf{0}$ , the received word  $\mathbf{r}$  may not be a valid transmitted code word  $\mathbf{v}$  and the decoder is fooled by the error pattern  $\mathbf{e}$ . In such a situation the error pattern  $\mathbf{e}$  is identical to a non zero code word and due to the inherent linear nature of the code the transmitted code word  $\mathbf{v}$  gets converted into another code word  $\mathbf{w}$  of  $C(n, k)$ . Error patterns of this type are called *undetectable error patterns*. One important fact to be noted here is that the syndrome  $\mathbf{s}$  of  $\mathbf{r}$  completely depends on the error pattern  $\mathbf{e}$  and not on the transmitted code word  $\mathbf{v}$ . The *binary symmetric channel* (BSC) model when the detector is included in the AWGN model is depicted in figure 4.3. In the AWGN and BSC, the probability that a transmitted bit will be received incorrectly is independent of the value of the bit.



**Figure 4.3 Binary Symmetric Channel (BSC)**

<sup>2</sup>  $e_i \in \mathbb{R}$  is real when channel is AWGN

<sup>3</sup> The inclusion of the detector in the model converts the AWGN to BSC

### 4.2.1 Error Detection<sup>1</sup>

Referring to figure 4.2 assume an error pattern of  $(l \leq n)$  errors will cause the received word  $\mathbf{r}$  to differ from the transmitted code word  $\mathbf{v}$  in  $l$  places i.e.  $[d(\mathbf{v}, \mathbf{r}) = l]$ . The detector observes the received word  $\mathbf{r}$  and declares an error event if  $s \neq \mathbf{0}$ . This process is called *error detection*. If the minimum distance of the block code is  $d_{min}$ , then an error pattern of  $(l \leq d_{min} - 1)$  errors will for sure result in a received word  $\mathbf{r}$  that is not a code word. Hence, a block code with minimum distance  $d_{min}$  is capable of detecting all the error patterns of  $d_{min} - 1$  or fewer errors. An error pattern of  $d_{min}$  errors is undetectable because there exists at least one pair of code words that differ in  $d_{min}$  locations, so it causes the received word  $\mathbf{r}$  to be another valid code word other than that was transmitted. The same holds true for error patterns of more than  $d_{min}$  errors. Thus, a block code with minimum distance  $d_{min}$  *guarantees* detecting all the error patterns of  $d_{min} - 1$  or fewer errors and is capable of detecting a large fraction of error patterns with  $d_{min}$  or more errors [7].

There are  $2^k - 1$  error patterns, which alters the transmitted code word  $\mathbf{v}$  into another code word  $\mathbf{w}$ . These  $2^k - 1$  error patterns are *undetectable* and the decoder accepts  $\mathbf{w}$  as the transmitted code word. The decoder is then said to have committed a *decoder error*. However, there are  $2^n - 2^k$  detectable error patterns. For large  $n$ ,  $2^k - 1$  is much smaller and only a small fraction of error patterns pass through the decoder being undetected. The error detection performance of a block codes is discussed in detail in chapter 6.

### 4.2.2 Error Correction

With the assumption that all code words are equally likely to be transmitted, the best decision rule at the receiver would be always to decode a received word  $\mathbf{r}$  into a transmitted code word  $\mathbf{v}$  that differs from the received word  $\mathbf{r}$  in the fewest positions (components or bits). This decision criterion is called *maximum-likelihood (ML) decoding*, (refer section 1.3). This is equivalent to minimizing the Hamming distance between  $\mathbf{r}$  and  $\mathbf{v}$ . Decoder based on this principle is called *minimum distance decoder* [10].

For a  $C(n, k)$  block code with minimum distance  $d_{min}$  the random error correcting capability can be determined as follows

$$2t + 1 \leq d_{min} \leq 2t + 2; t \text{ is a positive integer} \quad 4.3$$

It can be shown that the block code  $C$  is capable of correcting all the error patterns of  $t$  or fewer errors. Let  $\mathbf{v}$  and  $\mathbf{r}$  be the transmitted code word and received word respectively and  $\mathbf{w}$  be any other valid code word in  $C$ . The Hamming distance between  $\mathbf{v}$ ,  $\mathbf{r}$  and  $\mathbf{w}$  satisfy the triangle inequality:

$$\begin{aligned} d(\mathbf{v}, \mathbf{r}) + d(\mathbf{w}, \mathbf{r}) &\geq d(\mathbf{v}, \mathbf{w}) \\ d(\mathbf{w}, \mathbf{r}) &\geq 2t + 1 - t'; \quad \text{where } d(\mathbf{v}, \mathbf{r}) = t' \text{ and } t' \leq t \\ d(\mathbf{w}, \mathbf{r}) &> t \end{aligned} \quad 4.4$$

Consider the  $C(7, 4)$  code with  $d_{min} = 3$

From (4.3) we have  $t = (d_{min} - 1) / 2 = 1$  since  $d_{min}$  is odd and let

$$\begin{aligned} \mathbf{v} &= [\mathbf{1} \ 1 \ 0 \ 1 \ 0 \ 0 \ 0]^2 \\ \mathbf{r} &= [0 \ 1 \ 0 \ 1 \ 0 \ 0 \ 0] \\ \mathbf{w} &= [\mathbf{1} \ 1 \ 1 \ 0 \ 0 \ 1 \ 0] \end{aligned}$$

<sup>1</sup> The theoretical description in sub-sections 4.2.1-4.2.2 is reproduced from [7 Chapter 3]

<sup>2</sup> Components in bold differ from those in  $\mathbf{r}$

$d(v, r) = 1$  and  $d(w, r) = 4$ . From (4.4) we conclude that if an error pattern of  $t$  or fewer errors occurs, the received word  $r$  is closer to the transmitted code word  $v$  than to any other code word  $w$  in  $C$  in the Hamming distance sense. For all error patterns with  $l$  errors such that  $l > t$ , there exists at least one case where the received word  $r$  is closer to an incorrect code word  $w$  than the transmitted code word  $v$ , such that  $d(v, w) = d_{min}$  and the following conditions are satisfied [7]

- ◆  $e_1 + e_2 = v + w$
- ◆  $e_1$  and  $e_2$  do not have nonzero components in common places.

Consider the  $C(7,4)$  code with  $d_{min} = 3$

$$\begin{aligned} v &= [1\ 1\ 0\ 1\ 0\ 0\ 0] \\ e_1 &= [0\ 0\ 1\ 1\ 0\ 0\ 0] \text{ and } e_2 = [0\ 0\ 0\ 0\ 0\ 1\ 0] \\ r &= v + e_1 = [1\ 1\ 1\ 0\ 0\ 0\ 0] \\ w &= [1\ 1\ 1\ 0\ 0\ 1\ 0] \end{aligned}$$

$d(v, r) = 2$  and  $d(w, r) = 1$ . In this case, according to the (ML) decoding criterion, the decoder will select  $w$  as the transmitted code word instead of  $v$  and *decoder error* occurs. We conclude stating that, a block code with minimum distance  $d_{min}$  *guarantees* correcting all the error patterns of  $t = \lfloor (d_{min} - 1) / 2 \rfloor$  or fewer errors. The parameter  $t$  is called the random error correcting capability of the code. A  $t$ -error correcting linear block code  $C(n, k)$  is capable of correcting a total of  $2^{n-k}$  error patterns, including those with  $t$  or fewer errors.

### 4.3 Standard Array Decoding

With the knowledge of occurrence of an error event, the decoder is entrusted the task of determining the true error pattern  $e$ . Using the distributive property, we can write

$$s = r.H^T = (v + e).H^T = v.H^T + e.H^T \quad 4.5$$

The solution to the  $n-k$  linear equations of (4.5) have  $2^k$  solutions [7] and the true error pattern  $e$  is one of the  $2^k$  error patterns. If the channel is a BSC as shown in figure 4.3 the most probable error pattern has the smallest number of nonzero components and is chosen as the true error pattern in order to minimize the probability of a decoding error [7]. The received word  $r$  belongs to the vector space of  $2^n$   $n$ -tuples over GF(2). The  $2^n$   $n$ -tuples are partitioned into  $2^k$  disjoint subsets  $D_1, D_2, \dots, D_{2^k}$  such that  $v_i$  is contained in the subset  $D_i$  for  $1 \leq i \leq 2^k$ . The *standard array* is an array of rows called *cosets* and columns (sub sets) such that each of the  $2^k$  disjoint subsets contains one and only one code word [7]. If  $v$  is a transmitted code word then the received word  $r$  will fall in  $D_i$  for  $1 \leq i \leq 2^k$  if the error pattern is a coset leader. In such as case  $r$  will be decoded correctly into the transmitted code word  $v$ . However, if the error pattern is not a coset leader, an erroneous decoding will result. The  $2^{n-k}$  coset leaders including the all zero word are called *correctable error patterns*. The major drawback of the standard array decoding is that the array grows exponentially with  $k$  and becomes impractical for large  $k$ . The  $2^n$  entries can be reduced to  $2 * 2^{n-k}$  entries in a look up table using *syndrome decoding*. The syndrome  $s$  is a  $(n-k)$  tuple and there are  $2^{n-k}$  distinct syndromes. There exist a direct mapping between the  $2^{n-k}$  syndromes and the  $2^{n-k}$  coset leaders and hence is stored in the look-up table. Calculating the syndrome of the received word  $r$  and determining the coset leader  $e_i$  for  $1 \leq i \leq 2^{n-k}$ , having the same syndrome accomplish the decoding. The transmitted code word  $v = r + e_i$ . For large  $n-k$ , the implementation becomes impractical. Other than the linear structure, practical algebraic decoding schemes require additional properties in a code, which are discussed in section 5.4.

## 4.4 Types of Decoders<sup>1</sup>

**Complete decoder:** Given a received word  $r$  the decoder selects the transmitted code word  $v$  that minimizes  $d(v, r)$  according to the (ML) decoding criterion. The complete decoder experiences *decoder error* condition when it encounters an undetectable error pattern (refer section 4.2.1).

**Bounded-Distance decoder:** Given a received word  $r$  the decoder selects the transmitted code word  $v$  that minimizes  $d(v, r)$ , if and only if there exists  $v$  such that  $d(v, r) \leq t$ . If no such  $v$  exists, then a *decoder failure* is declared i.e. error pattern with  $l > t$  has occurred.

An interesting fact to be noted here is with  $l > t$  errors where a bounded-distance decoder will declare a decoder failure whereas a complete decoder will select an incorrect code word  $w$  if the received word is closer to  $w$  in Hamming distance than to the valid transmitted code word  $v$  thus resulting in decoder error condition. However, in many cases, a complete decoder is capable of correcting an error pattern of  $l > t$  errors. The theoretical performance and computer simulations presented in sections 6.1–6.2 with respect to the BCH and RS codes are based on *Bounded-distance* decoder.

## 4.5 Weight Distribution of a Block Code

The *Hamming weight* of a codeword  $v_i$  is the number of nonzero components of the code word and is denoted by  $w(v_i)$ . If  $A_i$  is the number of code words of weight  $i$  in a  $C(n, k)$  code, then  $A_0, A_1, \dots, A_n$  is called the weight distribution of  $C(n, k)$ . The weight distribution is expressed in polynomial form called the *weight enumerating function* (WEF) and is expressed as  $A(X) = A_0 + A_1X + A_2X^2 + \dots + A_nX^n$ . Consider the  $C(7, 4)$  code

| Message | Code words    | Weights | Message | Code words    | Weights |
|---------|---------------|---------|---------|---------------|---------|
| 0 0 0 0 | 0 0 0 0 0 0 0 | 0       | 0 0 0 1 | 1 0 1 0 0 0 1 | 3       |
| 1 0 0 0 | 1 1 0 1 0 0 0 | 3       | 1 0 0 1 | 0 1 1 1 0 0 1 | 4       |
| 0 1 0 0 | 0 1 1 0 1 0 0 | 3       | 0 1 0 1 | 1 1 0 0 1 0 1 | 4       |
| 1 1 0 0 | 1 0 1 1 1 0 0 | 4       | 1 1 0 1 | 0 0 0 1 1 0 1 | 3       |
| 0 0 1 0 | 1 1 1 0 0 1 0 | 4       | 0 0 1 1 | 0 1 0 0 0 1 1 | 3       |
| 1 0 1 0 | 0 0 1 1 0 1 0 | 3       | 1 0 1 1 | 1 0 0 1 0 1 1 | 4       |
| 0 1 1 0 | 1 0 0 0 1 1 0 | 3       | 0 1 1 1 | 0 0 1 0 1 1 1 | 4       |
| 1 1 1 0 | 0 1 0 1 1 1 0 | 4       | 1 1 1 1 | 1 1 1 1 1 1 1 | 7       |

$A(X) = 1 + 7X^3 + 7X^4 + X^7$  is the WEF of  $C(7, 4)$  code.. If the minimum distance of  $C(n, k)$  is  $d_{min}$ , then  $A_i$  to  $A_{dmin-1}$  are zeros. For certain binary BCH codes, according to the MacWilliams Identity [8] if  $A(X)$  and  $B(X)$  are the WEF's for an  $C(n, k)$  code and the dual  $C^\perp(n, n-k)$  code respectively, then  $A(X)$  and  $B(X)$  are related as

$$B(X) = 2^{-k} (1+X)^n A \left[ \frac{(1-X)}{(1+X)} \right] \quad 4.6$$

<sup>1</sup> The definitions in section 4.4 are reproduced from [12 Chapter 4]

# Chapter 5

## 5 BCH and Reed-Solomon Codes

The BCH codes are binary and form a class of multiple random error correcting cyclic codes. On the other hand, the RS codes are nonbinary cyclic codes with code word symbols from  $GF(q^m)$  and are the most powerful block codes having the capability of correcting random as well as burst errors. Since both the BCH and RS codes are cyclic in nature they can be implemented using high-speed shift-register based encoders/decoders. This property of the BCH and RS codes has enabled them to find its way in optical communication systems.

### 5.1 Linear Cyclic Codes<sup>1</sup>

Consider an  $n$ -tuple  $V_i = [v_0, v_1, \dots, v_{n-1}]$  of  $C(n, k)$  linear code, if the symbols of  $V_i$  are cyclically shifted one place to the right we obtain another  $n$ -tuple  $V_j = [v_{n-1}, v_0, v_1, \dots, v_{n-2}]$ . If  $V_j$  is also a code word in  $C$  then  $C(n, k)$  is called a *cyclic linear code*. The symbols can be cyclically right-shifted or left-shifted.

**Definition 5.1:** A  $C(n, k)$  linear code is said to be a *cyclic code* if every cyclic shift of a code word is also a code word in  $C$ . Apart from being linear, the cyclic codes possess interesting algebraic properties, which are explored subsequently. To explore the algebraic properties, the code word is expressed as a polynomial whose coefficients are the symbols of the code word. Thus, an  $n$ -tuple code word  $V_i = [v_0, v_1, \dots, v_{n-1}]$  in polynomial form is expressed as

$$V(X) = v_0 + v_1X + v_2X^2 + \dots + v_{n-1}X^{n-1} \quad 5.1$$

$V(X)$  is called the *code polynomial*<sup>2</sup>.

We state a few properties of cyclic codes without proof. They are proved in [7], [12].

Property I: There exists a *unique* nonzero code polynomial  $g(X)$  of minimum degree ( $r < n$ ) within the set of code polynomials in  $C$ , and is *monic*.

Property II: It follows from property I, the constant term in the unique nonzero code polynomial  $g(X)$  of minimum degree must be 1.

Property III: Every  $v(X)$  in  $C$  is a multiple of  $g(X)$  and can be expressed uniquely as  $v(X) = u(X)g(X)$  where  $u(X)$  is a polynomial of degree less than  $(n-r)$ . Hence,  $g(X)$  is called the *generator polynomial* of  $C$ .

Property IV: If there exists a polynomial  $g(X)$  of degree  $n-k$  and is a factor of  $X^n + 1$ , then  $g(X)$  generates an  $(n, k)$  cyclic code

---

<sup>1</sup> The properties in section 5.1 are reproduced from [7 chapter 4] for subsequent explanation in succeeding sections.

<sup>2</sup> The terms code word and code polynomial are used interchangeably.

Property V: If  $g(X)$  is the generator polynomial of a  $C(n, k)$  code, the dual  $(n, n-k)$  is also a linear cyclic code  $C^\perp$  and is generated by the polynomial  $X^k h(X^{-1})$ , where  $h(X) = \frac{X^n + 1}{g(X)}$

Encoding of a  $k$ -tuple message word  $u = (u_0, u_1, \dots, u_{k-1})$  into a code word  $v$  is accomplished using Property III, such that,  $v(X) = u(X)g(X)$ , where  $u(X)$  is the message polynomial of degree  $(k-1)$ . The resulting code word  $v$  is not in systematic format. The encoding in systematic format (refer section 4.1) can be achieved through the procedure mentioned below

- ◆ Multiply the message polynomial  $u(X)$  by  $X^{n-k}$ .
- ◆ Divide  $X^{n-k}u(X)$  by  $g(X)$  to obtain the remainder polynomial  $b(X)$ .
- ◆ Adding  $X^{n-k}u(X)$  to the remainder polynomial  $b(X)$  forms the code polynomial  $v(X)$  in systematic format as shown in figure 4.1, such that,  $v(X) = b(X) + X^{n-k}u(X)$

The  $C(7, 3)$  code in the systematic form with  $g(X) = (X^4 + X^2 + X + 1)$  is shown below

| Message Word $u$ | Message Poly. $u(X)$ | Code Word $v$ | Code Poly. $u(X)$       |
|------------------|----------------------|---------------|-------------------------|
| 0 0 0            | 0                    | 0 0 0 0 0 0   | 0                       |
| 1 0 0            | 1                    | 1 1 1 0 1 0 0 | $1 + X + X^2 + X^4$     |
| 0 1 0            | $X$                  | 0 1 1 1 0 1 0 | $X + X^2 + X^3 + X^5$   |
| 1 1 0            | $1 + X$              | 1 0 0 1 1 1 0 | $1 + X^3 + X^4 + X^5$   |
| 0 0 1            | $X^2$                | 1 1 0 1 0 0 1 | $1 + X + X^3 + X^6$     |
| 1 0 1            | $1 + X^2$            | 0 0 1 1 1 0 1 | $X^2 + X^3 + X^4 + X^6$ |
| 0 1 1            | $X + X^2$            | 1 0 1 0 0 1 1 | $1 + X^2 + X^5 + X^6$   |
| 1 1 1            | $1 + X + X^2$        | 0 1 0 0 1 1 1 | $X + X^4 + X^5 + X^6$   |

The received polynomial  $r(X)$  when divided by  $g(X)$  results in a remainder polynomial  $s(X)$  called the *syndrome* polynomial. If  $s(X) = 0$  then from Property III, it follows that  $r(X)$  is a valid code polynomial and hence there are no errors. On the other hand if  $s(X) \neq 0$  then  $r(X)$  is not a transmitted code word and error event is detected. In short, linear cyclic codes are decoded by computing the syndromes of the received polynomial. Decoding of cyclic codes is treated in detail in sections 5.4-5.5 where we discuss the algebraic and transform decoding techniques for the BCH and RS codes. We conclude our discussion on the general class of linear cyclic codes here and introduce the BCH and RS codes in the next section.

## 5.2 Description of BCH and RS Codes

**Definition 5.2.1:** The BCH code is defined below with usual notations [7]

Block Length:  $n = p^m - 1$   
 Number of parity check bits:  $n - k \leq mt$ ,  
 Where  $m$  is power of  $p$  GF ( $q = p^m$ ) in which  $g(X)$  has roots  
 Minimum distance:  $d_{min} \geq 2t + 1$

The  $n$ -tuple code word  $v = (v_0, v_1, \dots, v_{n-1})$  has symbols  $v_0, v_1, \dots, v_{n-1}$  from GF ( $p$ )

**Definition 5.2.2:** The RS code is defined below with usual notations [7]

Block Length:  $n = p^m - 1$   
 Number of parity check bits:  $n - k = 2t$ ,  
 Where  $m$  is power of  $p$  GF ( $q = p^m$ ) in which  $g(X)$  has roots  
 Minimum distance:  $d_{min} = 2t + 1$

The  $n$ -tuple code word  $v = (v_0, v_1, \dots, v_{n-1})$  has symbols  $v_0, v_1, \dots, v_{n-1}$  from GF ( $q = p^m$ )

### 5.2.1 Time Domain Description using the Generator Polynomial

Given an arbitrary generator polynomial  $g(X)$  to construct a cyclic code, the minimum distance of the code cannot be determined analytically but through a comprehensive search of the weights of the nonzero code words with the aid of computer. Fortunately, the minimum distance of a BCH code is known through the *BCH bound* [10]. The generator polynomial  $g(X)$  of a binary BCH code of length  $n = 2^m - 1$  is the minimum degree polynomial over  $GF(2)$ , which has roots in the extension field  $GF(2^m)$ . If  $\alpha$  is a primitive element in  $GF(2^m)$ , then the BCH bound states that if  $g(X)$  has the  $2t$  consecutive powers of  $\alpha$ , i.e.  $\alpha^{b+1}, \alpha^{b+2}, \alpha^{b+3}, \dots, \alpha^{b+2t}$  as the roots in  $GF(2^m)$  i.e.  $g(\alpha^{b+i}) = 0$  for  $1 \leq i \leq 2t$ , then the code generated by  $g(X)$  has minimum distance *at least* equal to  $2t+1$ . Therefore,  $d_o = 2t+1$  is guaranteed and referred to as “*design distance*” with the constraint on  $g(X)$ . Thus, the BCH bound imposes a lower bound on the minimum distance of a BCH code. However, in many cases the actual minimum distance exceeds the design distance. As mentioned earlier the minimum distance of a linear code is equal to the smallest number of columns of  $\mathbf{H}$  that sum to 0. In the proof of BCH bound [10], [12] it is shown that the constraint imposed on the  $g(X)$  ensures that no  $2t$  or fewer columns of the parity-check matrix  $\mathbf{H}$  sum to zero. The BCH codes discussed in the report are *primitive, narrow-sense* since  $\alpha$  is a primitive element in  $GF(2^m)$  and  $b=1$ . The conjugates of  $\alpha$  are also roots of  $g(X)$ . The generator polynomial is defined as the *least common multiple* of the minimal polynomials of the  $2t$  consecutive powers of  $\alpha$ .

$$g(X) = \text{LCM}[\phi_1(X), \phi_2(X), \phi_3(X), \dots, \phi_{2t}(X)] \quad 5.2$$

where  $\phi_1(X), \phi_2(X), \phi_3(X), \dots, \phi_{2t}(X)$  are the minimal polynomials of  $\alpha^i$  for  $1 \leq i \leq 2t$ .

If  $i$  is an even integer, then  $i = i'2^l$  where  $i'$  is odd number and  $l \geq 1$  then  $\alpha^i = (\alpha^{i'})^{2^l}$  is a conjugate of  $\alpha^{i'}$  (ex.  $\alpha^6 = (\alpha^3)^2$  so  $\alpha^6$  is a conjugate of  $\alpha^3$ ).  $\alpha^i$  and  $\alpha^{i'}$  have the same minimal polynomial  $\phi(X)$  and as a result  $g(X)$  reduces to

$$g(X) = \text{LCM}[\phi_1(X), \phi_3(X), \dots, \phi_{2t-1}(X)] \quad 5.3$$

Once  $g(X)$  is determined, Property III stated in section 5.1 is used to generate the code words  $v(X)$ . The code words generated are in nonsystematic format. We have explained the generation of code words in systematic format in section 5.1. The generator polynomials of the BCH (63, 57), (127, 113) and (255, 239) code presented in chapter 8 are

$$\begin{aligned} g(X)_{(63,57)} &= 1 + X + X^6 \\ g(X)_{(127,113)} &= 1 + X + X^2 + X^4 + X^5 + X^6 + X^8 + X^9 + X^{14} \\ g(X)_{(255,239)} &= 1 + X + X^5 + X^6 + X^8 + X^9 + X^{10} + X^{11} + X^{13} + X^{14} + X^{16} \end{aligned}$$

The generator polynomial  $g(X)$  of a nonbinary RS code of length  $n = 2^m - 1$  is the minimum degree polynomial over  $GF(q = 2^m)$  i.e. coefficients from  $GF(2^m)$ , which has roots in the extension field  $GF(2^m)$ . Thus, the generator polynomial of a  $t$ -error correcting RS code has as roots, the  $2t$  consecutive powers of  $\alpha$  and is of the form

$$g(X) = \prod_{i=0}^{2t-1} (X + \alpha^{b+i}) = \sum_{i=0}^{2t-1} g_i X^i \quad 5.4$$

where  $b$  is a positive integer constant.

By carefully choosing the integer  $b$ , the circuit complexity of the encoder and decoder can be reduced [21]. The generator polynomial for the RS codes presented in chapter 8 is constructed with  $b = 1$ . Next, we will show that the minimum distance of the RS codes is  $d_{min} = 2t + 1$ .

Using the *Singleton bound* [51], which states that the minimum distance of a  $(n, k)$  code is upper-bounded by  $d_{min} \leq n - k + 1$ . If  $g(x)$  has  $2t$  consecutive powers of  $\alpha$  as roots in  $GF(2^m)$ , then the BCH bound implies that  $d_{min} \geq 2t + 1$  which is a lower bound on minimum distance. For both the bounds to hold true with equality we have  $n - k = 2t$  i.e.  $d_{min} = 2t + 1$ . RS codes satisfy the Singleton bound with equality and hence called *Minimum Distance Separable* (MDS) codes. For any code with the same  $(n, k)$ , the minimum distance  $d_{min} = n - k + 1$  is the maximum possible value achieved and codes that achieve this value are called MDS codes. This is evident from the comparison shown below

| Code Type    | $(n, k)$ | Minimum distance |                    |
|--------------|----------|------------------|--------------------|
| Hamming code | (15, 11) | 3                |                    |
| BCH code     | (15, 11) | 3                | (definition 5.2.1) |
| RS code      | (15, 11) | 5                | (definition 5.2.2) |

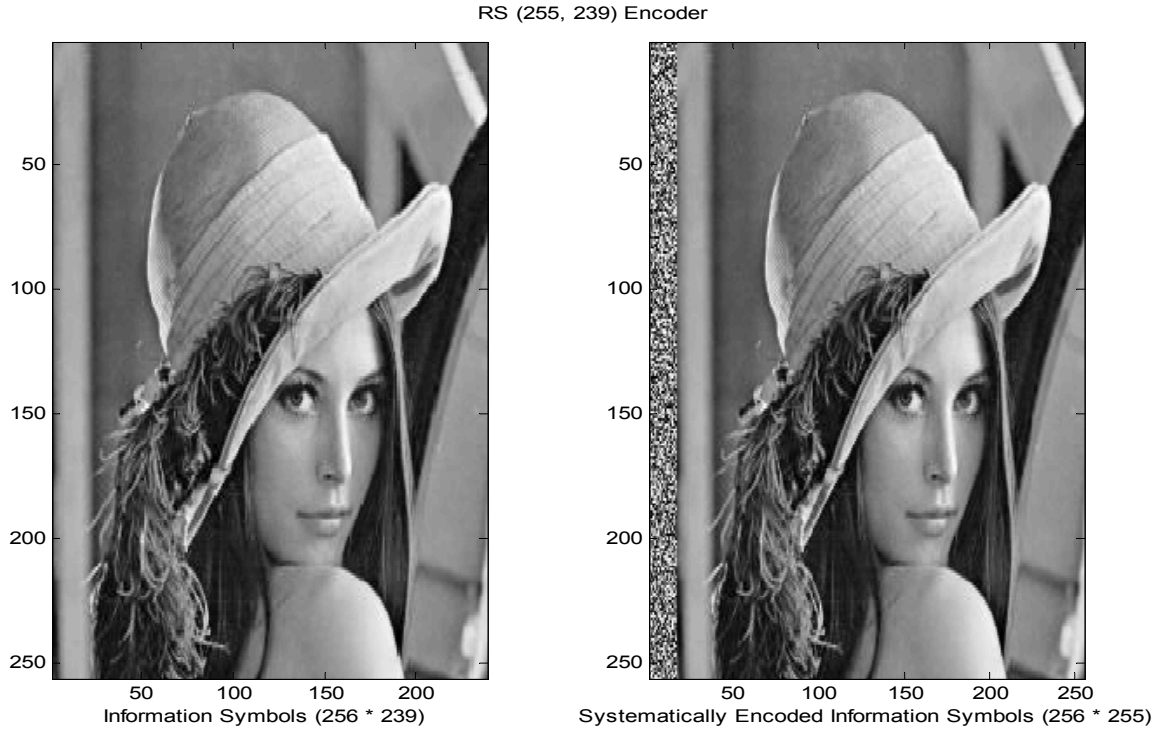
### 5.2.2 Systematic Encoding using the Polynomial Division Method

The systematic encoding procedure stated earlier for the cyclic codes in section 5.1 is applicable also to the BCH and RS codes. Instead of stating the procedure again, we will take an example for each of these codes. The BCH (255, 239) and RS (255, 239) codes in the systematic format generated by the polynomial division method are depicted in figures 5.1 and 5.2 respectively. The finite field  $GF(q = 2^8)$  is used in many applications because each of the  $2^8$  field elements can be represented as an 8-bit sequence or a byte [10]. RS codes of length  $n = 255$  are thus popular in optical communication systems. In chapter 8, we have presented the RS (255, 223), RS (255, 239) and RS (255, 247) codes which are constructed in systematic format in the time domain using the polynomial division method described in section 5.1.



**Figure 5.1 BCH (255, 239) Encoder<sup>1</sup>**

<sup>1</sup> The BCH (255, 239) code is generated by MATLAB function `codegenpoly.m` and `encodedivision.m`



**Figure 5.2 RS (255, 239) Encoder<sup>1</sup>**

### 5.3 The Galois Field Fourier Transform<sup>2</sup>

The code words when transmitted over the channel are physically a sequence of 0's and 1's, which are indexed in time. Analogous to the conventional discrete Fourier transform (DFT), similar interpretation for transform of code words is done over finite field [3].

**Definition 5.3.1:** The *finite field Fourier transform*  $V$  of an  $n$ -tuple vector (code word)  $v$  over  $GF(q)$ , where  $n$  divides  $q^m - 1$  for some positive integer  $m$ , is defined [3] as

$$V_j = \sum_{i=0}^{n-1} \alpha^{ij} v_i, \text{ for } j = 0, 1, \dots, n-1 \quad 5.5$$

where  $\alpha$  is an element in  $GF(q^m)$  having order  $n$ .

Note:  $j$  index for frequency domain components of vector (code word)  $V$   
 $i$  index for time domain components of vector (code word)  $v$

Similarly, the Inverse finite field Fourier transform,  $v$ , of an  $n$ -tuple vector  $V$  is defined as

$$v_i = \sum_{j=0}^{n-1} \alpha^{-ij} V_j, \text{ for } i = 0, 1, \dots, n-1 \quad 5.6$$

The vectors  $v$  and  $V$  form a Fourier transform pair denoted by  $v(x) \Leftrightarrow V(z)$

All the properties of the DFT hold true for the Galois Field Fourier transform (GFFT).

Properties of the GFFT are stated below without proof [12]

Property I:  $V_j = U_j G_j$  for  $j = 0, 1, \dots, n-1$  5.7

<sup>1</sup> The RS (255, 239) code is generated by MATLAB function `codegeneratorpolynomial.m` and `encoder.m`

<sup>2</sup> The definition and properties in section 5.3 are reproduced from [12 Chapter 8] for convenience

$$\text{if and only if } V_i = \sum_{k=0}^{n-1} u_k g_{i-k} \text{ for } i = 0, 1, \dots, n-1 \quad 5.8$$

where  $V_i = u_i * g_i$  is the convolution in time domain

Property II:  $\alpha^j$  is a root of the polynomial  $V(X)$  if and only if the  $j^{\text{th}}$  frequency component of the spectrum  $V(Z)$  equals zero

Property III:  $\alpha^{-i}$  is a root of  $V(Z)$  if and only if the  $i^{\text{th}}$  time component of  $V(X)$  equals zero.

### 5.3.1 Systematic Encoding in Frequency Domain

It is interesting to interpret the convolution in time domain given by (5.8) in terms of Property III mentioned in section 5.1 which states, every code polynomial  $V(X)$  in  $C$  is a multiple of  $g(X)$  of degree  $r$  and can be expressed uniquely as  $V(X) = u(X)g(X)$  where  $u(X)$  is a message polynomial of degree less than  $(n-r)$ . The convolution<sup>1</sup> results in  $n$  components of  $V$ . However, the convolution is equivalent to multiplication in the frequency domain such that  $V_j = U_j G_j$  for  $j = 0, 1, \dots, n-1$ . It can be verified that  $\alpha^j$  for  $j = 1, 2, \dots, 2t$   $G(\alpha^j) \& V(\alpha^j) = 0$ . The spectrum of  $u(X)$  is arbitrary, thus the resulting code can be completely specified by the spectrum of  $g(X)$  or  $V(X)$  itself to be zero in the components  $j$  for  $j = 1, 2, \dots, 2t$ . Given the background of GFFT we now describe the BCH and RS codes in the frequency domain:

**Definition 5.3.2:** A primitive  $t$ -error correcting BCH code of block length  $n = p^m - 1$  is the set of all words over  $GF(p)$  whose spectrum is zero in the  $2t$  consecutive components  $b, b+1, \dots, b+2t-1$ . The above definition is in accordance with the BCH bound, since the location  $j$  of the  $2t$  consecutive zero components in the spectra are the  $2t$  consecutive powers of  $\alpha$  which are roots of the generator polynomial  $g(X)$  over  $GF(2)$ . Thus, the BCH code defined in the frequency domain has design distance  $d_o = 2t + 1$ .

**Definition 5.3.3:** With the constraint imposed on the spectrum of  $V(X)$  by the BCH bound, a  $t$ -error correcting RS code over  $GF(q = p^m)$  of length  $p^m - 1$  contains all code words whose transforms have  $2t$  consecutive zeros. The encoding of the RS code in the frequency domain can be easily understood from the block schematic in figure 5.3.

Encoding the BCH code in the frequency domain does not appear to be of practical significance [4]. However, the coefficients of code vector  $V(X)$  in time domain and that of  $V(Z)$  in frequency domain of the RS codes are from  $GF(2^m)$  hence the implementation of the RS codes in frequency domain is significant. According to the Singleton bound the RS code has minimum distance  $d_{min} = 2t + 1 = n - k + 1$  i.e.  $n - k = 2t$ . Therefore, to encode the information symbols in the *frequency domain* the encoder sets the leftmost  $n - k$  symbols as the  $2t$  consecutive zeros in  $V(Z)$  i.e.  $V_j = 0$  for  $j = 0, 1, \dots, n - k - 1$  while the rightmost  $k$  symbols are set to information symbols from  $GF(2^m)$ . Note that the resulting code word is in frequency domain even though the  $j$  coordinates of  $V(Z)$  for  $2t + 1 \leq j \leq n$  are the symbols outputted by the source in real time. (This concept is slightly difficult to digest but is the essence behind frequency domain encoding). An inverse GFFT, defined in 5.6 is performed to obtain *time domain* RS code.

$$v_i = \sum_{j=0}^{n-1} \alpha^{-ij} V_j, \text{ for } i = 0, 1, \dots, n-1 \quad 5.9$$

<sup>1</sup> The code word obtained from the convolution is non-systematic form but does not matter for our analysis

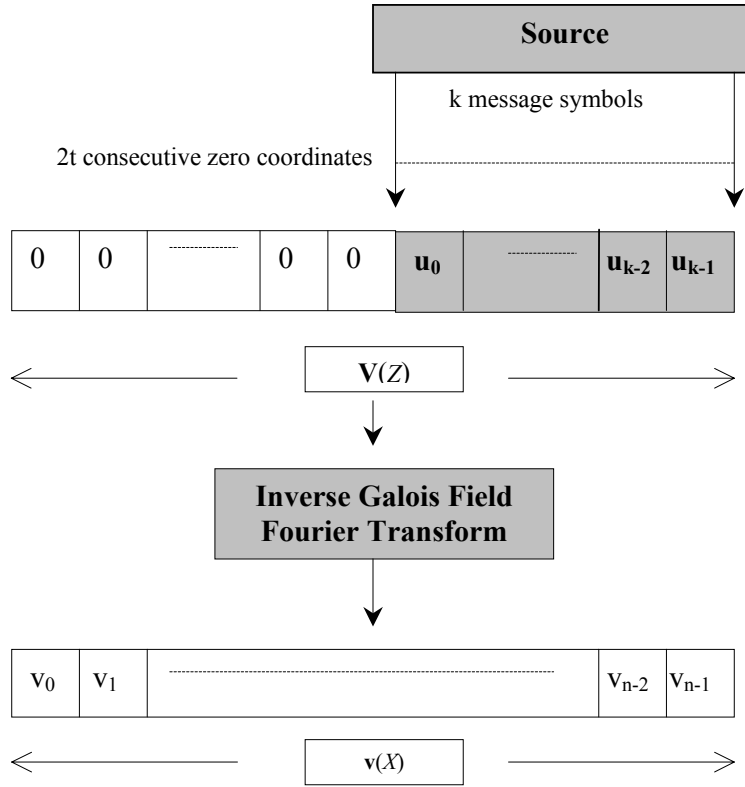


Figure 5.3 GFFT Encoder for Reed-Solomon Code<sup>1</sup>

## 5.4 Algebraic Decoding Algorithms for BCH and RS Codes<sup>1</sup>

In section 5.1 (Property III) we have seen that every code polynomial is a multiple of the generator polynomial and hence, it has the  $2t$  consecutive powers of  $\alpha$  as its roots. This forms the fundamental principle for decoding of the BCH and RS codes. We use the AWGN channel model shown in figure 4.2 for our analysis with the usual notations. A received word  $r$  is a valid transmitted code word  $v$  if and only if the received polynomial  $r(X)$  has the same  $2t$  consecutive powers of  $\alpha$  as roots. From our model, we note  $r = v + e$  so the received polynomial can be expressed as  $r(X) = v(X) + e(X)$ . The values obtained after evaluating  $r(X)$  at  $\alpha^i$  for  $1 \leq i \leq 2t$  is called *syndromes*  $s$  of the received polynomial  $r(X)$  over  $GF(2^m)$ . Conventionally, there will be no errors in  $r(X)$  if all the  $2t$  syndromes  $s = 0$  and if any  $s \neq 0$  then the decoder will raise an error event. This accomplishes the *error detection* at the receiver. If the received word  $r$  contains  $l \leq t$  errors and the decoder has declared an error event, the task of the decoder is to identify the location of  $l$  errors in a word of length  $n$ . Once the location of errors is identified, the symbols in the received word  $r$  at those locations need to be corrected such that the corrected word is most probably the valid transmitted code word  $v$ .

The syndromes are completely independent of the transmitted code word  $v$  and solely depend on the error pattern  $e$ . So, in mathematical form the syndromes  $s_i$  are expressed as

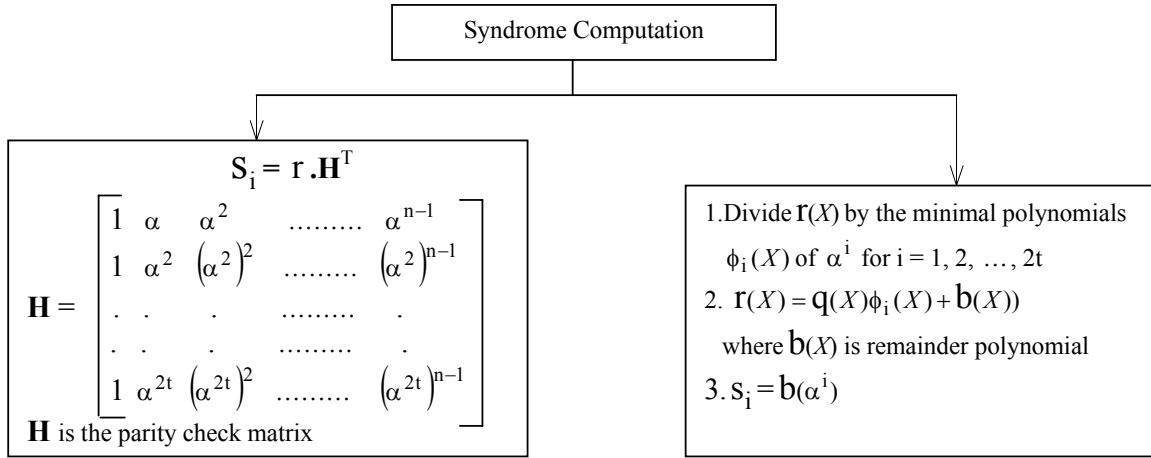
$$s_i = r(\alpha^i) = v(\alpha^i) + e(\alpha^i) = e(\alpha^i) = \sum_{k=0}^{n-1} e_k (\alpha^i)^k \quad ; \text{ for } i = 1, 2, \dots, 2t \quad 5.10$$

$$= \sum_{k=0}^{n-1} r_k (\alpha^i)^k$$

<sup>1</sup> S.B.Wicker, Error Control Systems for Digital Communication and Storage

<sup>1</sup> The equations in section 5.4 and subsections 5.4.1 and 5.4.2 are reproduced from [7 Chapter 6] for mathematical justification.

The syndrome computation can be performed by two methods



The computation of syndromes using the parity check matrix  $H$  is equivalent to syndrome computation given by 5.10. The syndrome calculation for the BCH and RS decoding is done using 5.10, which is more efficient than the minimal polynomial division method. The error polynomial can be written as

$$e(X) = e_0 + e_1X + e_2X^2 + \dots + e_{n-1}X^{n-1} \quad 5.11$$

Let us assume that there are  $l$  errors in coordinates  $j_1, j_2, \dots, j_l$  of the received word  $r$ , which are not known from the syndrome calculations. Since,  $r(X) = v(X) + e(X)$  the coordinates of  $e$  in these locations  $e_{j_1}, e_{j_2}, \dots, e_{j_l}$  will be 1 while others will be zero. Therefore, the error polynomial can be rewritten as

$$e(X) = X^{j_1} + X^{j_2} + \dots + X^{j_l} \quad 5.12$$

Thus, the syndromes are

$$S_i = \sum_{p=1}^l \left( \alpha^{j_p} \right)^i \text{ for } i = 1, 2, \dots, 2t$$

$$S_1 = \alpha^{j_1} + \alpha^{j_2} + \dots + \alpha^{j_l}$$

$$S_2 = \left( \alpha^{j_1} \right)^2 + \left( \alpha^{j_2} \right)^2 + \dots + \left( \alpha^{j_l} \right)^2 \quad 5.13$$

For convenience, we replace  $\alpha^{j_1} = \beta_1$  and so on.  $\beta_1, \beta_2, \dots, \beta_l$  are called *error location numbers* since they indicate the erroneous coordinates in  $r$

$$S_3 = (\beta_1)^3 + (\beta_2)^3 + \dots + (\beta_l)^3$$

$$\vdots$$

$$S_{2t} = (\beta_1)^{2t} + (\beta_2)^{2t} + \dots + (\beta_l)^{2t} \quad 5.14$$

The solution to the above  $2t$  equations has  $2^k$  possibilities each yielding a different error pattern. If there are  $l \leq t$  errors in the actual error pattern then the solution to the  $2t$  equations in 5.14 results in the most probable error pattern with smallest number of errors. However, the equations in  $\beta_1, \beta_2, \dots, \beta_l$  are non linear and difficult to solve directly. These  $2t$  equations are called *power-sum symmetric functions*. Peterson [41] showed that the power sum symmetric functions could be translated into a series of linear equations.

He introduced a polynomial  $\sigma(X)$  whose coefficients are unknown. The inverses of the roots of  $\sigma(X)$  are the error location numbers  $\beta_1, \beta_2, \dots, \beta_l$ . The polynomial  $\sigma(X)$  is therefore called *error locator polynomial* and has the form

$$\begin{aligned}\sigma(X) &= (1 + \beta_1 X)(1 + \beta_2 X) \dots (1 + \beta_l X) \\ &= \sigma_0 + \sigma_1 X + \sigma_2 X^2 + \dots + \sigma_l X^l\end{aligned}\tag{5.15}$$

where

$$\begin{aligned}\sigma_0 &= 1 \\ \sigma_1 &= \beta_1 + \beta_2 + \dots + \beta_l \\ \sigma_2 &= \beta_1 \beta_2 + \beta_2 \beta_3 + \dots + \beta_{l-1} \beta_l \\ &\vdots \\ \sigma_l &= \beta_1 \beta_2 \beta_3 \dots \beta_l\end{aligned}\tag{5.16}$$

The  $\sigma_l$ 's are known as elementary symmetric functions of  $\beta_l$ 's. From 5.14 and 5.16 the *Newton Identities* can be written as

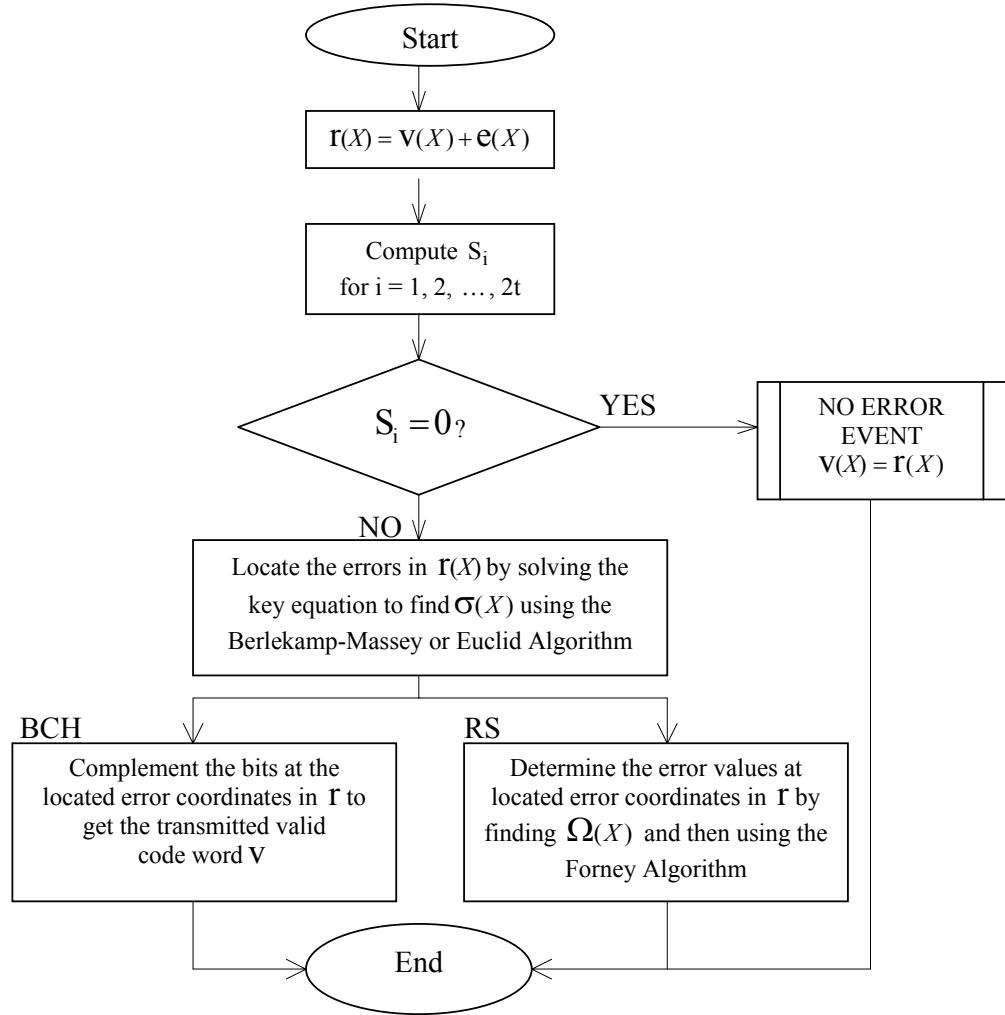
$$\begin{aligned}s_1 + \sigma_0 &= 0 \\ s_2 + \sigma_1 s_1 + 2\sigma_2 &= 0 \\ s_3 + \sigma_1 s_2 + \sigma_2 s_1 + 3\sigma_3 &= 0 \\ &\vdots \\ s_l + \sigma_1 s_{l-1} + \dots + \sigma_{l-1} s_1 + l\sigma_l &= 0 \\ s_{l+1} + \sigma_1 s_l + \dots + \sigma_{l-1} s_2 + \sigma_l s_1 &= 0 \\ &\vdots \\ s_{2t} + \sigma_1 s_{2t-1} + \dots + \sigma_{l-1} s_{2t-l+1} + \sigma_l s_{2t-l} &= 0\end{aligned}\tag{5.17}$$

The Newton identities are linear in  $l$  unknown coefficients  $\sigma_l$ 's of the error locator polynomial  $\sigma(X)$ . Since the BCH code is defined in  $\text{GF}(2)$ ,  $i\sigma_i = \sigma_i$ ; for odd  $i$  and  $i\sigma_i = 0$ ; for even  $i$ . The equations in 5.17 may have many solutions, however if  $l \leq t$  occur will give  $\sigma(X)$  of minimum degree, which satisfy the Newton identities. For  $l \leq t$ , the Newton identities can be expressed in matrix form and solved for the coefficients of  $\sigma(X)$  by the Peterson's direct solution decoding algorithm, Berlekamp's iterative algorithm or the Euclid's algorithm. It should be noted that though there may be a solution to 5.17, it might not lead to the correct  $\sigma(X)$ . There are two cases

- ◆ If the received word  $r$  is within the Hamming distance  $t$  of an incorrect code word  $w$  (refer section 4.2.1),  $\sigma(X)$  will decode the received word  $r$  into the incorrect code word  $w$ . This condition is undetectable, and is called a *decoder error*.
- ◆ If the received word is not within distance  $t$  of any code word, the error locator polynomial  $\sigma(X)$  may have repeated roots, or roots that do not lie in the smallest field containing the primitive  $n^{\text{th}}$  root of unity used to construct the code. This condition is called a *decoder failure* and is detectable.

Since we are using a Bounded-Distance decoder, when decoder failure arises we will accept received word  $r$  without further processing. Decoder error is undetectable, and occurs rarely with high probability for error patterns with exactly  $(d_{\min} + 1)/2$  errors (i.e.  $l > t$ ). We will not discuss the Peterson's algorithm since it involves computing the determinant of the syndrome matrix and inverting the syndrome matrix over  $\text{GF}(2^m)$  to determine the  $\sigma_l$ 's. A more efficient method to obtain  $\sigma(X)$ , which uses the highly structured syndrome matrix, is the Berlekamp algorithm. The complexity of Peterson's algorithm increases with the square of

the number of errors corrected whereas that of the Berlekamp algorithm increases linearly with the number of errors [12]. The algorithm is conceptually more complicated but computationally simpler. In the case of the binary BCH codes, once the location of errors is determined the bits at those coordinates in  $\mathbf{r}$  are complemented to correct the errors, whereas in the RS case the symbols are from  $GF(2^m)$ , the correct values at the erroneous coordinates need to be computed using the algebraic structure of the code word. Given the background of general decoding procedure for BCH and RS codes, the flow chart in figure 5.4 shows the working of a general decoder used to detect and correct errors. In section 5.4.1, we present the Berlekamp algorithm for decoding the BCH codes.



**Figure 5.4 General Working of a BCH/RS Decoder**

### 5.4.1 Berlekamp's Algorithm for BCH Codes

The Berlekamp's algorithm [2] for finding the error locator polynomial  $\sigma(X)$  is presented without proof. We introduce the *syndrome polynomial* of infinite degree of which the decoder knows the first  $2t$  coefficients.

$$s(X) = \sum_{i=1}^{\infty} s_i X^i = s_1 X + s_2 X^2 + \cdots + s_{2t} X^{2t} + s_{2t+1} X^{2t+1} + \cdots \quad 5.18$$

We define another polynomial called the *error evaluator polynomial*  $\Omega(X)$  such that

$$\begin{aligned}\Omega(X) &= [1 + s(X)]\sigma(X) \\ &= (1 + s_1X + s_2X^2 + \dots)(1 + \sigma_1X + \sigma_2X^2 + \dots) \\ &= 1 + (s_1 + \sigma_1)X + (s_2 + s_1\sigma_1 + \sigma_2)X^2 + (s_3 + s_2\sigma_1 + s_1\sigma_2 + \sigma_3)X^3 + \dots \\ &= 1 + \Omega_1X + \Omega_2X^2 + \dots\end{aligned}\quad 5.19$$

The coefficients of the *error evaluator polynomial*  $\Omega(X)$  are also unknown. Since only first  $2t$  coefficients of  $s(X)$  are known to the decoder, 5.19 can be rewritten as

$$[1 + s(X)]\sigma(X) = (1 + \Omega_1X + \Omega_2X^2 + \dots + \Omega_{2t}X^{2t}) \bmod X^{2t+1} \quad 5.20$$

5.20 is called the *key equation* because given  $s(X)$ , we wish to find  $\sigma(X)$  and  $\Omega(X)$ . The unknown polynomials  $\sigma(X)$  and  $\Omega(X)$  both have degree  $\leq t$  for  $l \leq t$ . The *error evaluator polynomial*  $\Omega(X)$  is not significant in decoding BCH codes. However in case of RS codes  $\Omega(X)$  once determined it is used to find the error magnitudes at the coordinates in  $r$  indicated by the error location numbers  $\beta_1, \beta_2, \dots, \beta_l$ . The problem of finding  $\sigma(X)$  and  $\Omega(X)$  looks difficult, but can be broken down into the form

$$[1 + s(X)]\sigma^{(\mu)}(X) = (1 + \Omega_1X + \Omega_2X^2 + \dots + \Omega_\mu X^\mu) \bmod X^\mu; \quad 5.21$$

for  $\mu = 1, 2, \dots, 2t$

In the case of binary BCH codes  $\sigma(X)$  can be determined for  $\mu = 1, 2, \dots, t$  iterations instead of  $2t$  iterations [2]. Once  $\sigma(X)$  is determined its roots are obtained by the *Chien search* [25]. The Chien search is an exhaustive search in which  $\sigma(X)$  is evaluated for all the  $2^m$  field elements. The field element for which  $\sigma(\alpha^i) = 0$  is the root. The method is feasible, since the number of elements in a Galois field is finite. The iterative procedure based on which the Berlekamp algorithm works is described in [2], [7]. The inverses of the roots are the error location numbers  $\beta_l$ 's. The decoding is accomplished by flipping the bits at the positions in  $r$  given by the error location numbers. In the next section, we present the Berlekamp-Massey algorithm for decoding the nonbinary RS codes. The Peterson-Gorenstein-Zieler decoding algorithm [30] for nonbinary RS codes is an extension of Peterson's direct decoding algorithm mentioned earlier. We will not present the algorithm because of the same limitations as the Peterson's algorithm.

#### 5.4.2 Berlekamp-Massey Algorithm for RS Codes

Decoding of the nonbinary RS codes involves not only determining the location of errors, but also their magnitudes. We re-write the error polynomial in 5.12

$$e(X) = e_{j_1}X^{j_1} + e_{j_2}X^{j_2} + \dots + e_{j_l}X^{j_l} \quad 5.22$$

For the RS case,  $e_{j_1}, e_{j_2}, \dots, e_{j_l}$  are field elements of  $GF(2^m)$ . The  $2t$  syndrome equations are

$$\begin{aligned}s_1 &= e_{j_1}(\beta_1) + e_{j_2}(\beta_2) + \dots + e_{j_l}(\beta_l)^3 \\ s_2 &= e_{j_1}(\beta_1)^2 + e_{j_2}(\beta_2)^2 + \dots + e_{j_l}(\beta_l)^{2t} \\ s_3 &= e_{j_1}(\beta_1)^3 + e_{j_2}(\beta_2)^3 + \dots + e_{j_l}(\beta_l)^3 \\ &\vdots \\ s_{2t} &= e_{j_1}(\beta_1)^{2t} + e_{j_2}(\beta_2)^{2t} + \dots + e_{j_l}(\beta_l)^{2t}\end{aligned}\quad 5.23$$

The  $\sigma_l$ 's and  $s_j$ 's are related by the Newton Identities and it is shown in [12] that the syndrome  $s_j$  can be expressed in recursive form as a function of  $\sigma_l$ 's and the earlier syndromes  $s_{j-1}, \dots, s_{j-l}$  such that

$$s_j = -(\sigma_l s_{j-l} + \sigma_{l-1} s_{j-l+1} + \dots + \sigma_1 s_{j-1}) = -\sum_{i=1}^l \sigma_i s_{j-i}; \quad 5.24$$

for  $j = l+1, l+2, \dots, 2t$

The above equation is realized in hardware with Massey's [38] linear feedback shift-register (LFSR). The structure of LFSR is shown in [12]. 5.24 is the equation for the classical *auto-regressive* filter [10]. Such a filter can be implemented as a LFSR with taps given by the coefficients of  $\sigma(X)$ . The filter taps  $\sigma_l$ 's are not known and determined iteratively such that the *smallest* LFSR generates the known sequence of  $2t$  syndromes. The smallest LFSR guarantees the error locator polynomial  $\sigma(X)$  of smallest degree. The working of the LFSR is not discussed in detail instead we continue with the Berlekamp–Massey algorithm.

The procedure for determining the error locator polynomial  $\sigma(X)$  remains the same as in the case of BCH codes except, now that  $\sigma(X)$  will be determined after  $2t$  iterations instead of  $t$  iterations. After the roots of the error locator polynomial  $\sigma(X)$  are determined and hence the error location numbers, the task of the decoder is to find the error magnitude at the error location numbers. Now, we utilize the error evaluator polynomial  $\Omega(X)$  introduced in 5.19. It has the same degree as  $\sigma(X)$ . With the known  $2t$  syndromes and coefficients of  $\sigma(X)$ , the error magnitudes are computed. The *Forney algorithm* [29] used to derive the error magnitudes is presented next. We re-write the key equation in 5.20

$$[1 + s(X)]\sigma(X) = (1 + \Omega_1 X + \Omega_2 X^2 + \dots + \Omega_{2t} X^{2t}) \bmod X^{2t+1}$$

The equation can be rearranged in terms of the known polynomial  $\sigma(X)$ , the error locator polynomial  $\Omega(X)$  and  $s(X)$  the syndrome polynomial

$$\Omega(X) = [1 + s(X)]\sigma(X) \bmod X^{2t+1} \quad 5.25$$

$\Omega(X)$  is naturally related to the error locations and error values by the  $t$  relations

$$\Omega(\beta_l^{-1}) = e_{j_l} \prod_{i \neq l} (1 - \beta_i \beta_l^{-1}); \text{ for } l = 1, 2, \dots, t \quad 5.26$$

The error magnitudes are computed using the expression

$$e_{j_l} = \frac{-\beta_l \Omega(\beta_l^{-1})}{\sigma'(\beta_l^{-1})} \text{ for } l = 1, 2, \dots, t \quad 5.27$$

where  $\sigma'(X)$  denotes the *formal derivative* of  $\sigma(X)$  with respect to  $X$ .

The formal derivative is similar to the usual derivative, but does not have the same interpretation. If  $f(X) = f_0 + f_1 X + f_2 X^2 + \dots + f_n X^n + \dots$  is a polynomial over GF(q), then the formal derivative  $f'(X)$  is defined as  $f'(X) = f_1 + 2f_2 X + \dots + n f_n X^{n-1} + \dots$ . The product and quotient rule apply to formal derivatives. Since  $f(X) \in \text{GF}(2^m)$ , then  $f'(X)$  has no odd power term.

Applying the definition of formal derivative to  $\sigma(X)$  5.27 can be written as

$$e_{j_l} = \frac{\Omega(\beta_l^{-1})}{\prod_{i=1, i \neq l}^t (1 + \beta_i \beta_l^{-1})} \quad 5.28$$

where  $e_{j_l}$  is the error value at the coordinate in  $r$  specified by  $\beta_l$

The iterative procedure based on which the Berlekamp-Massey algorithm works is described in [2], [7], [10] and [12]. After the  $2t$  iterations we have  $\sigma(X)$  whose roots are determined by the Chien search. The inverses of the roots are the error location numbers  $\beta_l$ 's. Now we know where the errors are in  $r$  but not their values. The decoder has to find the error magnitudes at these locations, which is accomplished by the Forney algorithm. The decoding is accomplished by adding the error pattern to  $r$  over GF (q). In the next section, we present the Euclid's algorithm for decoding the BCH and RS codes.

### 5.4.3 Euclid's Algorithm for BCH and RS Codes

The Euclid's algorithm is a recursive technique to find the greatest common divisor (GCD) of two polynomials [54]. If  $f(X)$  and  $g(X)$  are two polynomials where  $\deg(f(X)) \geq \deg(g(X))$  then the GCD is computed dividing  $f(X)$  by  $g(X)$  recursively such that the algorithm will always converge to a remainder polynomial  $d(X) = 0$  and the last nonzero polynomial  $d(X)$  is the GCD. The recursive relation between  $f(X)$  and  $g(X)$  is obtained by writing the initialization equation [4] such that

$$m(X)f(X) + n(X)g(X) = d(X) \quad 5.29$$

where  $m(X)$  and  $n(X)$  are intermediate polynomials obtained during the division process.

The division process is explained in short

- ◆ At the  $i^{\text{th}}$  iteration  $m(X)f^{(i)}(X) + n(X)g^{(i)}(X) = d^{(i)}(X)$
- ◆ For the  $i+1^{\text{th}}$  iteration  $f^{(i+1)}(X) = g^{(i)}(X)$  and  $g^{(i+1)}(X) = d^{(i)}(X)$
- ◆ The algorithm terminates when  $d^{(i)}(X) = 0$  and  $g^{(i)}(X)$  is the GCD of  $f(X)$  and  $g(X)$

In the case of decoding BCH and RS codes we are not interested in the GCD of  $f(X)$  and  $g(X)$  but the intermediate polynomials  $m(X)$  and  $n(X)$  at each iteration. We define a quotient polynomial [4]

$$q_i(X) = \left[ \frac{d_{i-2}(X)}{d_{i-1}(X)} \right]_0^\infty \text{ where } [ ]_0^\infty \text{ denotes nonnegative powers of } X \quad 5.30$$

then,

$$\begin{aligned} d_i(X) &= d_{i-2}(X) - q_i(X)d_{i-1}(X) \\ m_i(X) &= m_{i-2}(X) - q_i(X)m_{i-1}(X) \text{ and} \\ n_i(X) &= n_{i-2}(X) - q_i(X)n_{i-1}(X) \end{aligned} \quad 5.31$$

The initial conditions for the algorithm are

$$\begin{aligned} m_{-1}(X) &= n_0(X) = 1; m_0(X) = n_{-1}(X) = 0 \\ d_{-1}(X) &= f(X) \text{ and } d_0(X) = g(X) \end{aligned}$$

At each  $i^{\text{th}}$  iteration  $m_i(X)f(X) + n_i(X)g(X) = d_i(X)$ .

Coming back to the decoding of BCH/RS codes we re-write the key equation in 5.20 for our analysis

$$\sigma(X)[1 + s(X)] = \Omega(X) \bmod X^{2t+1}$$

From 5.29, we can express

$$n_i(X)g(X) = d_i(X) \bmod f(X) \quad 5.32$$

Comparing 5.29 with the key equation

$$n_i(X) = \sigma(X), \quad d_i(X) = \Omega(X) \quad \text{and} \quad f(X) = X^{2t+1}$$

To see that the approach produces the desired solution to the key equation we need to utilize the property of Euclid's algorithm [54] that states

$$\deg[n_i(X)] + \deg[d_{i-1}(X)] = \deg[f(X)] = 2t + 1$$

and

$$\deg[n_i(X)] + \deg[d_i(X)] < 2t + 1$$

For  $l \leq t$  errors the solution of interest has

$$\deg[\Omega(X)] = \deg[\sigma(X)] \leq t$$

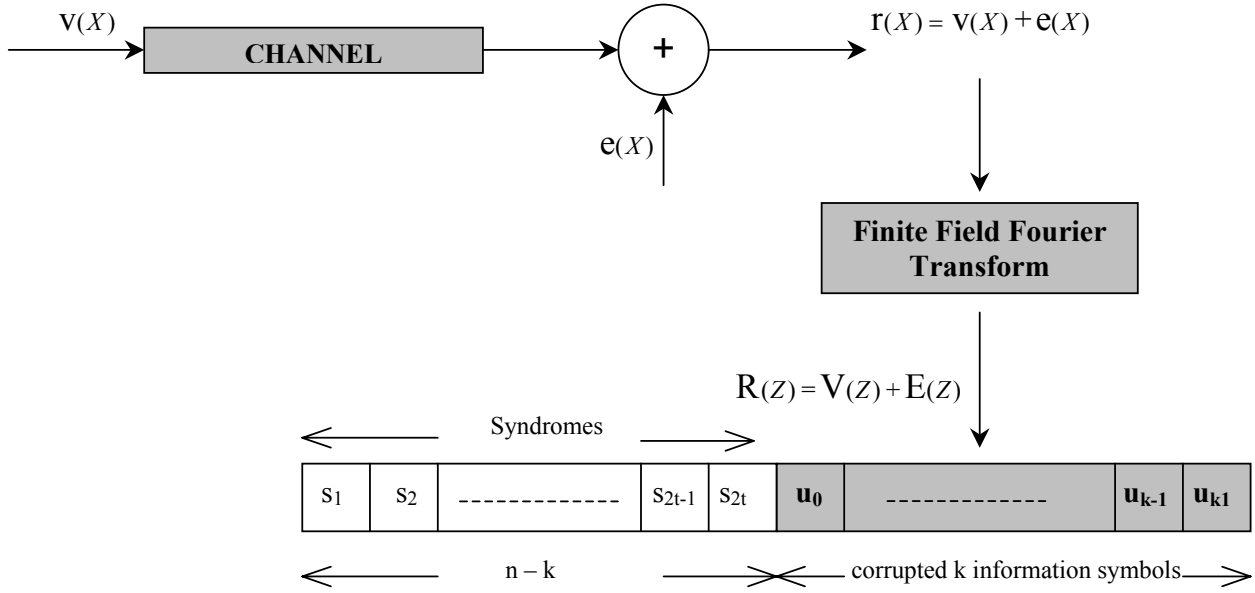
There exists only one polynomial  $\sigma(X)$  with degree no greater than  $t$ , which satisfies the key equation. This means that the intermediate result at the  $i^{\text{th}}$  iteration provide the solution to the key equation that is of interest. Thus, simply applying the Euclid's algorithm until  $\deg[d_i(X)] \leq t$  gives solution to the key equation. The rest of the decoding involves finding the roots of  $\sigma(X)$  by the Chien search. The inverses of the roots are the error location numbers as usual. Similarly, the Euclid's algorithm can be applied to decode the RS codes. In the next section, we will discuss decoding of RS codes in the frequency domain.

## 5.5 Transform Decoding of RS Code

In section 5.3.1, we had discussed encoding RS codes in the frequency domain. The time domain code word  $V(X)$ , which is not in systematic format, is obtained by taking the inverse GFFT of  $V(Z)$ . The transmitted code word  $V(X)$  is corrupted by channel noise and the received word  $R(X)$  is expressed as  $R(X) = V(X) + E(X)$ . Since the GFFT is linear [12], we have the following relationship

$$R(Z) = V(Z) + E(Z) \quad 5.33$$

Recall the constraint imposed on the spectrum of  $V(X)$  by the BCH bound, which defines a  $t$ -error correcting RS code of length  $n$  containing all code words whose transforms have  $2t$  consecutive zeros. If the received word  $R(X)$  is error free then the first  $2t$  consecutive coordinates of the spectrum of  $R(X)$  will be identically zero. Recalling Property II of the GFFT, if the first  $2t$  coordinates of  $R(Z)$  are zero means the corresponding  $2t$  consecutive powers of  $\alpha$  are roots of  $g(X)$  of the code. If any one of the  $2t$  consecutive coordinates of  $R(Z)$  is not equal to zero means that  $R(X)$  is not a valid code word  $V(X)$  and the corresponding  $2t$  values can be interpreted as the  $2t$  syndrome values of  $R(X)$ . Even though  $R(Z)$  is the frequency domain representation, the corrupted  $k$  information symbols of  $R(Z)$  are the symbols outputted by the source in real time as depicted in figure 5.6. It should be noted that  $R(Z)$  is in the systematic format. The Fourier transform technique is applied to avoid the systematic encoding of the code word using the polynomial division procedure at



**Figure 5.5 Frequency Domain Decoding**

the transmitter and to compute the syndromes at the receiver avoiding the parity-check matrix or the minimal polynomial method (refer section 5.4). Now, given the  $2t$  syndromes one could either use the Berlekamp-Massey or Euclid algorithm to find the error locator polynomial  $\sigma(X)$  as usual. Once  $\sigma(X)$  is determined in our discussion we set  $\Lambda(Z) = \sigma(X)$  where  $\Lambda(Z)$  is the error locator polynomial in the frequency domain. The error pattern  $E(Z)$  is the spectrum of  $e(X)$ . Thus, out of  $n$  coordinates of the transform of  $e(X)$ ,  $2t$  can be directly obtained from the syndromes such that  $S_j = R_j$  and  $E_j = S_j \forall j = 0, 1, \dots, 2t-1$ . Given the  $2t$  frequency coordinates of  $E(Z)$  and that *at most*  $t$  coordinates of  $e(X)$  are nonzero, the decoder must find the entire transform  $E(Z)$ . This is accomplished using the convolution Property I stated in section 5.3. The convolution of  $\Lambda(Z)$  and  $E(Z)$  in frequency domain is zero [3] i.e.

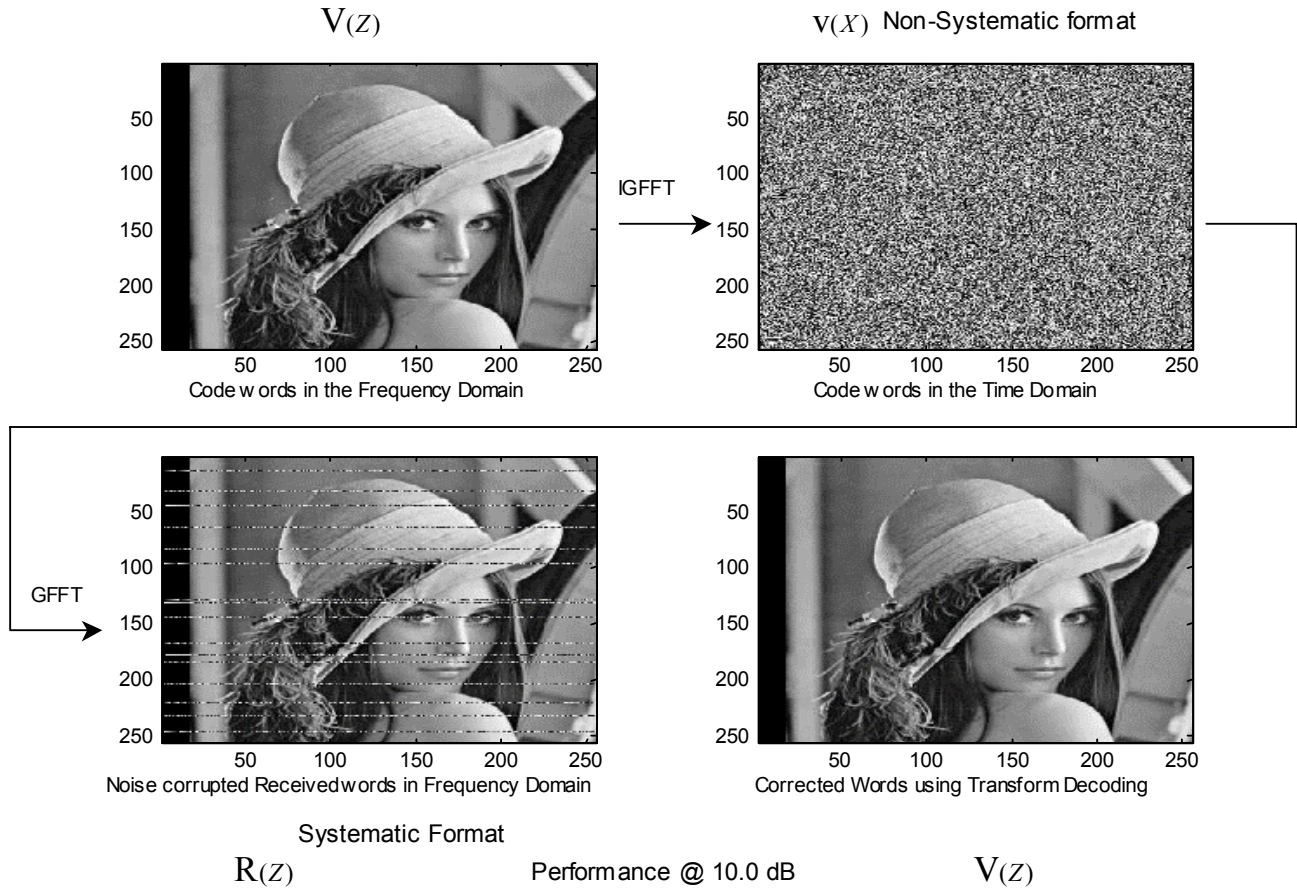
$$\Lambda * E = \sum_{k=0}^{n-1} \Lambda_k E_{j-k} = 0; \text{ for } j = 0, 1, \dots, n-1. \quad 5.34$$

The convolution can be considered as a set of  $n$  equations in  $k$  unknown coordinates of  $E(Z)$ . The error locator polynomial  $\Lambda(Z)$  in the frequency domain is such that  $\Lambda_0 = 1$ . The remaining coordinates of  $E(Z)$  can be obtained by recursive extension such that

$$E_j = \sum_{k=0}^{n-1} \Lambda_k E_{j-k}; \text{ for } j = 0, 1, \dots, n-1. \quad 5.35$$

Adding  $E(Z)$  to  $R(Z)$  over  $GF(2^m)$  completes the decoding in the frequency domain of vector  $R(Z)$ .

We conclude this chapter by summarizing the encoding/decoding schemes for BCH/RS codes. In Algebraic decoding the information symbols are encoded in systematic format in time domain using the polynomial division method and the received code word is decoded in time domain by computing the syndromes. The block schematic of the algebraic decoder in time domain is shown in [13]. The key equation is solved to obtain  $\sigma(X)$  either by Berlekamp or Euclid algorithm (BCH codes) and  $\sigma(X)$  and  $\Omega(X)$  are obtained either by the Berlekamp-Massey or Euclid algorithm (RS codes). The Chien search is used to find roots of  $\sigma(X)$ . The Forney algorithm is used to find  $e(X)$  which completes the decoding.



**Figure 5.6 Transform Decoding of RS (255, 239) Code**

In Transform decoding the information symbols are encoded in frequency domain and the received word is decoded in frequency domain. The block schematic of the transform decoder in frequency domain is shown in [13]. The syndromes are computed by performing Fourier transform of the received word. Either the Berlekamp-Massey or Euclid Algorithm determines the error locator polynomial  $\Lambda(Z)$  in the frequency domain. The error pattern in frequency domain is obtained from the convolution of  $\Lambda(Z)$  and known  $2t$  coordinates of  $E(Z)$ . The corrected received word in frequency domain is in systematic format and the rightmost  $k$  symbols are the information symbols.

In Hybrid decoding, the RS code is encoded in time domain by polynomial division method. The decoding is accomplished in frequency domain using transform decoding. At the receiver, the transmitted codeword  $v(X)$  in time domain is obtained by performing the inverse transform of the corrected word  $\hat{V}(Z)$ . In the next chapter, we will discuss in detail the error detection/correction performance of the BCH/RS codes.

# Chapter 6

## 6 Performance Analysis for BCH and RS Codes

The channel models used in the performance analysis of BCH and RS codes are the AWGN channel model in figure 4.2 and the BSC channel model in figure 4.3. In case of soft decision decoding the decoder experiences an AWGN with  $r'$  unquantized while for hard decision decoding a BSC becomes applicable with  $r'$  quantized by the threshold detector to binary output. First, we perform the analysis for hard decision decoding.

### 6.1 Error Detection Performance<sup>1</sup>

We have performed the error detection performance analysis for the BCH (31, 21) code as a reference to justify the claims in section 4.2.1. For a C (n, k) code the probability of undetected word error is bounded above by the probability of occurrence of an error pattern of weight  $d_{min}$  or greater [12] and is given by

$$P_u(E) \leq \sum_{i=d_{min}}^n \binom{n}{i} p^i (1-p)^{n-i} \quad 6.1$$

where

$P_u(E)$ : probability of an undetected word error

$p$ : transition probability of BSC

The probability of detected word error  $P_d(E)$  is bounded above by the probability that one or more bit-errors occur [12]

$$P_d(E) \leq \sum_{i=1}^n \binom{n}{i} p^i (1-p)^{n-i} = 1 - (1-p)^n \quad 6.2$$

If the WEF (refer section 4.5) of the code is known, exact expression for the probability of undetected word error can be obtained [12].

$$P_{u(exact)}(E) = \sum_{i=d_{min}}^n A_i p^i (1-p)^{n-i} \quad 6.3$$

where  $A_i$ : WEF of the code

Using the BSC model, the undetected and detected word error performance for the BCH (31, 21) code is presented in figure 6.1. The bound on  $P_d(E)$  is tight whereas the bound on  $P_u(E)$  is loose.

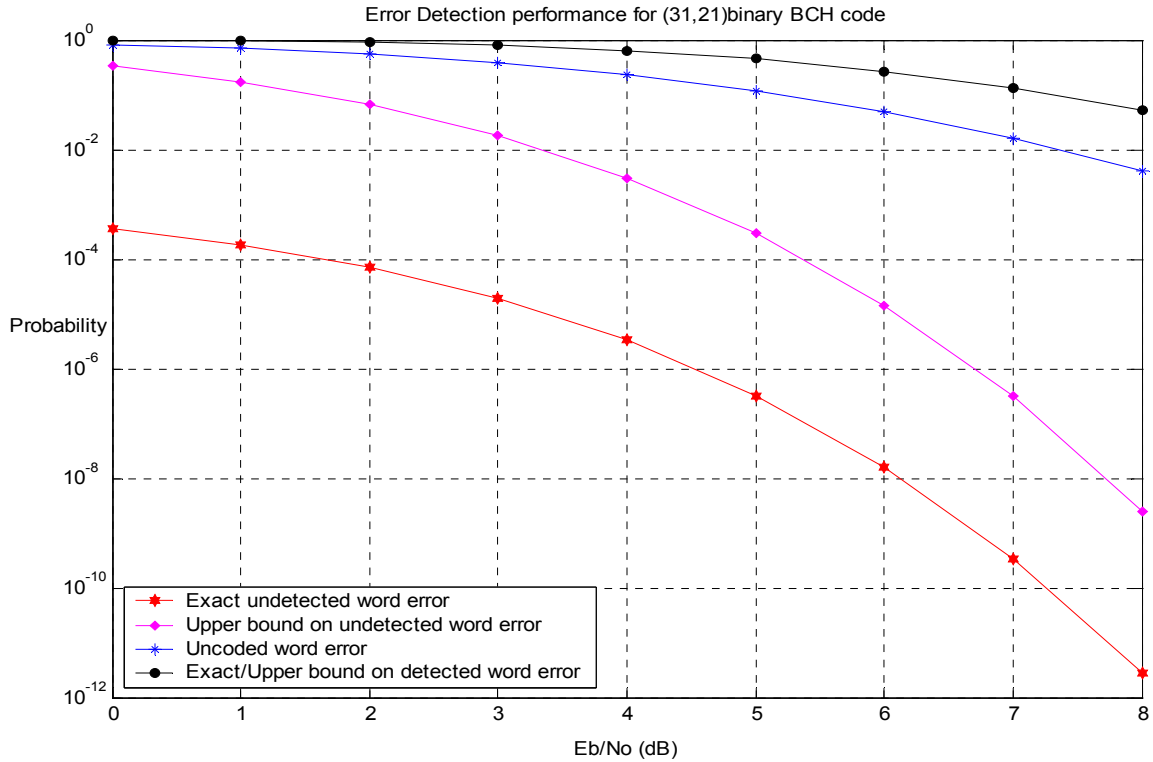
---

<sup>1</sup> The equations in section 6.1 are reproduced from [12 Chapter 10] to support the theoretical background.

| i   | A <sub>i</sub> | i  | A <sub>i</sub> | i  | A <sub>i</sub> | i  | A <sub>i</sub> | I     | A <sub>i</sub> |
|-----|----------------|----|----------------|----|----------------|----|----------------|-------|----------------|
| 0   | 1              | 9  | 18910          | 15 | 301971         | 21 | 41602          | 27-30 | 0              |
| 1-4 | 0              | 10 | 41602          | 16 | 301971         | 22 | 18910          | 31    | 1              |
| 5   | 186            | 11 | 85560          | 17 | 251100         | 23 | 7905           |       |                |
| 6   | 806            | 12 | 142600         | 18 | 195300         | 24 | 2635           |       |                |
| 7   | 2635           | 13 | 193500         | 19 | 142600         | 25 | 806            |       |                |
| 8   | 7905           | 14 | 251100         | 20 | 85560          | 26 | 186            |       |                |

**Table 6.1 Weight Distribution of the BCH (31, 21) Code**

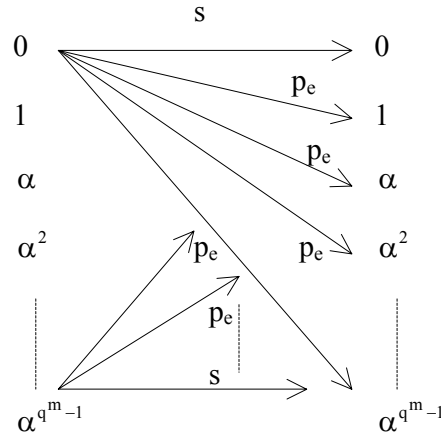
There are total of  $(2)^k = (2)^{21} = 2097152$  possible code words in the BCH (31, 21) code. Thus, from figure 6.1 it becomes evident that the number of undetected error patterns in comparison to that is detected by the decoder are significantly small. For the BCH (31, 21) code @ SNR = 4 dB, if  $10^6$  code words are transmitted then  $P_u(E) = 3 \times 10^{-6}$  i.e. only 3 erroneous code words pass through the decoder without being detected for errors.



**Figure 6.1 Word Error Detection Performance of BCH (31, 21) Code<sup>⊥</sup>**

Similarly, upper bound on the undetected WER performance of a RS code can be derived. However, in case of RS codes the code word symbols are elements of  $GF(2^m)$  and when transmitted over a BSC using BPSK experiences a  $2^m$ -ary *Uniform Discrete Symmetric Channel* (UDSC) as shown below. It is to be noted that the code word symbols are from  $GF(2^m)$  and transmitted over BSC using BPSK hence the channel symbols are from  $GF(2)$ .

<sup>⊥</sup> S.B.Wicker, Error Control Systems for Digital Communication and Storage



Thus, the probability of channel symbol error is the transition probability  $p$  of the BSC whereas the probability of code word symbol error is given by

$$p_{se} = 1 - (1 - p)^{m/b}; \quad b=1, (\text{BPSK}) \quad 6.4$$

The probability that a code word symbol is correctly received is  $s$  and given by

$$s = (1 - p)^{m/b}; \quad b=1, (\text{BPSK}) \quad 6.5$$

and the probability that a particular incorrect code word symbol other than the transmitted one is received is the probability of receiving any one of the  $q^m-1$  symbols and is given as

$$p_e = \frac{p_{se}}{q^m - 1}; \quad b=1, (\text{BPSK}) \quad \text{and} \quad q=2 \quad 6.6$$

The upper bound for the undetected and detected word error [12] is given by

$$P_u(E) \leq 1 - \sum_{i=0}^{d_{\min}-1} \binom{n}{i} (1-s)^i s^{n-i} \quad 6.7$$

$$P_d(E) \leq 1 - s^n$$

If the WEF of the RS code is known exact  $P_u(E)$  &  $P_d(E)$  can be obtained [12] and is given by

$$P_{u(\text{exact})}(E) = \sum_{i=d_{\min}}^n A_i p_e^i s^{n-i} \quad 6.8$$

$$P_{d(\text{exact})}(E) = 1 - s^n - P_u(E)$$

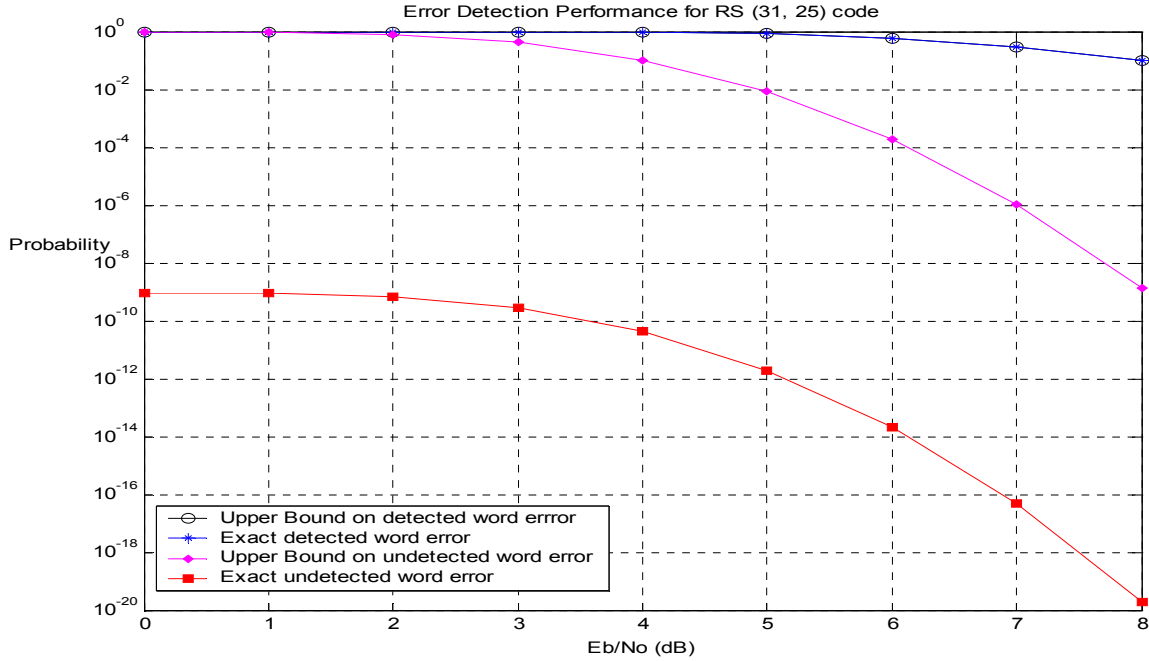
The WEF of the RS  $(31, 25)^\perp$  code used to compute the exact  $P_u(E)$  and  $P_d(E)$  is presented in table 6.2. There are  $(2^5)^{25} = 4.253529587 * 10^{37}$  possible code words in the RS  $(31, 25)$  code.

| I   | A <sub>i</sub>                     | I  | A <sub>i</sub>                     | I  | A <sub>i</sub>                     |
|-----|------------------------------------|----|------------------------------------|----|------------------------------------|
| 0   | 1                                  | 14 | 1.869428005780732*10 <sup>20</sup> | 23 | 1.470367535010005*10 <sup>32</sup> |
| 1-6 | 0                                  | 15 | 6.567923738966960*10 <sup>21</sup> | 24 | 1.519379786177005*10 <sup>33</sup> |
| 7   | 81516825                           | 16 | 2.036056358893236*10 <sup>23</sup> | 25 | 1.318821654401641*10 <sup>34</sup> |
| 8   | 6.113761875*10 <sup>9</sup>        | 17 | 5.569212981703244*10 <sup>24</sup> | 26 | 9.434647219950197*10 <sup>34</sup> |
| 9   | 4.974700107*10 <sup>11</sup>       | 18 | 1.342799130032614*10 <sup>26</sup> | 27 | 5.416186367008447*10 <sup>35</sup> |
| 10  | 3.38504593713*10 <sup>13</sup>     | 19 | 2.848147628437624*10 <sup>27</sup> | 28 | 2.398596819675169*10 <sup>36</sup> |
| 11  | 2.00366246194569*10 <sup>15</sup>  | 20 | 5.29755458889398*10 <sup>28</sup>  | 29 | 7.692051869992783*10 <sup>36</sup> |
| 12  | 1.035214581003193*10 <sup>17</sup> | 21 | 8.602219594346889*10 <sup>29</sup> | 30 | 1.589690719798509*10 <sup>37</sup> |
| 13  | 4.690321324809471*10 <sup>18</sup> | 22 | 1.212130942839789*10 <sup>31</sup> | 31 | 1.589690719798508*10 <sup>37</sup> |

**Table 6.2 Weight Distribution of the RS (31, 25) Code**

<sup>⊥</sup> The WEF is computed using MATLAB function weight.m

Using the BSC and UDSC model the undetected and detected word error performance for the reference RS (31, 25) code is presented in figure 6.2. As expected, the bound on  $P_d(E)$  is tight whereas the bound on  $P_u(E)$  is very loose. For the RS (31, 25) code @ SNR = 4 dB, if  $10^{11}$  code words are transmitted then  $P_{u(\text{exact})}(E) = 4 * 10^{-11}$  which means only 4 erroneous code words pass through the decoder without being detected for errors.



**Figure 6.2 Word Error Detection Performance of RS (31, 25) Code**

## 6.2 Error Correction Performance<sup>1</sup>

We have used the Bounded-distance decoder (refer section 4.4), which according to ML criterion selects the most likely code word  $\hat{v}$  if the received word  $r$  is within Hamming distance  $t = \lfloor (d_{\min} - 1) / 2 \rfloor$  of  $\hat{v}$ . If  $\hat{v} \neq v$  then decoder word error has occurred and if there is no  $\hat{v}$  within Hamming distance  $t$  of  $r$  then it is decoder failure. First, we present the error correction performance for the BCH codes. Using the BSC model, the probability that the decoder commits an erroneous decoding [12] is upper-bounded by

$$P(E) \leq \sum_{i=t+1}^n \binom{n}{i} p^i (1-p)^{n-i} = 1 - \sum_{i=0}^t \binom{n}{i} p^i (1-p)^{n-i} \quad 6.9$$

The probability of decoder failure is upper-bounded by the probability that  $r$  is far away from  $\hat{v}$  in Hamming distance  $t$ , and hence is the same as the bound on  $P(E)$

$$P(F) \leq 1 - \sum_{i=0}^t \binom{n}{i} p^i (1-p)^{n-i} \quad 6.10$$

If the WEF of the code is known, exact probability of word error can be obtained [12]

$$P_{\text{exact}}(E) = \sum_{i=d_{\min}}^n A_i \sum_{k=0}^t P_k^i \quad 6.11$$

<sup>1</sup> The equations in section 6.2 are reproduced from [12 Chapter 10] to support the theoretical and simulated WER and BER performance of BCH and RS codes.

$P_k^i$  is the probability that  $r$  is exactly Hamming distance  $k$  from a weight  $i$  code word

$$P_k^i = \sum_{r=0}^k \binom{i}{k-r} \binom{n-i}{r} p^{i-k+2r} (1-p)^{n-i+k-2r} \quad 6.12$$

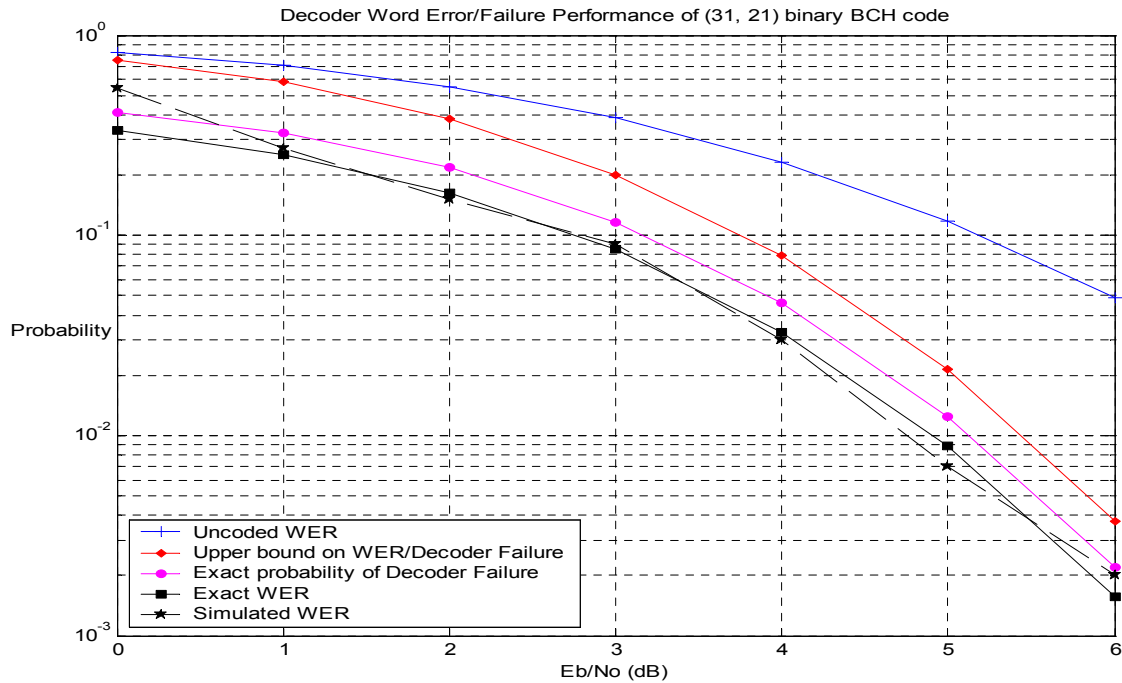
The probability of decoder failure is the probability that  $r$  is not within Hamming distance  $t$  of a correct or incorrect code word for the Bounded-distance decoder [12]

$$P_{\text{exact}}(F) = 1 - \sum_{i=0}^t \binom{n}{i} p^i (1-p)^{n-i} - P_{\text{exact}}(E) \quad 6.13$$

The exact probability of information bit-error can be determined if the relationship between the weight of the message blocks and the weight of the corresponding code words is available [12] which is difficult to compute for large  $k$ . The upper and lower bound on probability of information bit-error is given by

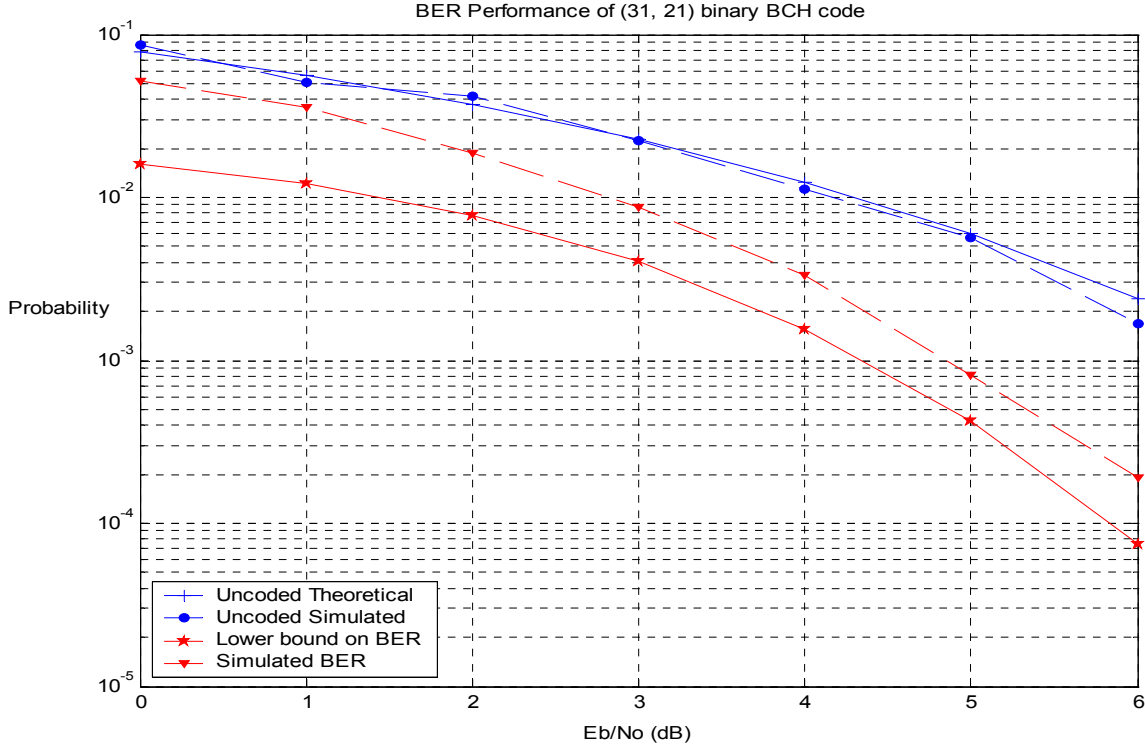
$$P_{\text{exact}}(E) \geq P_{b(\text{inf})}(E) \geq \frac{1}{k} P_{\text{exact}}(E) \quad 6.14$$

The decoder word error and decoder failure performance for the reference BCH (31, 21) code is presented in figure 6.3. For the bounded-distance decoder the probability of decoder word error is lower than the probability of decoder failure, which is justified by the error detection performance in figure 6.1. The simulated WER will have contribution from the decoder error as well as decoder failure, but the decoder failure contribution will be dominant in the simulated WER because in the event of decoder failure the decoder delivers  $r$  as the valid transmitted codeword  $v$ , which in turn is erroneous decoding. The probability of information bit-error is presented in figure 6.4. The simulated BER curve is below the uncoded curve and above the lower bound BER curve. The simulated error correction performance for the reference BCH (31, 21) code is consistent with the theoretical performance and hence we confirm the confidence in the software implementation of the BCH encoder/decoder, which is used to simulate the performance of the Block Turbo Codes presented in chapter 7.



**Figure 6.3 Word Error Correction Performance of BCH (31, 21) Code<sup>⊥</sup>**

<sup>⊥</sup> S.B.Wicker, Error Control Systems for Digital Communication and Storage



**Figure 6.4 Bit-Error Correction Performance of BCH (31, 21) Code**

Next, we present the error correction performance of the RS codes, which is quite similar to the BCH case with a slight modification. Using the BSC and UDSC model, the probability that the decoder commits an erroneous decoding [12] is upper-bounded by

$$P(E) \leq \sum_{i=t+1}^n \binom{n}{i} (1-s)^i s^{n-i} = 1 - \sum_{i=0}^t \binom{n}{i} (1-s)^i s^{n-i} \quad ; n = q^m - 1 \quad 6.15$$

The upper bound on the probability of decoder failure is the probability that  $r$  lies outside the decoding sphere of the transmitted code word  $v$  and is same as 6.13. If the WEF of the code is available, the exact expression for decoder word error and failure can be obtained [12] and is given by

$$P_{\text{exact}}(E) = \sum_{i=d_{\min}}^n A_i \sum_{k=0}^t P_k^i \quad 6.16$$

where  $P_k^i$  is the probability that  $r$  is exactly Hamming distance  $k$  from a weight  $i$  code word  $v_i$

$$P_k^i = \sum_{r=0}^k \binom{i}{k-r} \binom{n-i}{r} p_e^{i-k+r} (1-p_e)^{k-r} s^{n-i-r} (1-s)^r \quad 6.17$$

and

$$A_i = \binom{n}{i} (q-1) \sum_{j=0}^{i-d_{\min}} (-1)^j \binom{i-1}{j} q^{i-j-d_{\min}} \quad 6.18$$

The probability of decoder failure is the probability that  $r$  is not within Hamming distance  $t$  of a correct or incorrect code word for the Bounded-distance decoder [12]

$$P_{\text{exact}}(F) = 1 - \sum_{i=0}^t \binom{n}{i} (1-s)^i s^{n-i} - P_{\text{exact}}(E) \quad 6.19$$

The upper and lower bound on probability of information bit-error is given by

$$P_{\text{exact}}(E) \geq P_{\text{b(inf)}}(E) \geq \frac{1}{(k * m)} P_{\text{exact}}(E) \quad 6.20$$

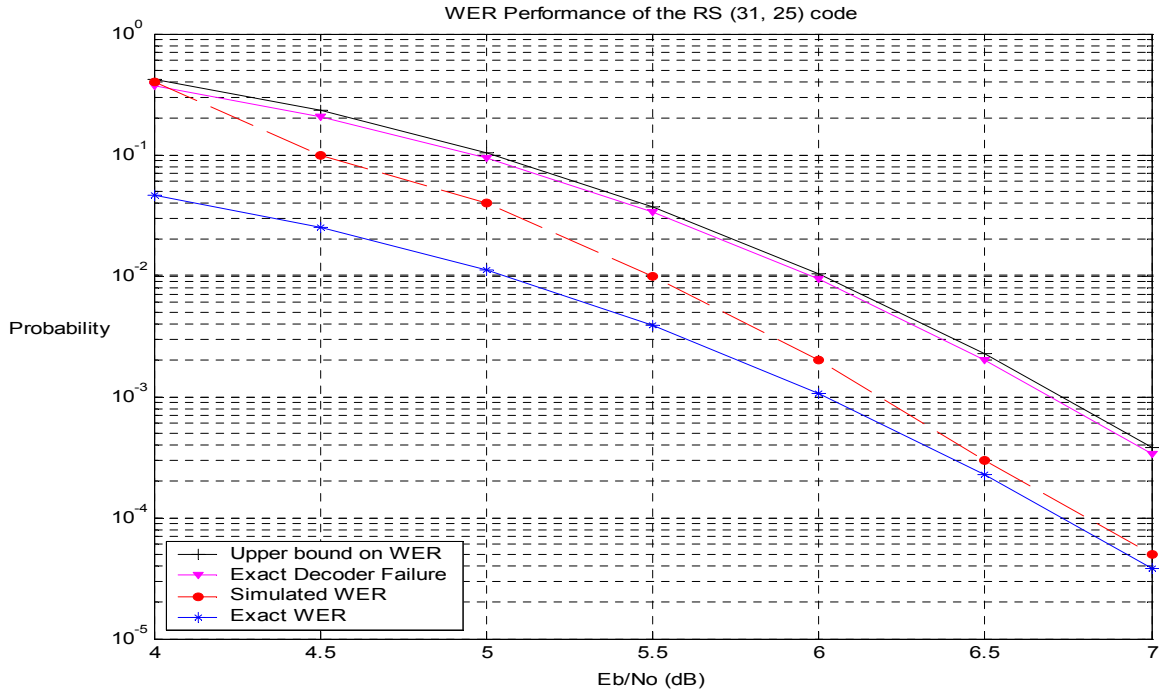
The upper bound and exact probability of word error and decoder failure for the RS (31, 25) code is presented in figure 6.5. The simulated WER will be mainly dominated by the probability of decoder failure at low SNR while it is dominated by decoder word error as well as decoder failure at high SNR. For large  $n$  it is difficult to compute  $A_i$  and therefore we have used an approximation to compute the information bit-error rate at the output of the decoder. From the undetected word error performance in figure 6.2 we have seen that the probability that the decoder allows an invalid code word  $w$  other than the one transmitted  $v$  is negligibly small. For the Bounded-distance decoder we assume that errors occur independently from each other and from 6.19 and figure 6.5 it is evident that the probability of decoder failure is dominant than the exact probability of decoder word error, which occurs due to undetected erroneous code words that pass without being detected and hence can be neglected. With these assumptions the probability of code word symbol error at the input of the decoder is

$$p_{\text{se}} = 1 - (1 - p)^{m/b}; \quad b=1, (\text{BPSK}) \quad 6.21$$

$$\text{where } p = Q\left(\sqrt{\frac{2 * R_c * E_b}{N_o}}\right); \quad R_c \text{ is the code rate of the code.}$$

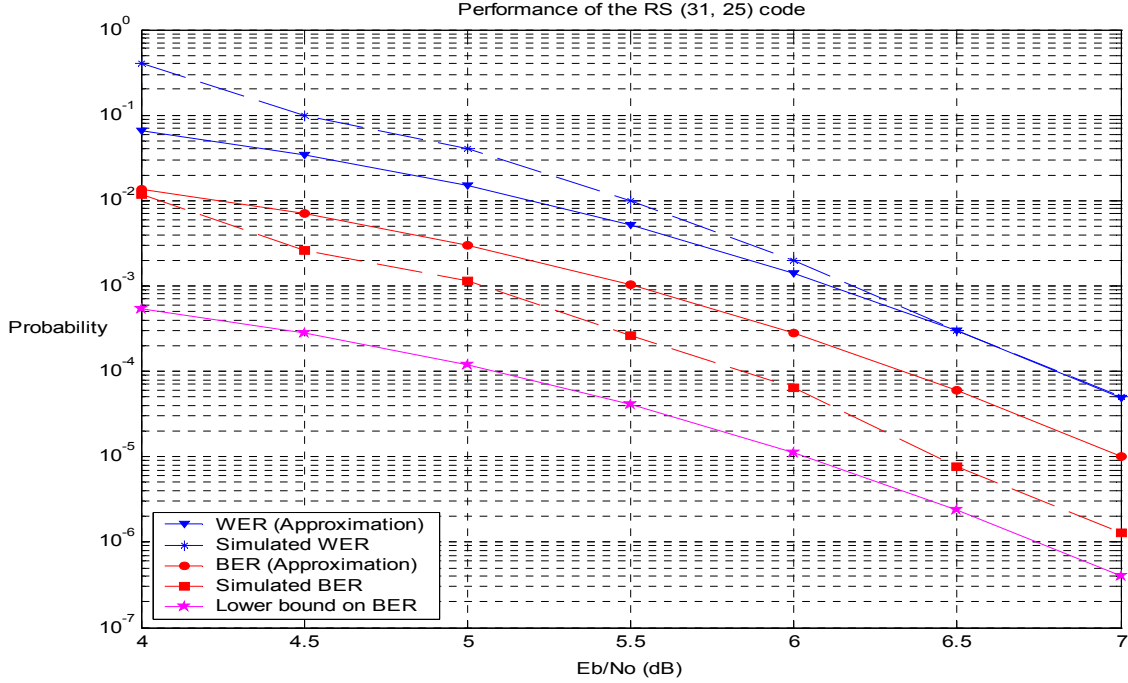
The value of  $p$  gives the information BER at the input of the decoder. The probability of Uncorrectable word error [36] is given by

$$P_u(E) \approx \sum_{i=t+1}^n \frac{i}{n} \binom{n}{i} p_{\text{se}}^i (1 - p_{\text{se}})^{n-i} \quad ; n = q^m - 1 \quad 6.22$$



**Figure 6.5 Decoder Word Error Performance of RS (31, 25) Code<sup>⊥</sup>**

<sup>⊥</sup> S.B.Wicker, Error Control Systems for Digital Communication and Storage



**Figure 6.6 Approximate Word Error and Bit-Error Performance of RS (31, 25) Code**

The information BER at the output of the decoder [36] is given by

$$P_{b(\text{output})} \approx 1 - (1 - P_u(E))^{1/m} \quad 6.23$$

From the error correction performance of the RS (31, 25) code in figures 6.5 and 6.6, we observe that the exact probability of word error given by 6.16 and the approximate word error rate given by 6.22 are in coherence. The simulated information BER is above the lower bound given by 6.20 and below the approximation given by 6.23, which may be due to the nonbinary nature of the RS codes. We extend the above approximation based on (6.21, 6.22 and 6.23). to analyze the theoretical performance of the RS (255, 247), RS (255, 239) and RS (255, 223) codes presented in chapter 8. We have confirmed the analytical approximation with the exact WER performance for the reference RS (31, 25) code. The simulated WER and BER correction performance for the reference RS (31, 25) code is consistent with the approximated performance. Hence, we confirm the confidence in the software implementation of the RS encoder/decoder, which is used to simulate the performance of the high dimension RS codes presented in chapter 8 and the serially concatenated RS codes presented in chapter 7.

### 6.3 Optical Receiver Sensitivity

The power budget of a communication system ensures that sufficient power is available at the receiver to maintain reliable performance [1]. The *receiver sensitivity* is then defined as the minimum average received power  $\bar{P}_{\text{rec}}$  required by the receiver to operate at a specified BER

$$\bar{P}_T = \bar{P}_{\text{rec}} + C_L + M_s \quad 6.24$$

$\bar{P}_T$ : average transmitted optical power (dBm)

$\bar{P}_{\text{rec}}$ : average received optical power (dBm)

$C_L$ : channel loss,  $\alpha$  is fiber attenuation (dB/km) and  $L$  is length of fiber (km)

$M_s$ : Margin that accounts for other losses (dB)

An optical receiver is said to be more *sensitive* if it achieves the same performance with less optical power incident on it. The performance criterion for digital receivers is governed by the BER. A commonly used criterion for digital optical receivers requires  $\text{BER} \leq 1 \times 10^{-9}$ . Since the BER depends on the  $\bar{P}_{\text{rec}}$ , in the next section our focus of discussion will be on BER and Q. For the 10 Gigabit Ethernet standard, the acceptable BER is  $\leq 1 \times 10^{-12}$ .

### 6.3.1 Relation between BER and Q & Q and SNR

Figure 6.7 shows schematically the fluctuating signal generated by the photo detector. The signal fluctuates around an average value  $I_1$  or  $I_0$ , corresponding to a 1 or 0 in the received bit stream. The decision circuit makes a decision on 1 or 0 by comparing the sampled value with a threshold value  $I_D$ . In the presence of receiver noise for bit 1 an error occurs when the decision circuit detects for  $I_1 < I_D$  and vice-versa. Thus, the bit-error probability is defined as

$$\text{BER} = P(1)P_{e_1} + P(0)P_{e_0} \quad 6.25$$

where  $P(1)$  and  $P(0)$  are the probabilities of receiving bits 1 and 0 respectively.  $P_{e_1}$  and  $P_{e_0}$  are the probabilities of error given 1 and 0 are transmitted respectively.

$P(1) = P(0) = 1/2$  since 1's and 0's are equally likely and the BER becomes

$$\text{BER} = \frac{1}{2} [P_{e_1} + P_{e_0}] \quad 6.26$$

$p(I|0)$  and  $p(I|1)$  is the probability density function of the sampled value  $I$  conditioned on a 0 and a 1 is transmitted respectively. In section 2.1.1 in (2.1)  $i_T(t)$  is the thermal noise described by Gaussian statistics with zero mean and variance  $\sigma_T^2$ . The statistics of shot-noise  $i_s(t)$  in (2.1) is also approximately Gaussian for both p-i-n and APD receivers [1]. Since the sum of two Gaussian random variables is also a Gaussian random variable, the sampled value of  $I$  has a Gaussian probability density function with variance  $\sigma^2 = \sigma_T^2 + \sigma_s^2$ . The variance  $\sigma_1^2$  and  $\sigma_0^2$  is the variance of noise depending on the bit (1 or 0) received. The probability of error given 0 is transmitted is given by

$$P_{e_0} = \int_{I_D}^{\infty} p(I|0) dI = \frac{1}{2} \text{erfc} \left( \frac{I_D - I_0}{\sigma_0 \sqrt{2}} \right) \quad 6.27$$

Similarly probability of error given 1 is transmitted is

$$P_{e_1} = \int_{-\infty}^{I_D} p(I|1) dI = \frac{1}{2} \text{erfc} \left( \frac{I_D - I_1}{\sigma_1 \sqrt{2}} \right) \quad 6.28$$

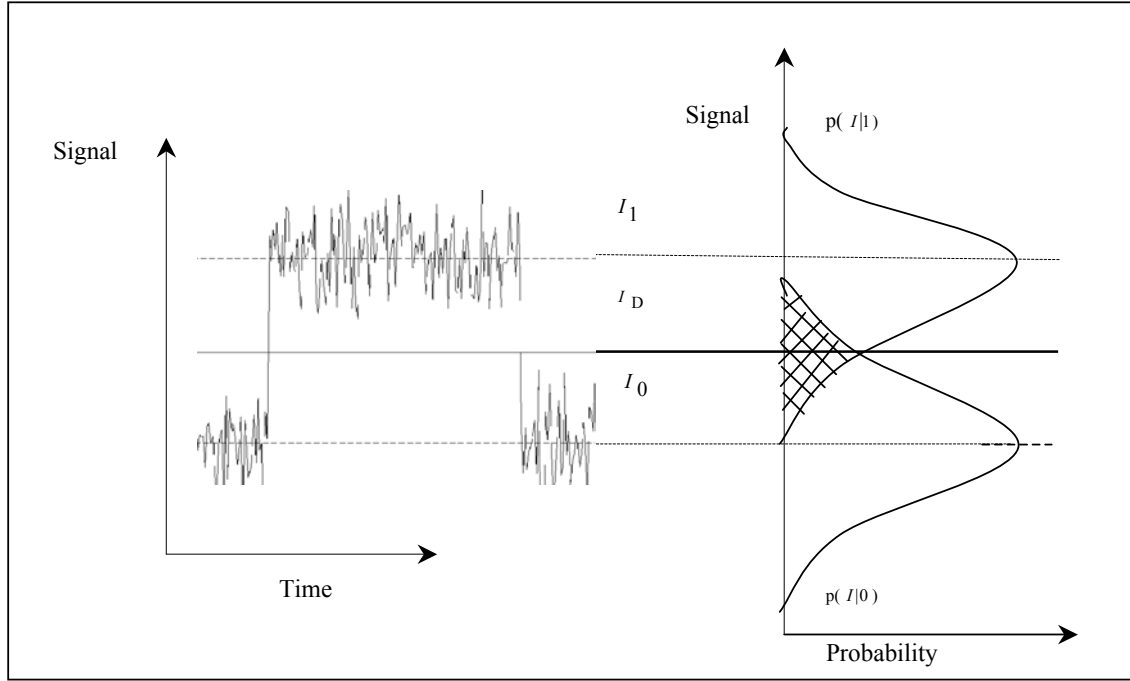
where erfc is the complementary error function defined as

$$\text{erfc}(x) = \frac{2}{\sqrt{\pi}} \int_x^{\infty} \exp(-y^2) dy$$

$$\therefore \text{BER} = \frac{1}{4} \left[ \text{erfc} \left( \frac{I_D - I_1}{\sigma_1 \sqrt{2}} \right) + \text{erfc} \left( \frac{I_D - I_0}{\sigma_0 \sqrt{2}} \right) \right] \quad 6.29$$

Thus, the BER depends on the decision threshold  $I_D$ . In practice,  $I_D$  is optimized to minimize the BER. The minimum occurs when  $I_D$  is such that

$$\frac{I_1 - I_D}{\sigma_1} = \frac{I_D - I_0}{\sigma_0} = Q \quad 6.30$$



**Figure 6.7 Fluctuating Signal at the Receiver**

The shaded region shows the probability of incorrect detection<sup>1</sup>

For most p-i-n receivers  $\sigma_1 = \sigma_0$  where noise is dominated by thermal noise ( $\sigma_T \gg \sigma_s$ ) then

$$Q = \frac{I_1 - I_0}{2\sigma_1} \text{ and } I_D = \frac{I_1 + I_0}{2}$$

Thus, the BER with the setting of the decision threshold in the middle is given by

$$\therefore \text{BER} = \frac{1}{2} \left[ \text{erfc} \left( \frac{Q}{\sqrt{2}} \right) \right] \quad 6.31$$

Also with  $\sigma_1 = \sigma_0$  and  $I_1 = -I_0$

$$Q = \frac{I_1}{\sigma_1}$$

$$\therefore Q^2 = \frac{I_1^2}{\sigma_1^2} = \frac{E_b}{N_o/2} = \frac{2E_b}{N_o} \quad 6.32$$

where  $E_b$  is the energy per bit  
 and  $N_o/2$  is the variance of the noise

$$\therefore \text{BER} = \frac{1}{2} \left[ \text{erfc} \left( \frac{\sqrt{2E_b/N_o}}{\sqrt{2}} \right) \right] \quad 6.33$$

6.33 has the same form as 3.1 for probability of bit-error of BPSK.

<sup>1</sup> G.P.Agrawal, Fiber-Optic Communication Systems

From 6.32 we have

$$Q^2 = \frac{2E_b}{N_o} = 2 * \text{SNR} \quad 6.34$$

$$\therefore Q(\text{dB}) = 3(\text{dB}) + \text{SNR}(\text{dB})$$

6.31 can be used to calculate the minimum optical power required by the receiver to operate reliably below a specified BER. It is shown [1] that the Q factor is directly proportional to the average received power  $\bar{P}_{\text{rec}}$ . The BER improves as Q increases since the erfc function is a decreasing function of its argument.

## 6.4 Hardware Implementation of Galois Field Arithmetic

The encoding/decoding schemes for the BCH/RS codes involve addition and multiplication over finite fields. Such finite field operations can be realized by logic gates and flip-flops and shift-registers. The systolic array structure mentioned below can be used to implement multiplication and division of two polynomials, multiplication of two elements of GF ( $2^m$ ) and computation of the inverse of a field element [10]. FEC has widely been used in low speed applications, however at speeds beyond 10 Gbits/s their implementation becomes extremely challenging due to excessive complexity and power consumption [18]. Since, the finite field arithmetic is the underlying operation in the encoding/decoding algorithms, efforts have been made to perform them efficiently [34], [35], [39]. The low overhead constraint, typically 7 to 25% for 10 Gbits/s data rates, requires high-speed electronics with bandwidth from 10.7 to 12.5 GHz [45].

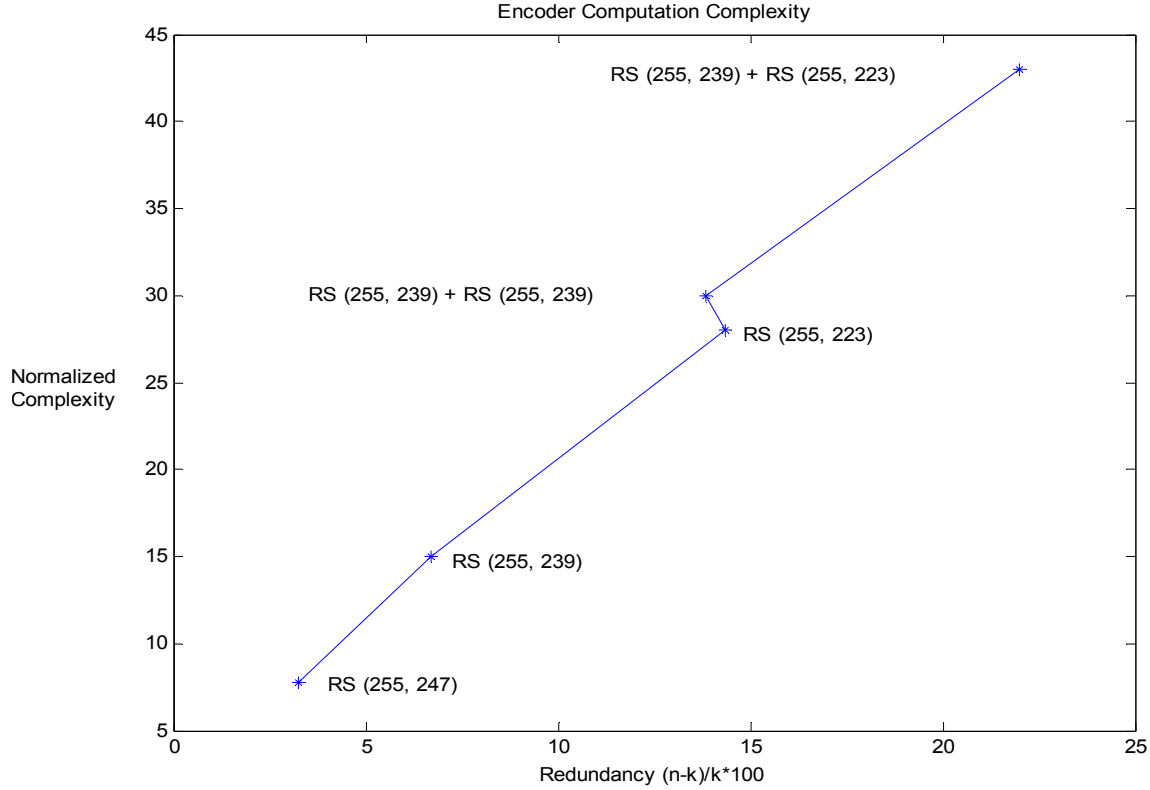
### 6.4.1 Encoder Architecture

The BCH and RS codes are constructed by the polynomial division method in time domain, which requires multiplication in GF ( $2^m$ ). The structure of the encoder is shown in [10]. The circuit complexity of the structure depends on the multiplier used. For the parallel-type multiplier, the circuit complexity could be as high as  $O(m^2t)$ . However, using the Berlekamp's bit serial dual basis multiplier, the complexity can be reduced to  $O(mt)$  [21]. An encoder with a similar complexity can also be obtained using the triangular basis multiplier [35]. The structure of the RS encoder in frequency domain can be achieved by rewriting 5.6 in the iterative form

$$V = (\dots(V_{n-1}\alpha^{-i} + V_{n-2})\alpha^{-i} + \dots + V_1)\alpha^{-i} + V_0 \quad 6.35$$

where  $V_j$  are the coordinates of  $V(Z)$

Such as structure is called a systolic array [37]. The computational complexity of the polynomial division method and the transform method is the same [10]. However, the transform technique is advantageous when the number of information symbols  $k$  is variable. It has a circuit complexity of  $O(mn)$  [10]. Although the structure has a higher circuit complexity than a structure based on polynomial division method, it is more suitable as a scalable encoder since its operation does not depend on  $t$ . The computational complexity of the encoder involves  $k$  multiplications for the multiplier, at most  $k(n-k)$  multiplications for determining  $b(X)$  and at most  $k$  additions to obtain  $V(X)$  [13]. Hence, the total computational complexity is at most  $k + k(n-k)$  multiplications and  $k$  additions over GF ( $q$ ). Next, we analyze the computational complexity of the RS codes presented in section 8.1 and 8.2. Since  $k(n-k) \gg k$ , from figure 6.8 it is evident that the computational complexity of the encoder increase linearly with the redundancy in the code.



**Figure 6.8 Encoder Computational Complexity vs. Redundancy<sup>⊥</sup>**

### 6.4.2 Decoder Architecture

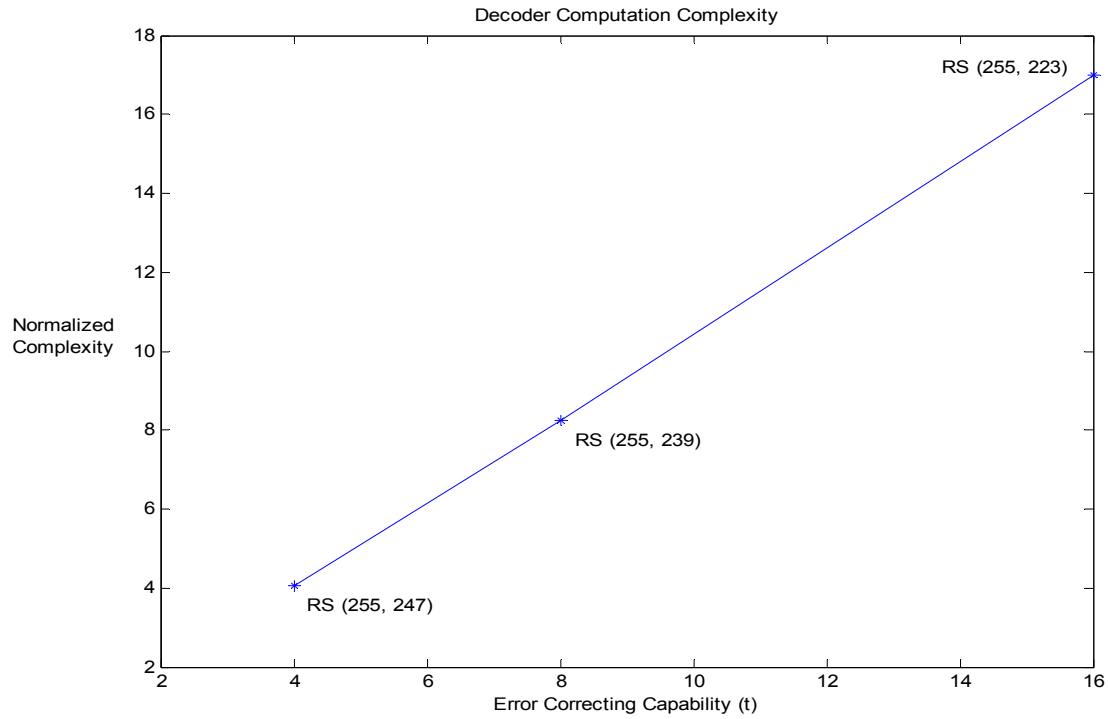
The first step in decoding is to compute the  $2t$  syndromes. 5.11 is re-expressed in the iterative form

$$S_i = (\dots(r_{n-1}\alpha^i + r_{n-2})\alpha^i + \dots + r_1)\alpha^i + r_0 \text{ for } i = 1, 2, \dots, 2t \quad 6.36$$

The computation of one syndrome requires  $(n-1)$  additions and  $(n-1)$  multiplications [7]. It can be shown that  $s_{2i} = s_i^2$  and hence  $2t$  syndrome components can be computed with  $(n-1)t$  additions and  $nt$  multiplications [7]. From (5.3), we notice  $g(X)$  is a product of at most  $t$  minimal polynomials, therefore, at most  $t$  feedback shift-registers, each consisting of at most  $m$  stages, are needed for LFSR implementation. The second step in algebraic or transform decoding is to find the  $\sigma(X)$ . If the Berlekamp algorithm with  $t$  iterations (refer section 5.4.1) is used then both the software as well as hardware implementation requires at most  $t$  additions and  $t$  multiplications to compute each  $\sigma^{(u)}(X)$  and  $d_u$  and since there are  $t$  of each, the total is at most  $2t^2$  additions and multiplications respectively [7]. The speed of hardware implementation depends on how much is done in parallel. The last step is computing error location numbers  $\beta_i$ , which requires substituting the  $2^m$  field elements into  $\sigma(X)$  of degree  $t$  to determine its roots. This requires  $nt$  additions and multiplications respectively [7]. The hardware implementation can be realized using Chien's search circuit [25]. The algebraic BCH decoder based on Berlekamp algorithm requires at most  $2t(n+t)$ ; for  $n \gg 1$  additions and multiplication respectively. The same holds true for the RS decoder except the Berlekamp-Massey algorithm with  $2t$  iterations is used to find  $\sigma(X)$ .

<sup>⊥</sup> Normalized w.r.t  $n$

A fast shift-register based implementation of the Berlekamp algorithm for BCH/RS has a circuit complexity of  $O(n^2)$ . The Euclid algorithm requires at most  $4t^2$  additions and multiplications respectively over GF (q) [13], which is the same required by the Berlekamp–Massey algorithm for the RS codes. For the BCH codes, Berlekamp algorithm with  $2t^2$  computations is efficient than the Euclid algorithm. The Euclid’s algorithm has a circuit complexity of  $O(n^2)$  [10].



**Figure 6.9 Decoder Computational Complexity vs. Error Correcting Capability<sup>⊥</sup>**

<sup>⊥</sup> Normalized w.r.t. n

# Chapter 7

## 7 Concatenated Coding

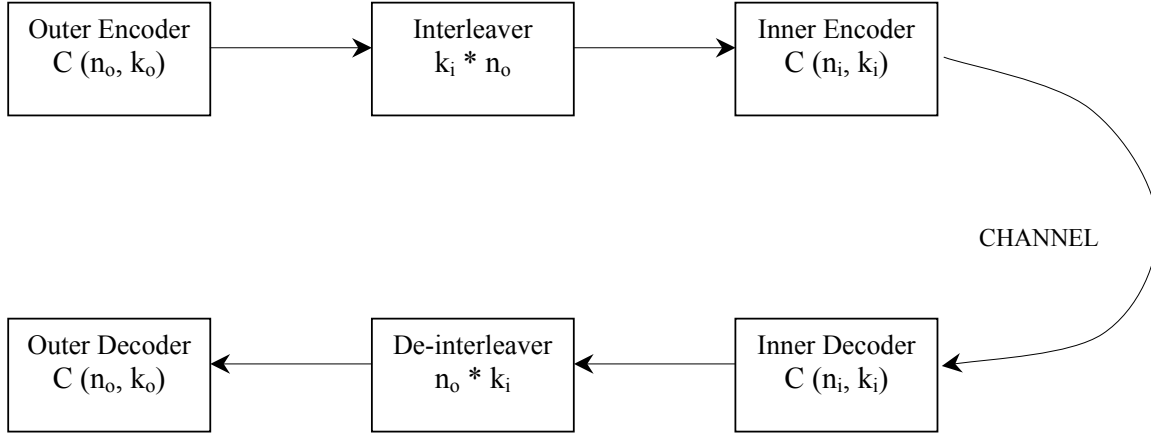
In this chapter, we present the basic principle behind concatenated coding technique. The idea behind concatenated coding [5] is to construct powerful error correcting codes by concatenating two or more codes called *component codes* separated by an *interleaver*. The resultant code is decoded component-wise with reasonable decoding complexity. The performance of a concatenated code depends upon the interleaver size and its structure and the type of component code used. We have considered hard decision decoding for the concatenated codes with RS codes as component codes and hard decision as well as soft decision decoding for the concatenated codes with BCH codes as the component codes. The hard decision decoder for the RS and BCH component codes is the bounded-distance algebraic decoder based on the Berlekamp–Massey algorithm and the Berlekamp algorithm respectively (refer section 5.4.1 and 5.4.2). The soft-input/soft-output (SISO) decoder for the soft decision decoding of the concatenated code with BCH codes as component codes is based on the Chase algorithm using the usual algebraic decoder.

### 7.1 Concatenated Coding Strategies

It is well known fact that long code-length codes yield powerful codes [49]-[50], but at the same time, the decoding complexity increases with the code length. A typical concatenated code consists of a cascade of an outer and an inner code where the outer code is the usual RS code and the inner code is the convolutional code [5]. The optimum decoder for the resultant code is the ML decoder, which is highly complex, however the two component decoders form a sub-optimal decoder for the resultant code with low decoding complexity with respect to the ML decoder. The concatenated strategy in figure 7.1 is known as *Serial Concatenation*. The role of the interleaver in case of serial concatenation is to randomize the errors due to the inner decoder. Another strategy based on concatenation of the component codes in parallel through an interleaver [23] is known as *Parallel Concatenation*. Here the interleaver is an integral part of the resultant code. When the component codes are block codes in both the concatenation<sup>1</sup> schemes then the resultant code is called *Product Code* [27] or *Serial Concatenated Block Code* (SCBC) and *Parallel Concatenated Block Code* (PCBC) [20] respectively. If the convolutional codes are used as component codes then it is termed as *Serial Concatenated Convolutional Code* (SCCC) and *Turbo Code* [23] or *Parallel Concatenated Convolutional Code* (PCCC) [20] respectively.

---

<sup>1</sup> The concatenated codes are defined with respect to a row-column interleaver of depth D and span N

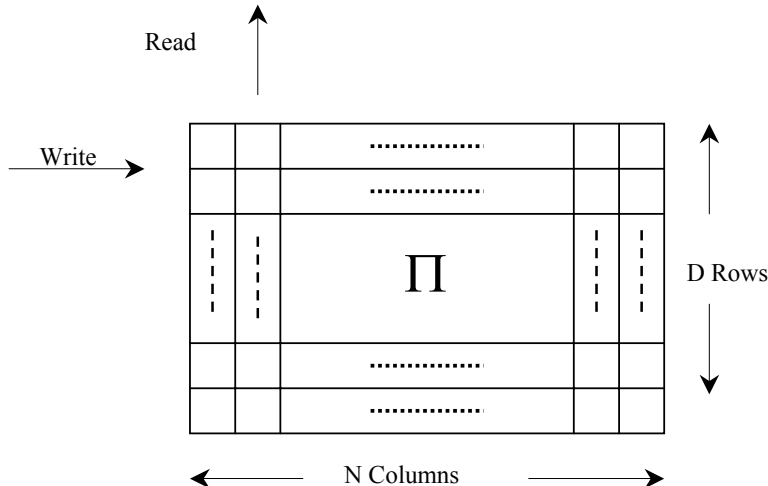


**Figure 7.1 Serial Concatenated Coding Scheme**

In general concatenated code based on convolutional codes is called *Convolutional Turbo Code* (CTC) and that on block codes is called *Block Turbo Code* (BTC) [42]. In the next section we present the row-column interleaver and in section 7.3 we justify the use of BTC with serial concatenation strategy for error control coding in optical communication system.

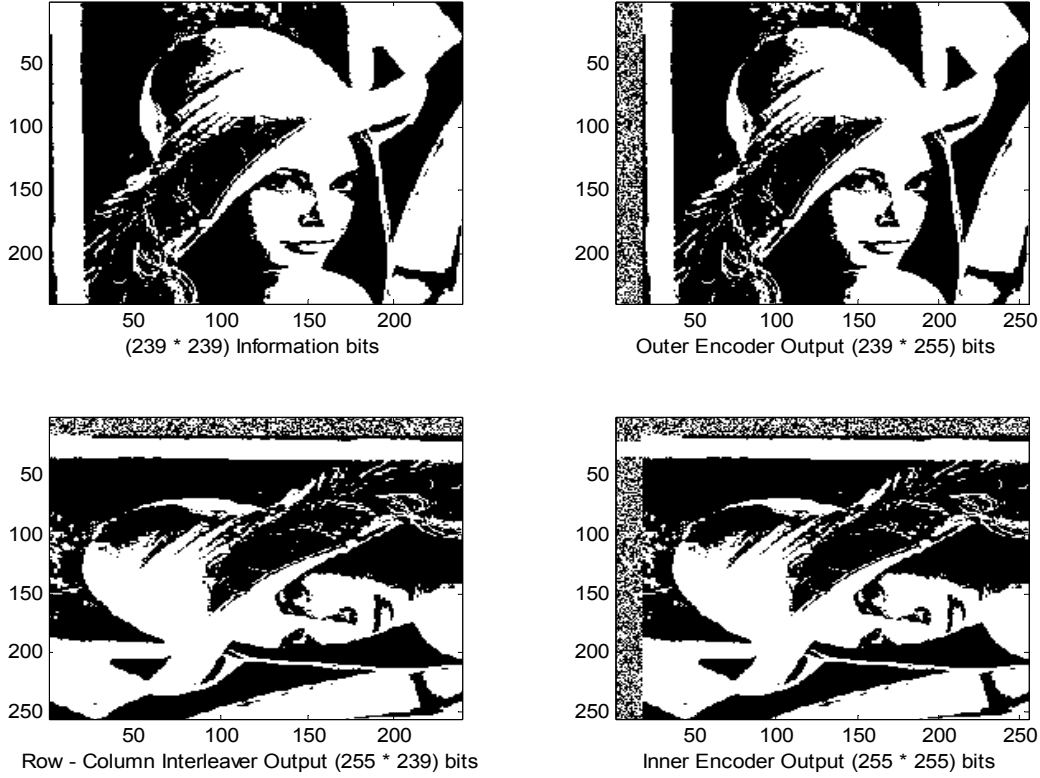
## 7.2 Interleaver

Interleaving is a process of rearranging the ordering of a symbol sequence in a one-to-one deterministic format [11]. The inverse of this process is deinterleaving which restores the received sequence to its original order. A  $D \times N$  *block* or *row-column interleaver* is characterized by two parameters, viz., the depth denoted by  $D$  and the span of the interleaver denoted by  $N$ . The input sequence is written into the matrix row-wise and read out column-wise in an uniform fashion as shown in figure 7.2. The end-to-end interleaver-deinterleaver delay is  $2ND$  and the memory requirement is  $ND$  for the interleaver and deinterleaver. There exist other interleaver structures like random, helical etc [11]. Since the rows and columns of product codes are independent [40] the extrinsic information on the different rows (or columns) is independent and random. Thus, nonuniform interleaving does not provide any significant performance improvement[44] whereas nonuniform interleaving plays an important role for optimum performance of CTC [23]. Since there is no gain from nonuniform interleaving in BTC [44], we have selected the simple uniform row-column interleaver for serial concatenation of the component block codes. The BTC that is apparently the SCBC with RS codes as component codes is presented in section 7.4.1 and the BTC based on BCH codes is presented in section 7.4.2.



**Figure 7.2 Row-Column Interleaver**

Serial Concatenation of BCH (255, 239) & BCH (255, 239) code



**Figure 7.3 BCH (255, 239) + BCH (255, 239) Concatenated Code**

### 7.3 Block Turbo Codes

Earlier we have seen that the low overhead constraint is the prime criterion for selecting error control codes for optical communication systems that means the selected code should have high code rate. It is shown in [44] that the BTC is more efficient than CTC for high code rates. CTC will not be discussed any further. Consider two block codes  $C^1(n_1, k_1, d_{\min 1})$  and  $C^2(n_2, k_2, d_{\min 2})$  concatenated serially as well as in parallel. Let  $C_s$  and  $C_p$  denote the resultant concatenated code. The code rate of the serially concatenated code  $C_s$  is the product of the code rates of the inner and outer codes respectively and is given by

$$R_s = R_1 * R_2 \quad 7.1$$

The Hamming distance of  $C_s$  is also equal to the product of the Hamming distances of  $C^1$  and  $C^2$  respectively and is given by

$$d_{\min s} = d_{\min 1} * d_{\min 2} \quad 7.2$$

In the case of PCBC the resultant code rate and minimum distance of  $C_p$  is given by

$$R_p = \frac{R_1 R_2}{R_1 + R_2 - R_1 R_2} \quad 7.3$$

$$d_{\min p} = d_{\min 1} + d_{\min 2} - 1 \quad 7.4$$

The asymptotic coding gain of the concatenated codes assuming ML decoding is

$$(G_a)_i = 10 \log_{10} (R_i * d_{\min i}) \quad i = s, p \quad 7.5$$

For any combination of linear block codes, the serial concatenation yields a higher asymptotic coding gain [44] and is shown in table 7.1. It can be noted that as the code length increases, for a given minimum distance  $(G_a)_s - (G_a)_p$  increases. Experimental results show that the performance of the two coding schemes are within  $\pm 0.1$  dB at a BER of  $10^{-5}$  [32],[40]. The lower value of  $(G_a)_p$  in case of PCBC means that the asymptotic coding gain is reached at a higher BER as compared to  $(G_a)_s$  attained at much lower BER. The requirement of output BER in optical communication systems is of the order of  $\leq 10^{-12}$ . From the above comparison, even though the code rates of SCBC and PCBC are comparable, it is evident the choice of SCBC becomes appropriate for lower BER applications. Henceforth, we refer to the SCBC presented in the report as BTC or product codes.

|                | Serial Concatenation (SCBC)<br>BCH (63, 57) + BCH (63, 57) | Parallel Concatenation (PCBC)<br>BCH (63, 57) + BCH (63, 57) |
|----------------|------------------------------------------------------------|--------------------------------------------------------------|
| Code Rate      | 0.818                                                      | 0.826                                                        |
| Redundancy (%) | 22.16                                                      | 21.05                                                        |
| $d_{\min}$     | 9                                                          | 5                                                            |
| $G_a$ dB       | 8.67                                                       | 6.16                                                         |

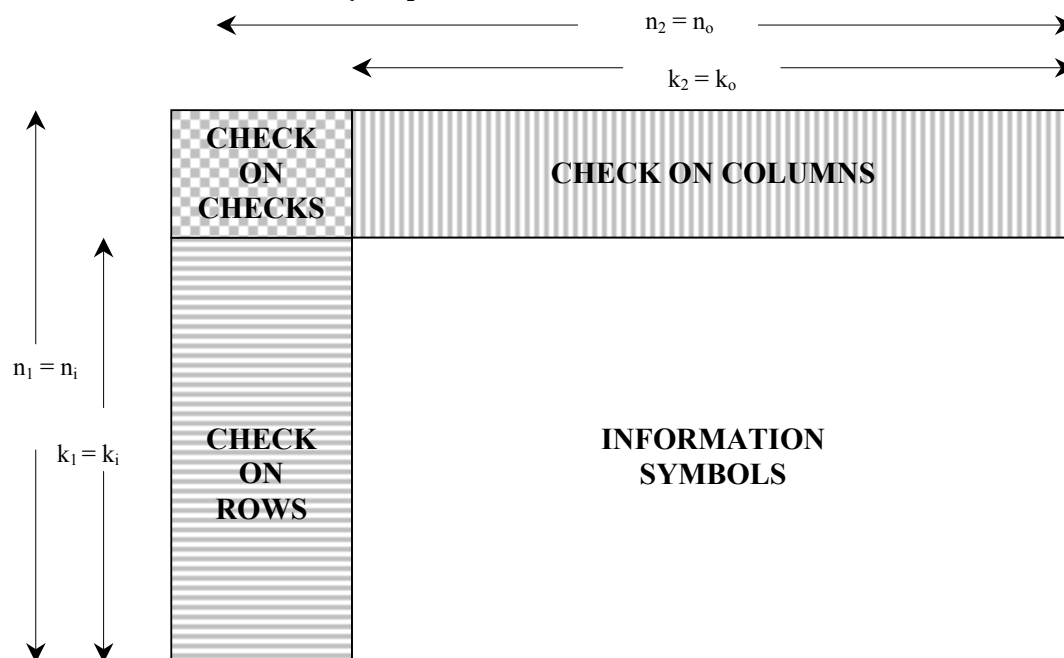
**Table 7.1**

## 7.4 Product Codes

Product codes (or iterated codes) [27], [8] are serially concatenated codes using two or more short block codes to form long block codes. If  $C^1(n_1, k_1, d_{\min 1})$  and  $C^2(n_2, k_2, d_{\min 2})$  are two systematic linear block codes, then the product code  $P = C^1 \otimes C^2$  is obtained by

- ◆ placing  $(k_1 * k_2)$  information symbols in a matrix of  $k_1$  rows and  $k_2$  columns
- ◆ coding the  $k_1$  rows using code  $C^2$
- ◆ coding the  $n_2$  columns using code  $C^1$

The resultant product code [8]  $P(n, k, d_{\min})$  has  $n = n_1 * n_2$ ,  $k = k_1 * k_2$ ,  $d_{\min} = d_{\min 1} * d_{\min 2}$  and code rate is given by  $R = R_1 * R_2$ .



**Figure 7.4 Construction of Product Code**

Referring to figure 7.4 it is evident that the first  $(n_2 - k_2)$  columns of the matrix P are code words of  $C^1$  and the first  $(n_1 - k_1)$  rows of matrix P are code words of  $C^2$  [8]. If  $t_1 = \lfloor (d_{\min 1} - 1)/2 \rfloor$  and  $t_2 = \lfloor (d_{\min 2} - 1)/2 \rfloor$  are the random error correcting capabilities of the component codes respectively, then the maximum random error correcting capability of the product code P is

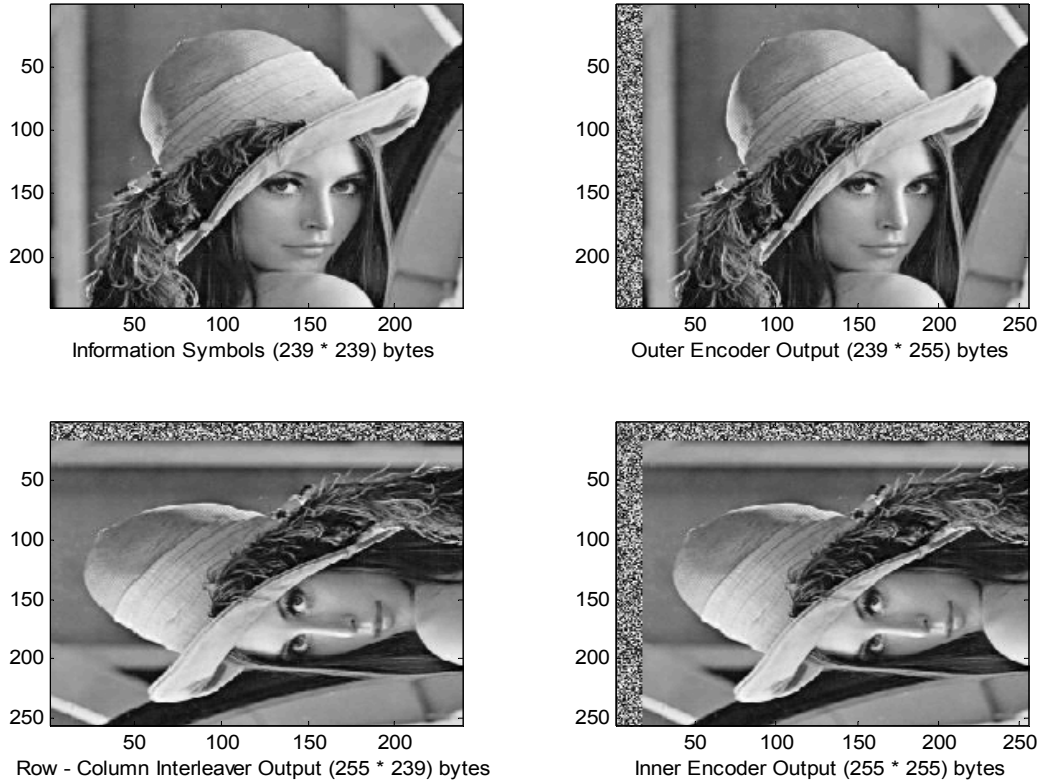
$$t = \lfloor (d_{\min} - 1)/2 \rfloor = 2t_1t_2 + t_1 + t_2 \quad 7.6$$

Since the decoding involves two-step (rows after column or vice-versa) procedure, it is incapable of correcting all the error patterns with  $t$  or fewer errors in the code matrix P. However, such a decoding process is rather simple and efficient. Another advantage of product codes is that they have an inherent burst-error handling capability.

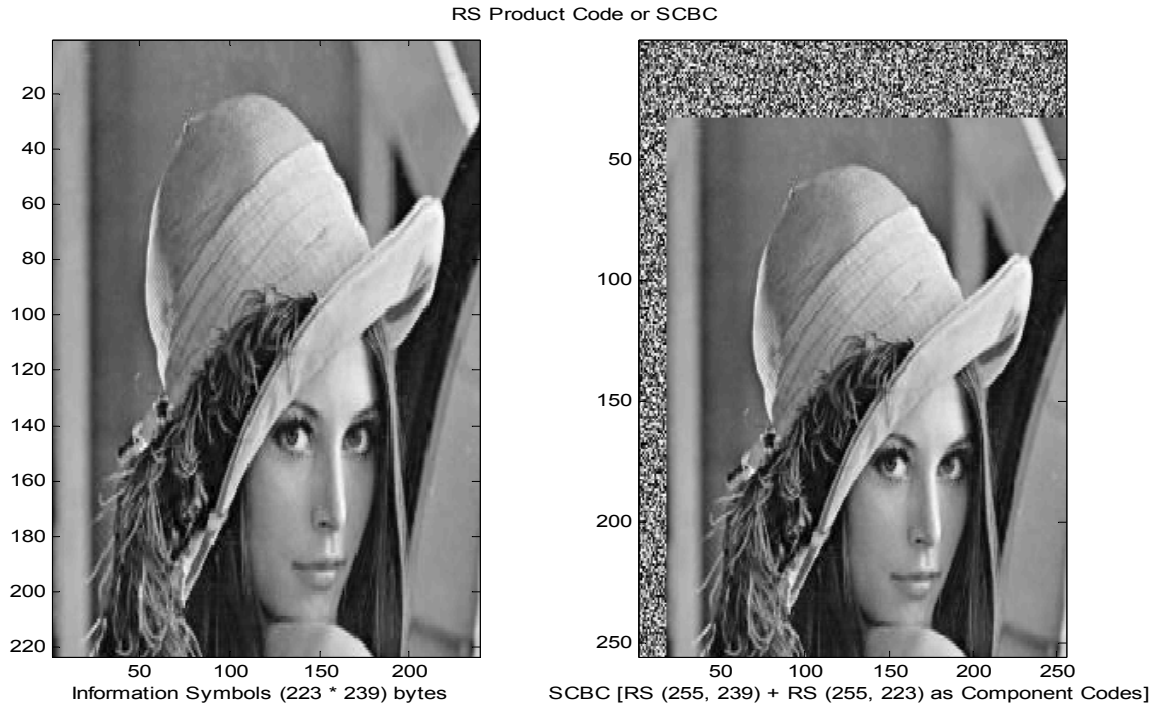
#### 7.4.1 RS Product Codes

In our work we have used the classical method described in section 7.4 for the construction of RS product codes, where the information symbol matrix contains  $k_1 * k_0$  q-ary information symbols. The codes  $C^1$  and  $C^2$  presented in section 8.2 have the same code length  $n_1 = n_0$ . The resultant code design scheme is easy to understand by following figure 7.1 where the span N of the interleaver is equal to the code length  $n_0$  of the outer code and the depth D is equal to the information length  $k_1$  of the inner code. Figure 7.5 shows the serially concatenated RS (255, 239) and RS (255, 239) code with a row-column interleaver of length 255 bytes and depth 239 bytes. Figure 7.6 depicts the serially concatenated RS (255, 239) and RS (255, 223) with a row-column interleaver of length 255 bytes and depth 223 bytes. The output of the inner encoder is transposed to show that SCBC with row-column interleaver is apparently the product code discussed in section 7.4.

Serial Concatenation of RS (255, 239) & RS (255, 239) code



**Figure 7.5 Serial Concatenation of RS Codes**



**Figure 7.6 RS Product Code**

The rows and columns of matrix  $P$  are code words  $v = (v_1, v_2, \dots, v_{n_i}) \forall i = 1, 2$  of  $C^2$  and  $C^1$  respectively. The code word  $v$  is transmitted using BPSK signaling over the AWGN channel such that  $q$ -ary symbols are mapped to the  $m$ -tuple format and then  $(0 \rightarrow -1)$  and  $(1 \rightarrow +1)$ . The received word

$$r' = v + e \quad 7.7$$

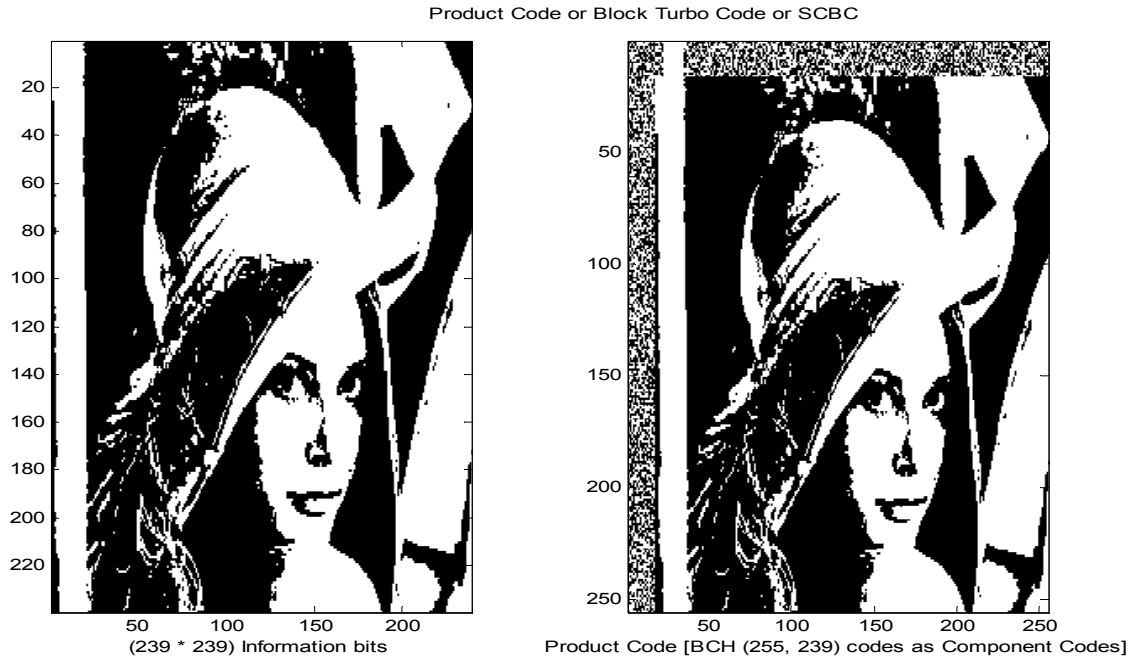
where  $e = (e_1, e_2, \dots, e_n)$  is an AWGN process with variance  $N_0/2$

The ML decoding is optimum decoding (refer section 1.3) for the product codes such that the optimum decoder selects  $\hat{v} = v_j$  iff

$$|r' - v_j|^2 < |r' - v_i|^2 \quad ; \forall i \neq j \quad 7.8$$

$$|r' - v_i|^2 = \sum_{l=1}^n (r_l - v_{il})^2 ; \forall i = 1, 2, \dots, q^k \quad 7.9$$

is the squared Euclidean distance between  $r'$  and  $v_j$ . The optimum decoder will perform an exhaustive search for the code word  $\hat{v}$ , but the decoding complexity increases exponentially with  $k$ . In our work the product codes based on RS component codes are decoded by sequentially decoding the rows and columns of  $P$  by hard-decision algebraic bounded-distance decoder (refer section 4.4) which is sub-optimal but has significant reduced decoding complexity compared to the optimum ML decoder. We have simulated the different RS product codes on a Gaussian channel with sequential row by column hard-input/hard-output (HIHO) component decoders using the Berlekamp-Massey algorithm. Soft-decision decoding of the component RS codes [15] with SISO decoders will definitely provide additional coding gain but the decoding complexity will be extremely high. Another limiting factor is the high data rate at which optical communication systems operate. Implementing the SISO decoders at such a high data rate is impracticable. The computer simulation results are presented in chapter 8 section 8.2. Further iterated decoding of RS product codes is performed using the component HIHO decoders.



**Figure 7.7 BTC  $(255, 239)^2$**

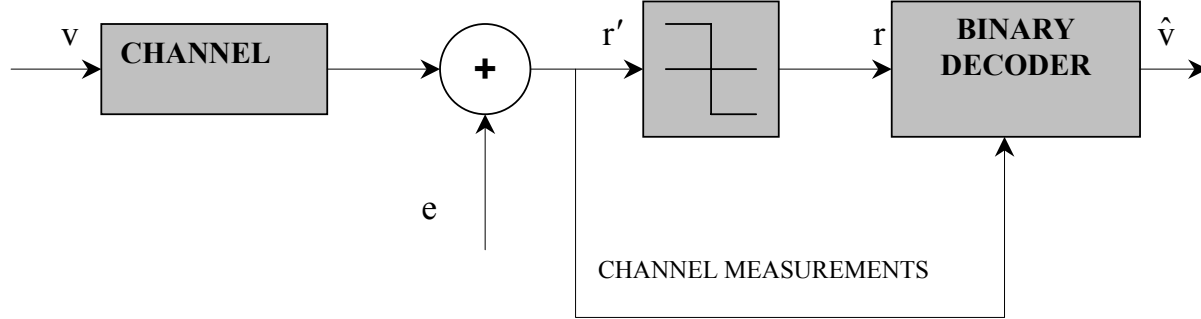
#### **7.4.2 BCH Product Codes**

In our work we have used the classical method described in section 7.4 for the construction of BCH product codes, where the information symbol matrix contains  $k_1 \times k_2$  information bits from GF (2) where  $k_1 = k_2 = 57, 113$  and  $239$  respectively. The codes  $C^1$  and  $C^2$  presented in (chapter 8 section 8.3) have the same code length  $n_1 = n_2 = 63, 127$  and  $255$  respectively. The parameters of the different product codes  $P$  is presented in table 8.7. The BTC based on BCH component codes are sequentially decoded row (column) by column (row) of  $P$  using SISO decoder based on Chase Algorithm 2 [24]. A near optimum performance can be achieved if we iterate the sequential soft-decoding of  $P$  [42] thus reducing BER after each iteration. Iterated decoding of BTC based on HIHO is of no interest because it is sub-optimal than SISO decoding. At low SNR values for the HIHO decoding, it was observed from the simulations that instead of reducing the errors with each half iteration, the decoding errors committed by one component decoder affected the other resulting in more errors. Therefore, iterated decoding of BTC with HIHO decoders is not pursued in our work.

The MAP algorithm [19] performs ML decoding and gives soft-output for each bit. It is optimal at each decoding stage to decode the components codes, however the decoding complexity is prohibitive. The SISO decoder proposed in [32] based on the trellis description of the block codes provide good performance. However, the solution is limited to small block codes since the number of states in the trellis of a block code increases exponentially with the number of redundant bits. In our work we have used the SISO decoder whose soft output is an estimate of the log-likelihood ratio (LLR) of the binary hard output of the Chase decoder[42]. In section 7.4.3 we explain the soft-decoding of the linear block codes and in sections 7.4.4 and 7.4.5 we describe an iterative decoding algorithm for any product code built using the linear block codes.

### 7.4.3 Soft-Decoding of Linear Block Codes<sup>1</sup>

The optimum ML decoder will perform an exhaustive search for the code word  $\hat{v}$  such that the squared Euclidean distance between  $r'$  and  $v_j$  is minimum. The decoding complexity increases exponentially with  $k$  and becomes prohibitive for block codes with  $k > 6$  [6].



**Figure 7.8 Chase Decoder**

For the BTC presented the exhaustive search for  $\hat{v}$  is not a realistic solution. We use the Chase algorithm 2 [24], which is sub-optimal having low complexity compared to optimum ML decoding. The main feature of the Chase algorithm is that, at high SNR, the ML codeword  $\hat{v}$  is located in the sphere of radius  $(d_{\min}-1)$  centered on  $r = (r_1, r_2, \dots, r_n)$  where  $r_i \in \{0,1\}$  with very high probability. The search for  $\hat{v}$  can be limited to those code words lying in the sphere of radius  $(d_{\min}-1)$  centered on  $r$  using the channel measurements  $r'$  as shown in figure 7.8. The procedure used to identify the set of the most probable code words is explained below

- ◆ Determine the position of the  $p = \left\lfloor \frac{d_{\min}}{2} \right\rfloor$  least reliable bits of  $r$  using  $r'$ . The reliability of bits  $r_i$  of  $r$  is defined using the LLR of the channel measurements  $r'_i$  of  $r'$ . The LLR is a real number representing the soft decision out of a demodulator and given as

$$\begin{aligned}
 L(v | r'_i) &= \ln \left[ \frac{\Pr(v = +1 | r'_i)}{\Pr(v = -1 | r'_i)} \right] = \ln \left[ \frac{p(r'_i | v = +1) \Pr(v = +1)}{p(r'_i | v = -1) \Pr(v = -1)} \right] \\
 L(v | r'_i) &= \ln \left[ \frac{p(r'_i | v = +1)}{p(r'_i | v = -1)} \right] + \ln \left[ \frac{\Pr(v = +1)}{\Pr(v = -1)} \right] \\
 L(v | r'_i) &= L(r'_i | v) + L(v) \\
 L'(\hat{v}) &= L_c(r') + L(v)
 \end{aligned} \tag{7.10}$$

where  $L(r'_i | v)$  is the LLR of the channel measurements  $r'_i$  of  $r'$  under the conditions that  $v_i = +1$  or  $v_i = -1$  may have been transmitted, and  $L(v)$  is the a priori LLR of the coded bits  $v_i \forall i = 1, 2, \dots, n$ . Under the assumption of equally likely occurrence of the coded bits  $-1$  and  $+1$  yields an initial a priori LLR value of  $L(v) = 0$ . The LLR of the channel measurements  $r'_i$  [53] is given by

$$L(r'_i | v) = \ln \left[ \frac{p(r'_i | v = +1)}{p(r'_i | v = -1)} \right] = \frac{2}{(N_o / 2)} r'_i \tag{7.11}$$

<sup>1</sup> The equations in subsection 7.4.3 are reproduced from [42] for understanding the soft-decoding procedure

where  $p(r'_i | v)$  is the conditional pdf of  $r'_i$ . If we normalize the LLR with respect to  $2/(N_o/2)$  the relative reliability of  $r_i$  is then given by  $|r'_i|$

- ◆ Form test patterns  $T^q \forall q = 1, 2, \dots, 2^p - 1$ , defined as all the combination of sequences with at least a “1” in the  $p$  least reliable positions
- ◆ Decode  $r$  and  $Z^q = r \oplus T^q$  using the usual algebraic decoder. Store the set of code words  $v_j \forall j = 1, 2, \dots, 2^p$
- ◆ The ML code word  $\hat{v}$  is then given by applying decision rule 7.8 with the subset of code words  $v_j \forall j = 1, 2, \dots, 2^p$ . It is to be noted that the components of the code words are mapped from  $\{0, 1\}$  to  $\{-1, +1\}$  before computing the squared Euclidean distance in 7.9.

The Chase algorithm yields for each row (column) the decision  $\hat{v}$  of the component block code for a given input received word  $r$ . The decoding of the product code is achieved by iterating the soft-decoding procedure with maximum efficiency. In order to do so we must now compute the reliability of the decisions given by the Chase algorithm before decoding the column (row) of  $P$ . In order to generate soft decoder output, we have to compute the reliability of each of the bits of  $\hat{v}$ . For a systematic code, it is shown [23] that the LLR (soft output) out of the decoder is equal to

$$L(\hat{v}) = L'(\hat{v}) + L_e(\hat{v}) \quad 7.12$$

where  $L'(\hat{v})$  is the LLR of a data bit out of the demodulator (input to the decoder) and  $L_e(\hat{v})$ , called the extrinsic LLR, represents extra knowledge that is gleaned from the decoding process. From (7.10 and 7.12)

$$L(\hat{v}) = L_c(r') + L(v) + L_e(\hat{v}) \quad 7.13$$

The soft decision  $L(\hat{v})$  is a real number that provides a hard decision as well as the reliability of that decision. The sign of  $L(\hat{v})$  denotes the hard decision and the magnitude of  $L(\hat{v})$  denotes the reliability of that decision.

The reliability of decision  $\hat{v}_i$  is defined using the LLR of the transmitted bits  $v_i$  [42] and is given by

$$L(\hat{v}_i | r') = \ln \left[ \frac{\Pr(v_i = +1 | r')}{\Pr(v_i = -1 | r')} \right] \quad 7.14$$

Here the computation of LLR differs from 7.10 such that we must take into account the fact that  $\hat{v}$  is one of the  $2^k$  code words of  $C(n, k)$ . Thus, by considering the different code words of  $C(n, k)$ , the numerator of (7.14) can be written as

$$\Pr(v_i = +1 | r') = \sum_{v^j \in S_i^{+1}} \left( \Pr\{v = v^j | r'\} \right) \forall i = 1, 2, \dots, n \text{ \& } j = 1, 2, \dots, 2^k \quad 7.15$$

where  $S_i^{+1}$  is the set containing the index of code words  $\{v^j\}$  such that  $v_i^j = +1$ . Similarly, the denominator of (7.14) can be written as

$$\Pr(v_i = -1 | r') = \sum_{v^j \in S_i^{-1}} \left( \Pr\{v = v^j | r'\} \right) \forall i = 1, 2, \dots, n \text{ \& } j = 1, 2, \dots, 2^k \quad 7.16$$

where  $S_i^{-1}$  is the set containing the index of code words  $\{v^j\}$  such that  $v_i^j = -1$ .

By applying Bayes rule to 7.15 and 7.16 and assuming that the different code words are uniformly distributed, we obtain for  $L(\hat{v}_i | r')$  the following expression

$$L(\hat{v}_i | r') = \ln \left[ \frac{\sum_{v^j \in S_i^{+1}} p\{r' | v = v^j\}}{\sum_{v^j \in S_i^{-1}} p\{r' | v = v^j\}} \right] \quad 7.17$$

where

$$p(r' | v = v^j) = \left( \frac{1}{\sqrt{\pi N_o}} \right)^n \exp \left( -\frac{|r' - v^j|^2}{N_o} \right) \quad 7.18$$

is the probability density function of  $r'$  conditioned on  $v$ . This function decreases exponentially with the Euclidean distance between  $r'$  and  $v^j$ .

It is proved [42] that the approximated LLR (soft output) out of the decoder is equal to

$$L(\hat{v}_i) = \frac{2}{(N_o/2)} \left( r'_i + \sum_{l=1, l \neq i}^n r_l v_l^{+1(i)} p_l \right) \quad 7.19$$

where

$$p_l = \begin{cases} 0, & \text{if } v_l^{+1(i)} = v_l^{-1(i)} \\ 1 & \text{if } v_l^{+1(i)} \neq v_l^{-1(i)} \end{cases}$$

The approximated LLR can be normalized with respect to  $2/(N_o/2)$  and we obtain the following equation

$$r_i'' = r'_i + w_i \quad 7.20$$

with

$$w_i = \sum_{l=1, l \neq i}^n r_l v_l^{+1(i)} p_l$$

The normalized LLR  $r_i''$  is taken as the soft output of the decoder. It has the same sign as  $\hat{v}_i$  and its absolute value indicates the reliability of the decision. 7.20 indicate that  $r_i''$  is given by the soft-input  $r'_i$  plus  $w_i$  which is a function of the two code words at minimum Euclidean distance from  $r'$  and  $\{r_l\}$  with  $l \neq i$ . The term  $w_i$  plays the same role as the extrinsic LLR  $L_e(\hat{v})$  in CTC [23]. The extrinsic information is a random variable with a Gaussian distribution as shown in figure 7.9. The extrinsic LLR plays a very important role in the iterative decoding of product codes since it is fed back to the decoder input, to serve as a refinement of the a priori value for the next iteration.

#### 7.4.4 Soft-Input Soft-Output Decoder

The computation of the exact LLR of  $\hat{v}_i$  using (7.14-7.17) is tedious and in 7.20 we have given an expression for the approximated normalized LLR of  $\hat{v}_i$ . Next, we explain the computation of the approximated normalized LLR in 7.20. Computing the reliability of decision  $\hat{v}_i$  at the output of the Chase decoder requires two code words (refer 7.18) from the set of code words  $v_j \forall j=1,2,\dots,2^p$  obtained by the Chase algorithm. Obviously, binary decision  $\hat{v}$  is one of these two code words and we must find the second one, which we shall call  $\tilde{v}$ .  $\tilde{v}$  can be

viewed as a competing code word of  $\hat{v}$  at minimum Euclidean distance from  $r'$  with  $\hat{v}_i \neq \tilde{v}_i$ . Given the code words  $\hat{v}$  and  $\tilde{v}$ , it can be shown that the soft output is given by the following equation [42]

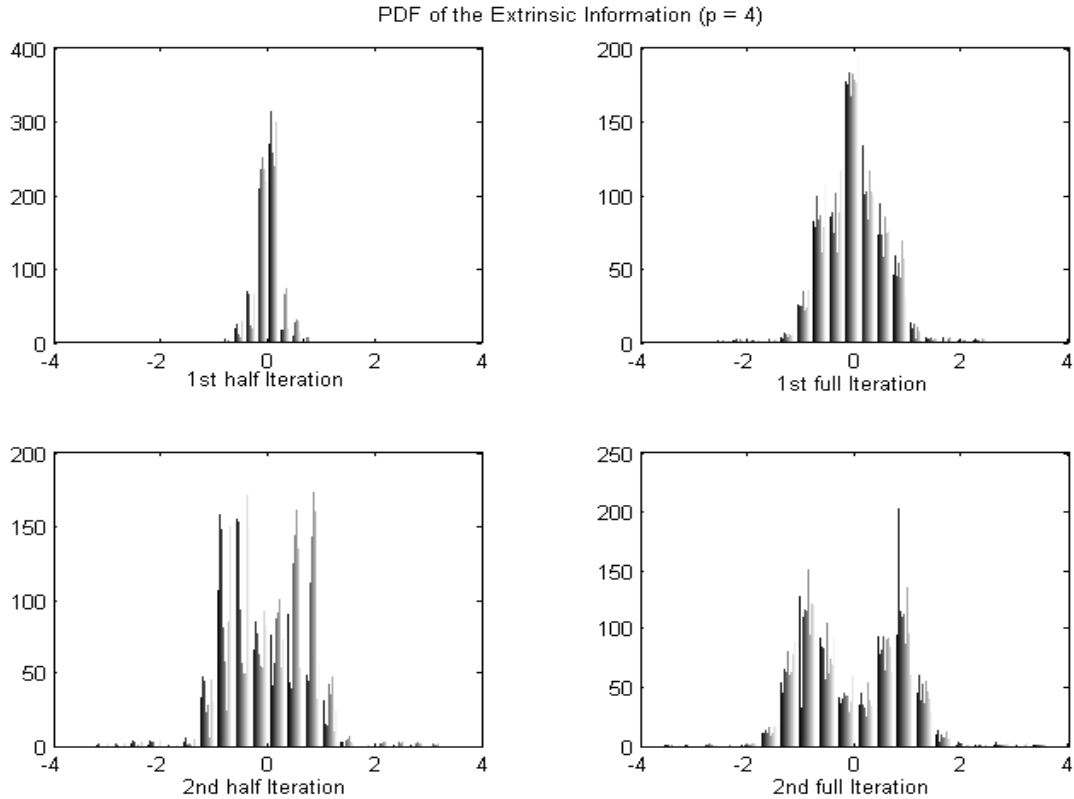
$$r_i'' = \left( \frac{|r' - \tilde{v}|^2 - |r' - \hat{v}|^2}{4} \right) \hat{v}_i \quad 7.21$$

To find code word  $\tilde{v}$ , one must increase the size of the space scanned by the Chase algorithm, which is achieved by increasing the number of least reliable bits  $p$  used to form the test patterns  $T^q$ . The probability of finding  $\tilde{v}$  increases with the value of  $p$  but at the same time the decoder complexity increases exponentially with  $p$  and we must trade-off complexity vs. performance. It is possible in some cases that we are not able to find the competing code word  $\tilde{v}$  and we must use another method for computing the soft output. We use the solution proposed in [43] such that

$$r_i'' = \beta * \hat{v}_i \quad \text{with } \beta \geq 0 \quad 7.22$$

This rough approximation of the soft output is justified by the fact that

- ◆ The sign of soft output  $r_i''$  is equal to  $\hat{v}_i$  while only its absolute value or reliability is a function of  $\tilde{v}$ .
- ◆ If  $\tilde{v}$  is not found in the space scanned by the Chase algorithm, then  $\tilde{v}$  is most probably far from  $r'$  in terms of Euclidean distance.
- ◆ If  $\tilde{v}$  is very far from  $r'$  then the probability that decision  $\hat{v}_i$  is correct is relatively high and the reliability of  $\hat{v}_i$  is relatively high.



**Figure 7.9 PDF of Extrinsic Information**

An equation is given in [15] for computing  $\beta$ . In fact  $\beta$  can be considered as an average value of the reliability of those decisions  $\hat{v}_i$  for which there is no competing code word  $\tilde{v}$ , while 7.21 gives a bit-by-bit estimation of the reliability. It is clear that the soft output given by 7.22 is less accurate than the one using 7.21. However, it is obvious a more accurate estimation of the soft output is necessary for those decisions where the competing code word  $\tilde{v}$  is at a slightly greater distance from  $r'$  than  $\hat{v}$ . When  $\tilde{v}$  is very far away from  $r'$  an average value of the reliability can be considered as sufficient.

#### 7.4.5 Turbo Decoding of Product Code

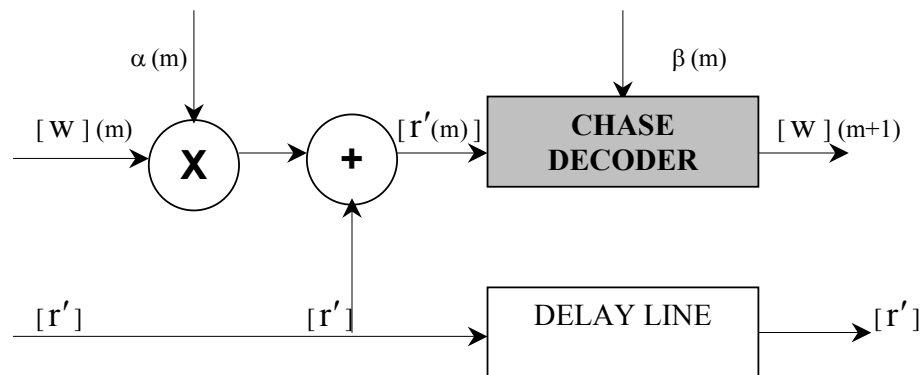
On receiving the matrix  $[r']$  corresponding to code words  $v$  of the transmitted matrix  $P$ , the elementary decoder as shown in figure 7.10 performs the soft-decoding of the rows (columns) of  $P$ . Soft-input decoding is performed using the Chase algorithm and soft-output is computed using 7.21 and 7.22. By subtracting the soft-input from the soft-output, we obtain the extrinsic LLRL<sub>e</sub>( $\hat{v}$ ) =  $[w]$ . The soft input for the decoding of the columns (rows) at the second step decoding of  $P$  is given by

$$[r'(m)] = [r'] + \alpha(m)[w(m)] \quad 7.23$$

The elementary decoder computes the soft-output  $[r''(m)]$  and delivers at its output

$$[w(m+1)] = [r''(m)] - [r'(m)] \quad 7.24$$

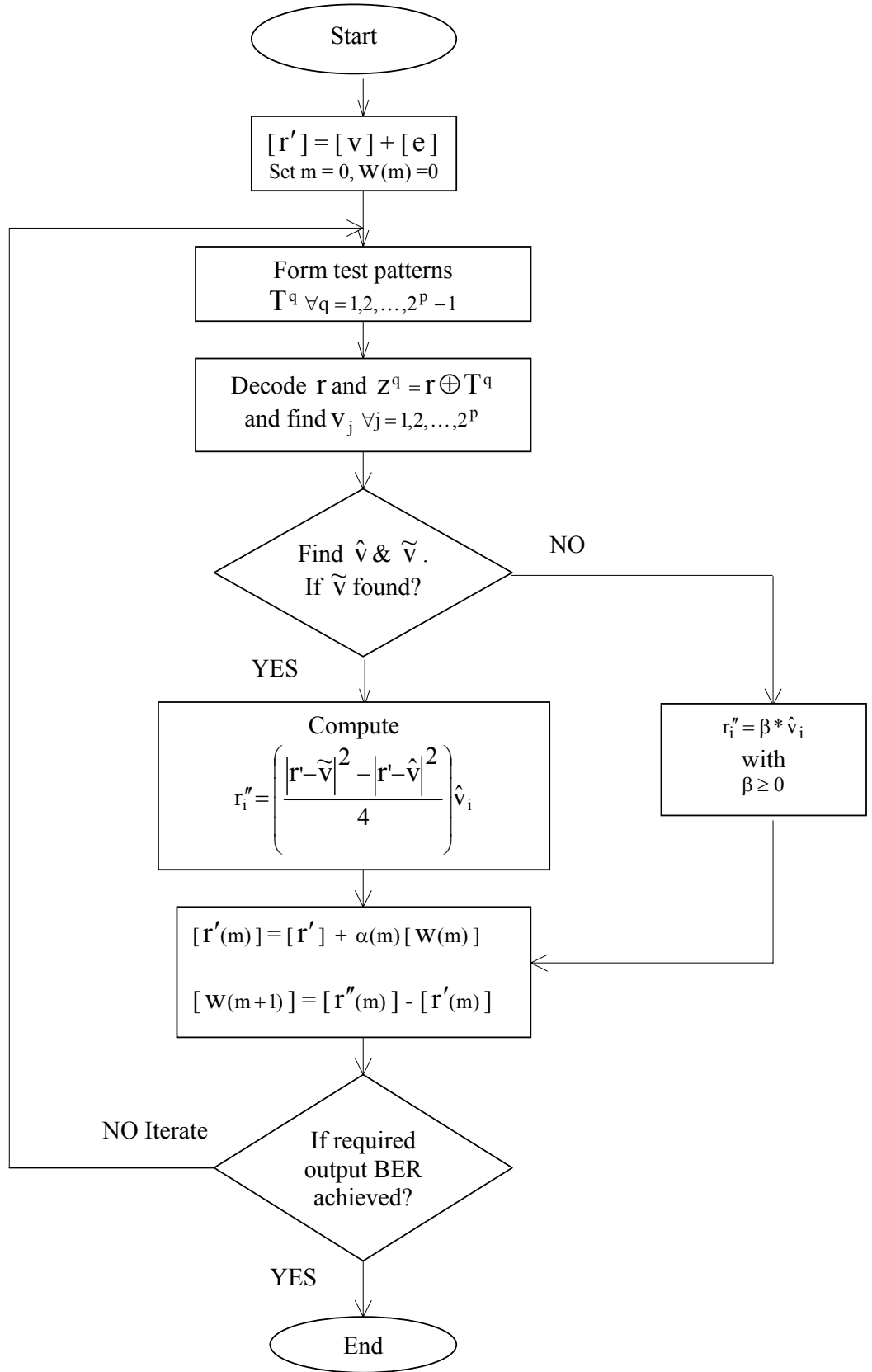
$\alpha$  is a scaling factor that takes into account the fact that the standard deviation of samples in matrix  $[r']$  and  $[w]$  are different.



**Figure 7.10 Block Diagram of Elementary Block Turbo Decoder<sup>⊥</sup>**

We conclude this chapter by summarizing the iterative decoding procedure with the help of the flow chart depicted in figure 7.11. One full iteration of the turbo decoder corresponds to a row followed by a column decoding of the matrix of the product code.

<sup>⊥</sup> Reproduced from [43] for convenience



**Figure 7.11 Flow Chart for Turbo Decoding**

# Chapter 8

## 8 Theoretical and Simulated Results

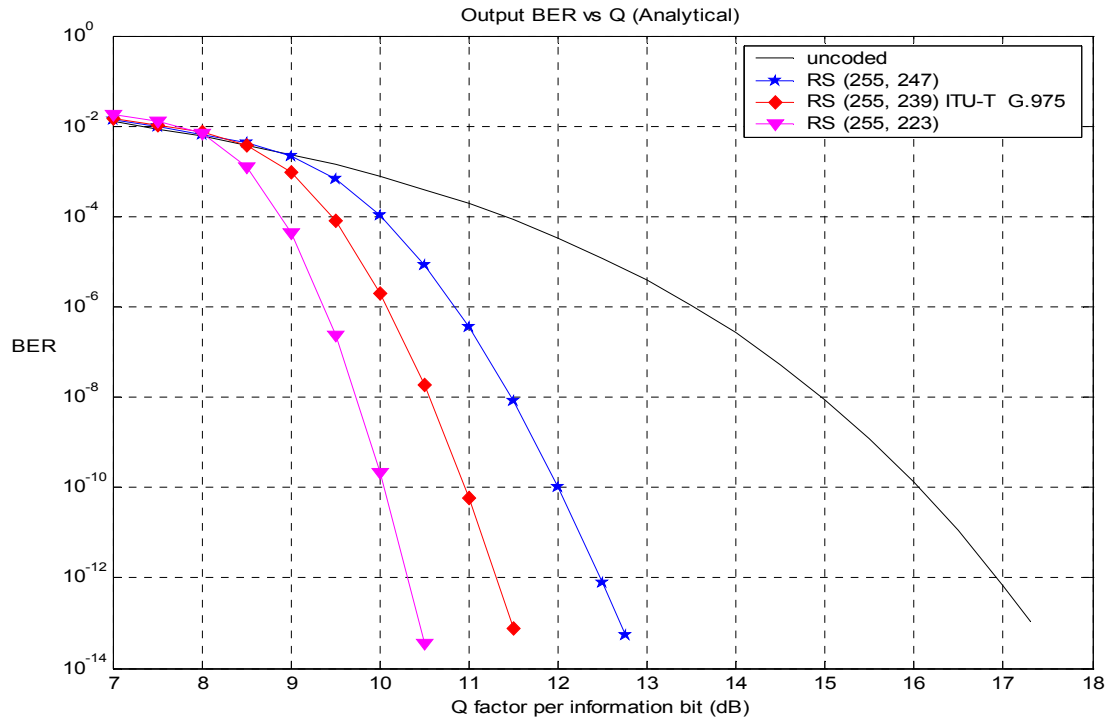
We have performed the Monte-Carlo simulation on the AWGN channel with BPSK signaling for the different codes discussed in chapters 5 and 7. In section 8.1, we present the performance of different RS codes in terms of coding gain and output BER. The performance of the RS product codes is presented in section 8.2 while that of BTC based on BCH codes is presented in section 8.3.

### 8.1 Performance of RS Codes

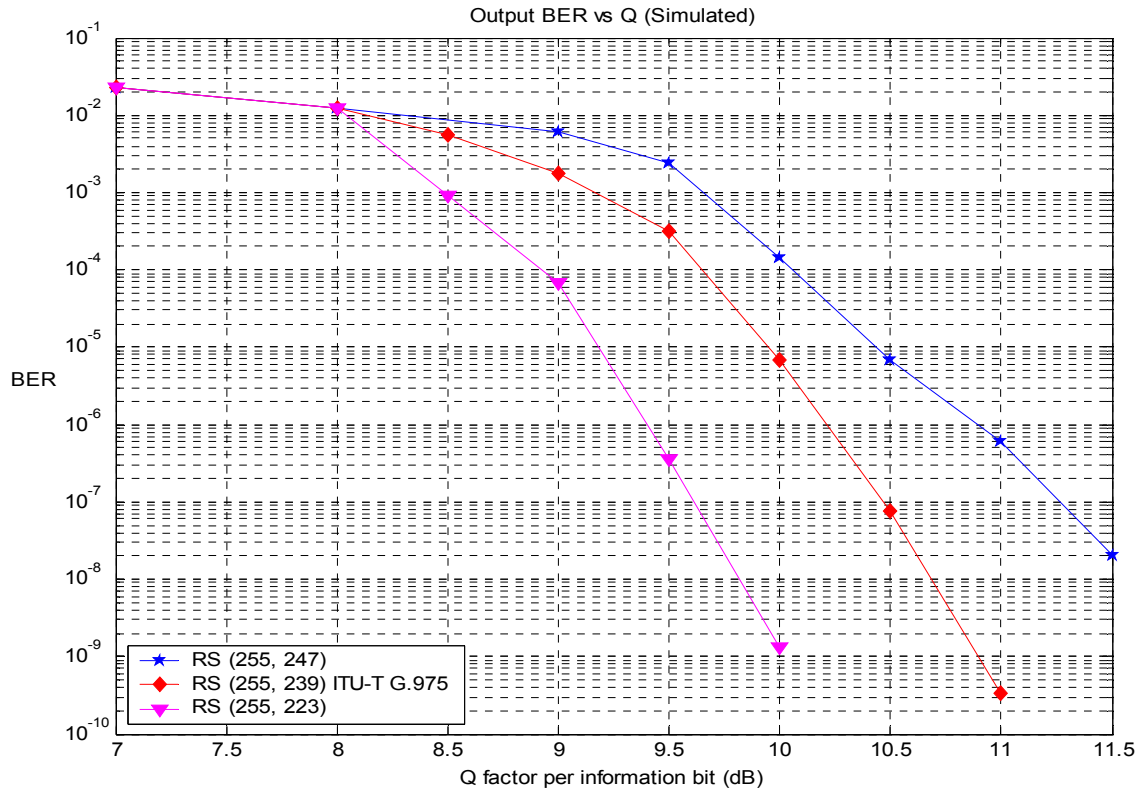
The different RS code parameters are presented in table 8.1. From table 8.1 we observe as the code rate decreases the redundancy in the code, the minimum distance of the code and the asymptotic coding gain increases. The code rate and redundancy are inversely proportional to each other. Since it is difficult to calculate the WEF of the candidate RS codes, we have used the approximation given by (6.21, 6.22 and 6.23) to compute the analytical output BER. The validity of the approximation was confirmed for the reference RS (31, 25) code in section 6.2. We extend the approximation for the RS codes presented in table 8.1 to compute the approximate BER analytically. The analytical results for the candidate RS codes are presented in figure 8.1. The Monte-Carlo simulations were performed piece-wise for an output BER of  $\leq 10^{-8}$  for the candidate RS codes. The simulated results presented in figure 8.2 are confirmed with the analytical results in figure 8.1. The simulated curves presented in figure 8.2 are extrapolated for an output BER of  $\leq 10^{-12}$ . These extrapolated curves are presented in figure 8.3. Net electrical coding gain (NECG) is commonly used to quantify FEC performance and indicates an improvement in the SNR or Q factor at the receiver due to FEC. The comparison of the performance of the candidate RS codes in terms of coding gain is presented in table 8.2. The coding gain comparison of the candidate RS codes for an output BER of  $\leq 10^{-8}$  is obtained from figure 8.1 and figure 8.2 whereas that for an output BER of  $\leq 10^{-12}$  is obtained from figure 8.1 and figure 8.3.

| Candidate RS (n, k)<br>Codes | Code Rate<br>(k/n) | Redundancy<br>(n-k)/k (%) | $d_{\min}$<br>( $2*t+1$ ) | $G_a$ (dB)<br>Asymptotic |
|------------------------------|--------------------|---------------------------|---------------------------|--------------------------|
| RS (255, 247)                | 0.9686             | 3.24                      | 9                         | 9.4                      |
| RS (255, 239)                | 0.9372             | 6.69                      | 17                        | 12.02                    |
| RS (255, 223)                | 0.8745             | 14.35                     | 33                        | 14.6                     |

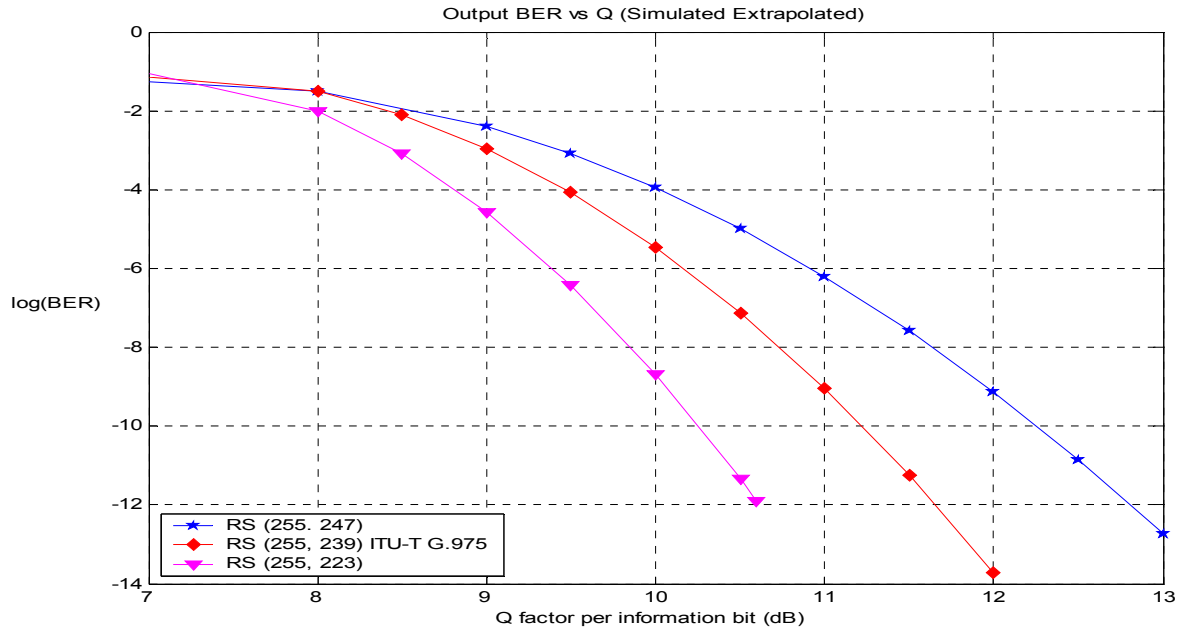
**Table 8.1 RS Code parameters**



**Figure 8.1 Theoretical Performance of the RS Codes**



**Figure 8.2 Simulated Performance of the RS Codes**



**Figure 8.3 Simulated Performance of the RS Codes (Extrapolated to  $\leq 10^{-12}$ )**

It can be noticed from table 8.2 the increased redundancy in the code pays in terms of NECG. The RS (255, 239) code with almost twice the redundancy compared to the RS (255, 247) code offers roughly more than 1 dB additional coding gain. Approximately 2 dB additional coding gain is offered by the RS (255, 223) code with almost 4.5 times more redundancy than the RS (255, 247) code. The theoretical curve for the ITU-T G.975 standard in figure 8.1 is confirmed with [36] while the simulated curve presented in figure 8.2 is confirmed with [16]. The simulated results for RS (255, 223) and RS (255, 247) are in coherence with [15] and [18] respectively.

| Candidate RS (n,k) Codes | Redundancy (n-k)/k (%) | NECG (dB) @ $\text{BER} \leq 10^{-8}$ |             | NECG (dB) @ $\text{BER} \leq 10^{-12}$ |                          |
|--------------------------|------------------------|---------------------------------------|-------------|----------------------------------------|--------------------------|
|                          |                        | (Theoretical)                         | (Simulated) | (Theoretical)                          | (Sim.Ext. <sup>⊥</sup> ) |
| RS (255, 247)            | 3.24                   | 3.5                                   | 3.37        | 4.46                                   | 4.2                      |
| RS (255, 239)            | 6.69                   | 4.42                                  | 4.28        | 5.63                                   | 5.3                      |
| RS (255, 223)            | 14.35                  | 5.25                                  | 5.15        | 6.62                                   | 6.33                     |

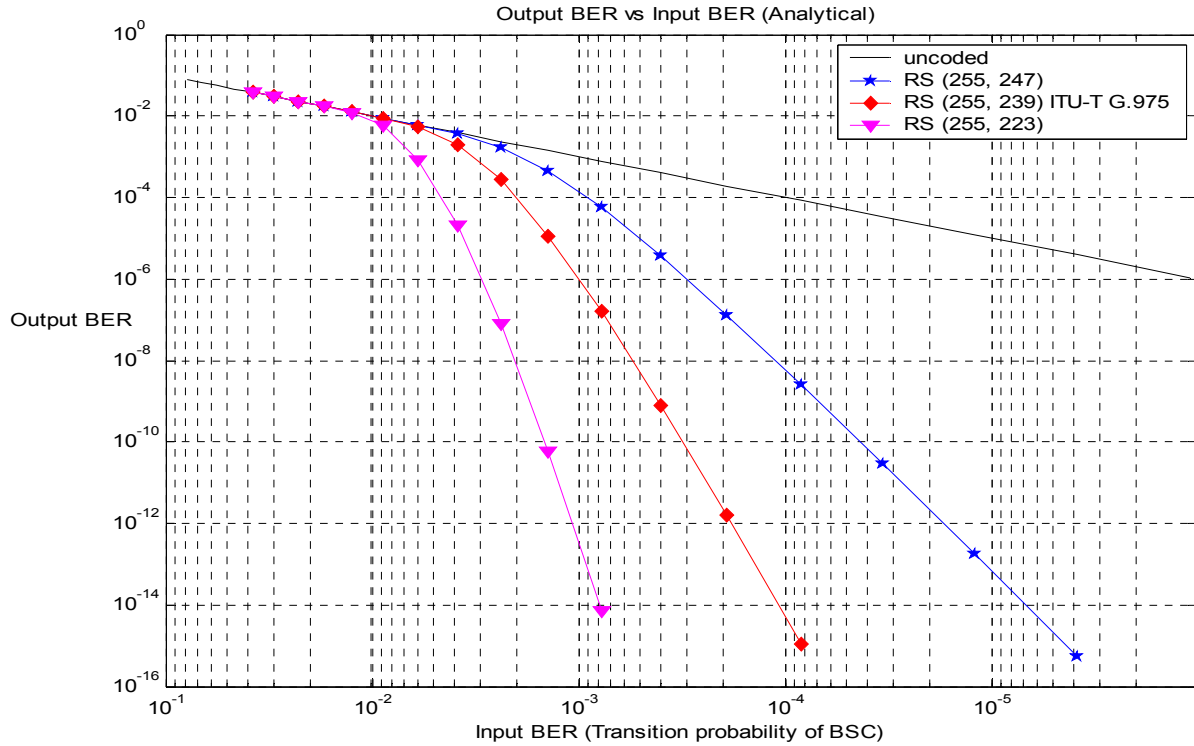
**Table 8.2 Coding Gain Comparison of the RS Codes at Output BER of  $10^{-8}$  and  $10^{-12}$**

The (theoretical and simulated) performances of the candidate RS codes in terms of output BER vs. input BER are depicted in figures 8.4 and 8.5 respectively. The output BER performance of the candidate codes at an input BER of  $10^{-3}$  is presented in table 8.3. From table 8.3 it can be observed that for a fixed value of input BER the output BER decreases as the redundancy in the code increases. For the ITU-T standard with almost twice the redundancy in the RS (255, 247) code the output BER is reduced approximately by a factor of  $10^{-2}$ . The output BER (theoretical) of  $5 \cdot 10^{-15}$  for input BER of  $10^{-4}$  for the ITU-T G.975 standard is confirmed with [36].

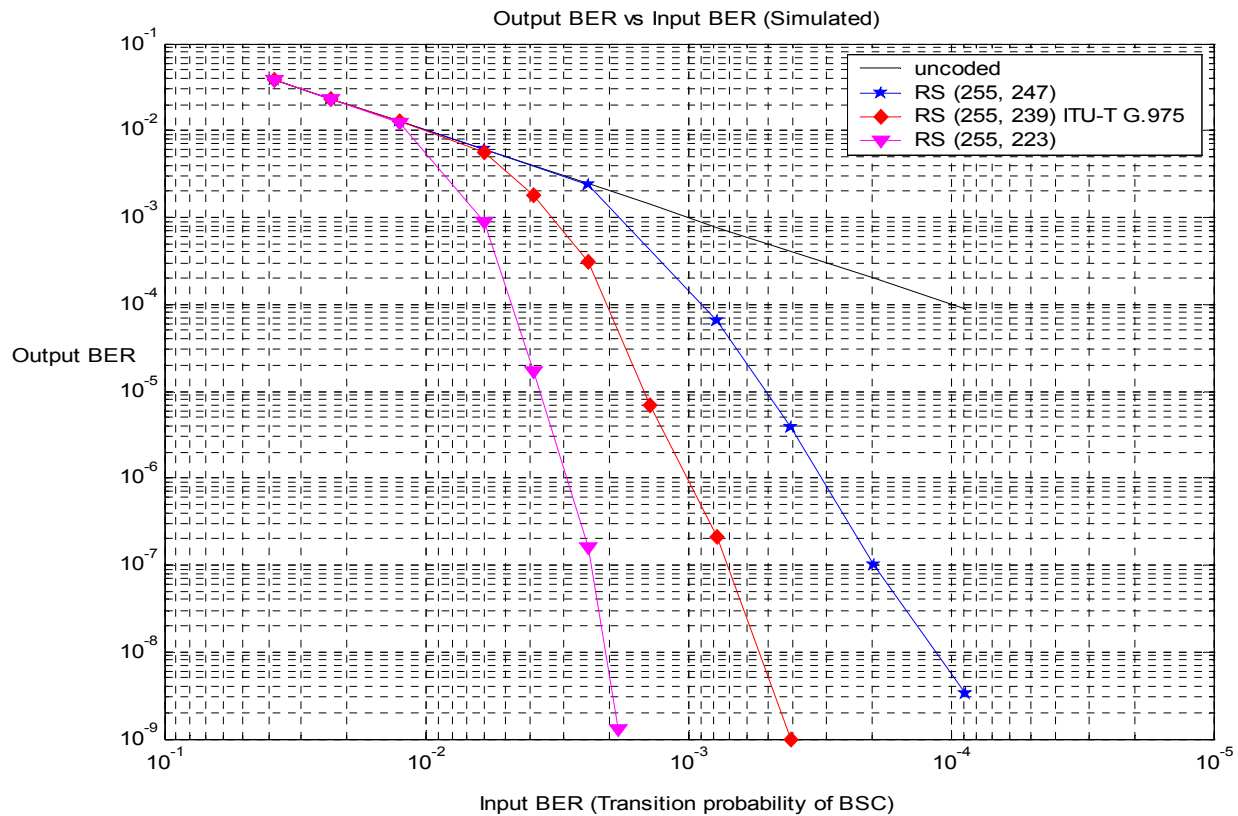
| Candidate RS (n, k) Codes | Redundancy (n-k)/k (%) | Output BER (Theoretical) | Output BER (Simulated) |
|---------------------------|------------------------|--------------------------|------------------------|
| RS (255, 247)             | 3.24                   | $1 \cdot 10^{-4}$        | $1.5 \cdot 10^{-4}$    |
| RS (255, 239)             | 6.69                   | $9 \cdot 10^{-7}$        | $1 \cdot 10^{-6}$      |
| RS (255, 223)             | 14.35                  | $3 \cdot 10^{-13}$       | -                      |

**Table 8.3 Output BER Comparison of the RS Codes at Input BER of  $10^{-3}$**

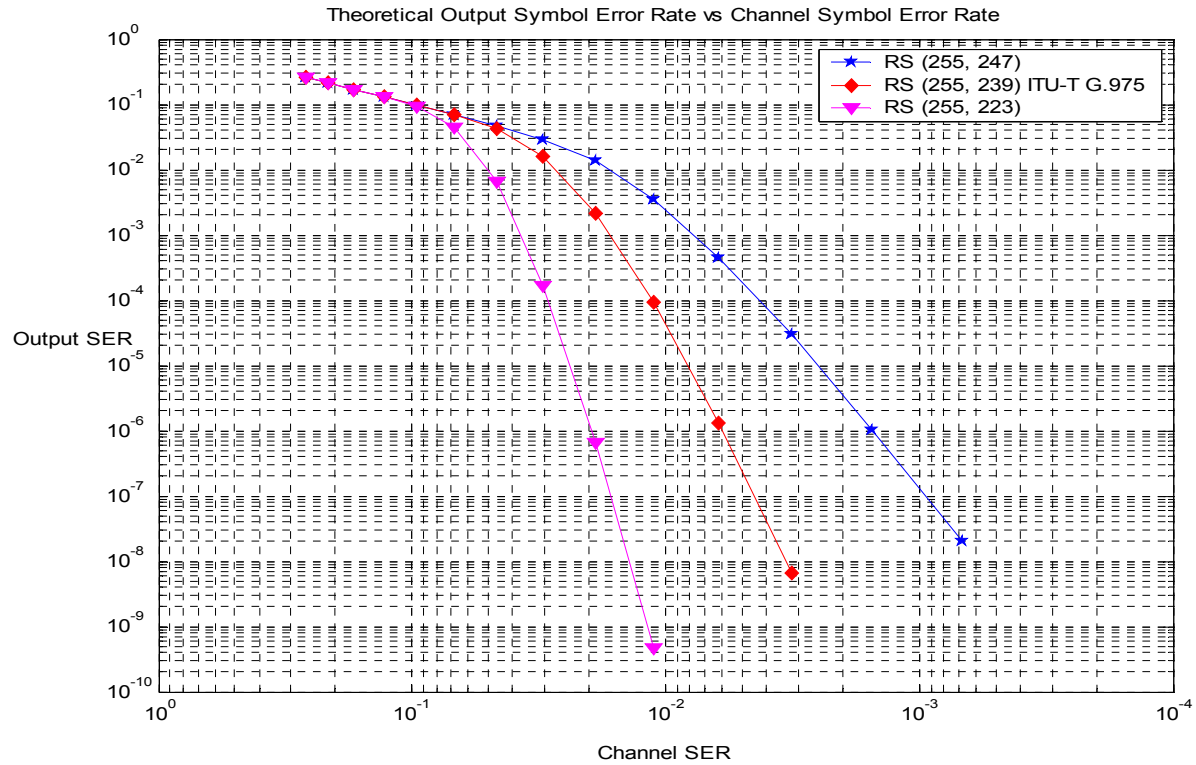
<sup>⊥</sup> Extrapolating the simulated BER curves to  $10^{-13}$



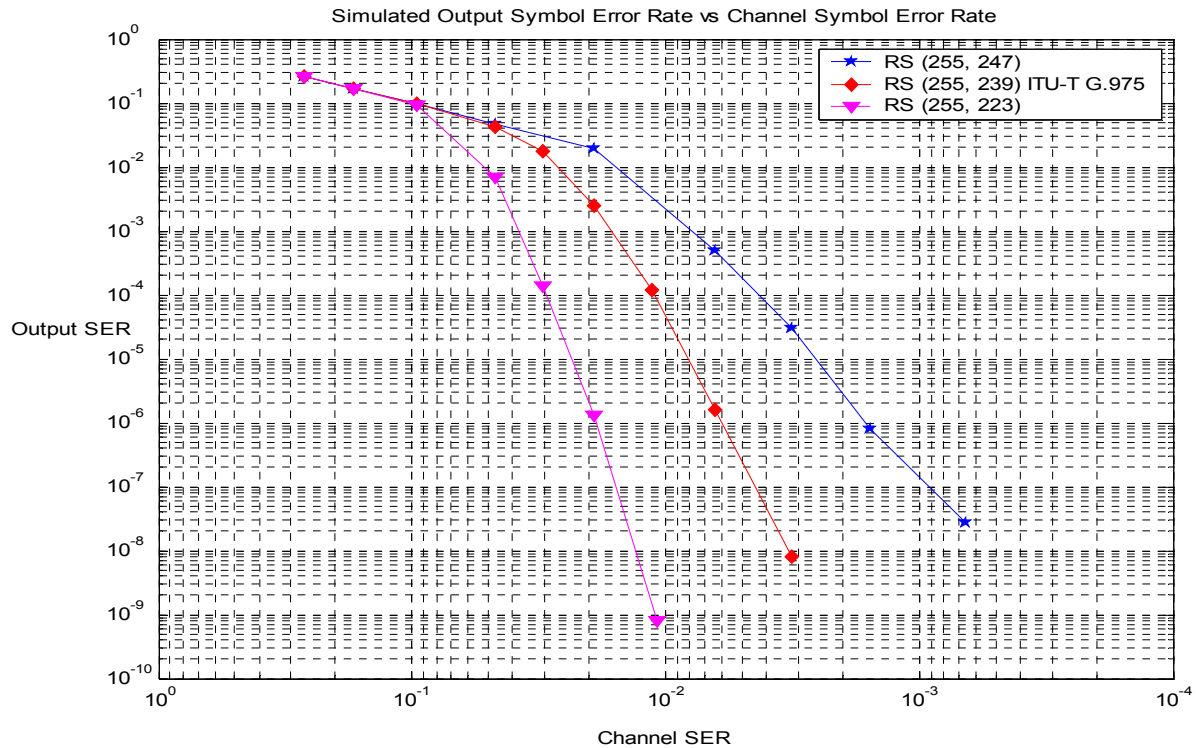
**Figure 8.4 Theoretical Output BER vs. Input BER Performance**



**Figure 8.5 Simulated Output BER vs. Input BER Performance**



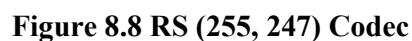
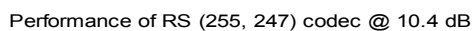
**Figure 8.6 Symbol Error Rate Performance (Theoretical)**



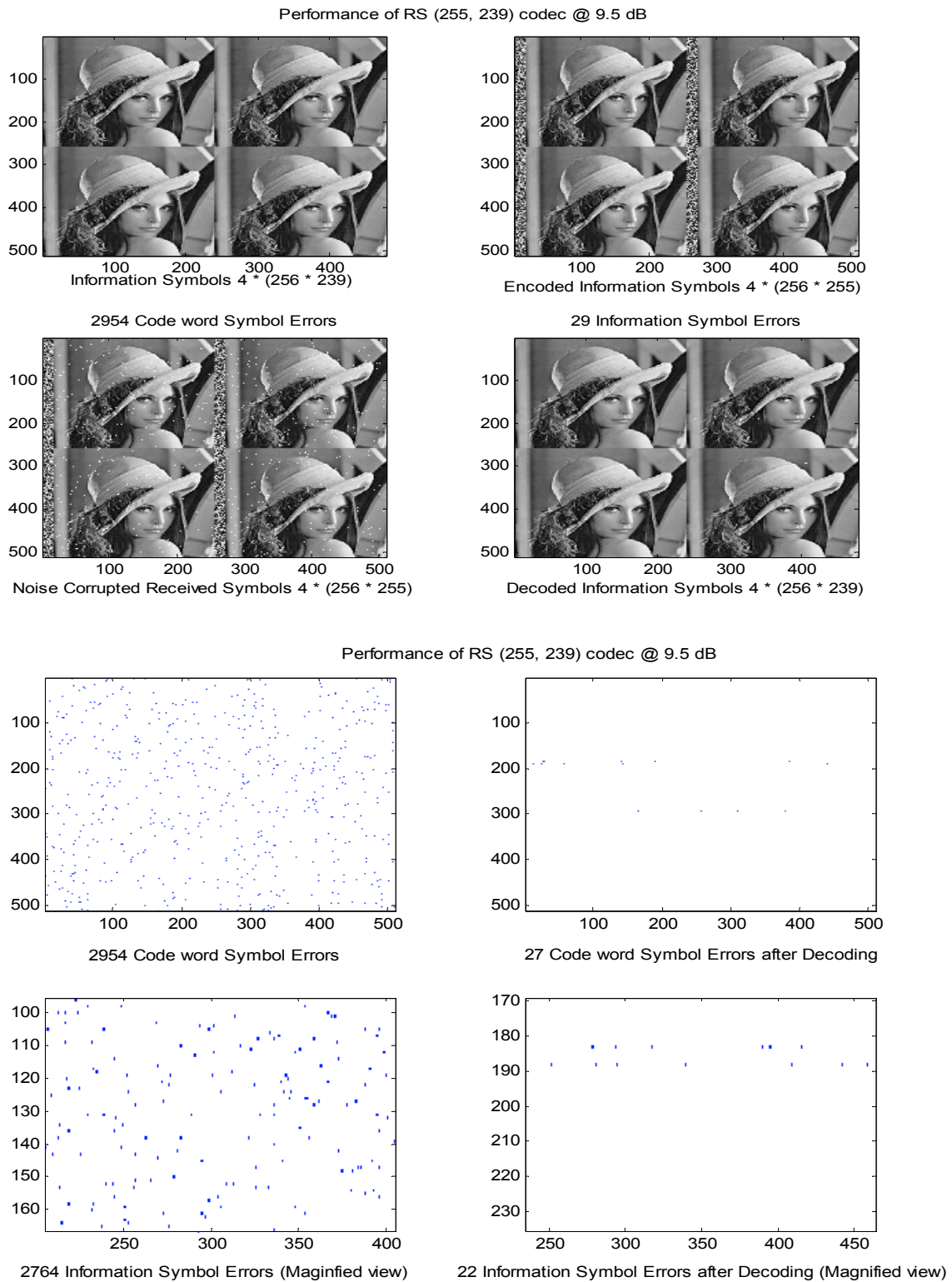
**Figure 8.7 Symbol Error Rate Performance (Simulated)**

The symbol error rate performance in figures 8.6 and 8.7 depicts the symbol errors corrected by the decoder versus the errors introduced by the channel in the symbols of the code word that are elements of  $GF(2^8)$ . A symbol error will occur irrespective of the number of bit errors in the symbol (refer sub section 3.3.3).

Performance of RS (255, 247) codec @ 10.4 dB



71

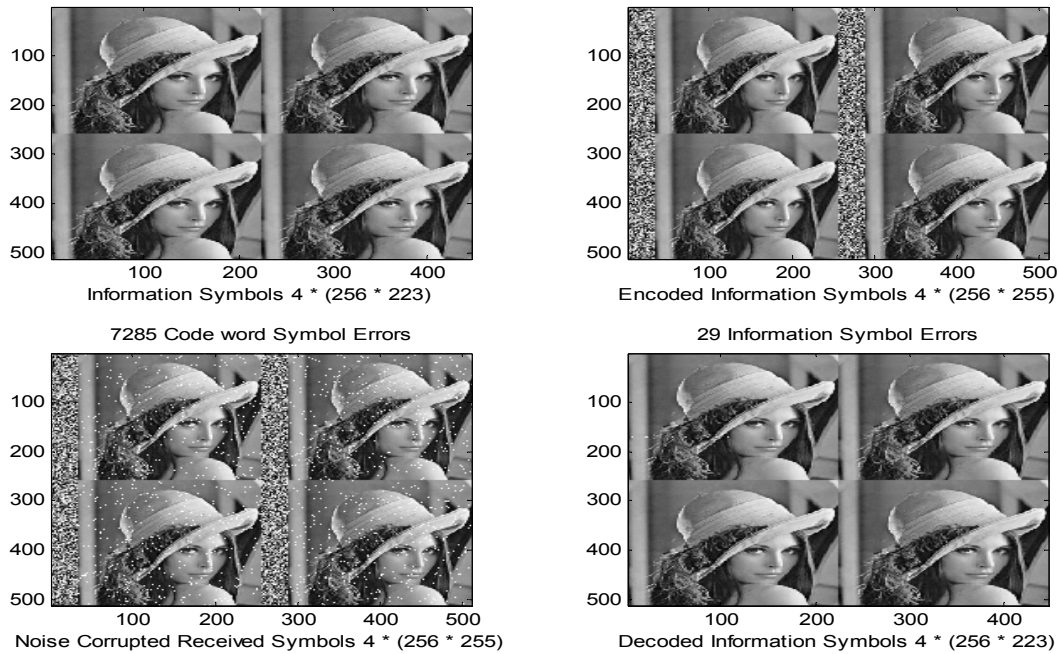


**Figure 8.9 RS (255, 239) Codec**

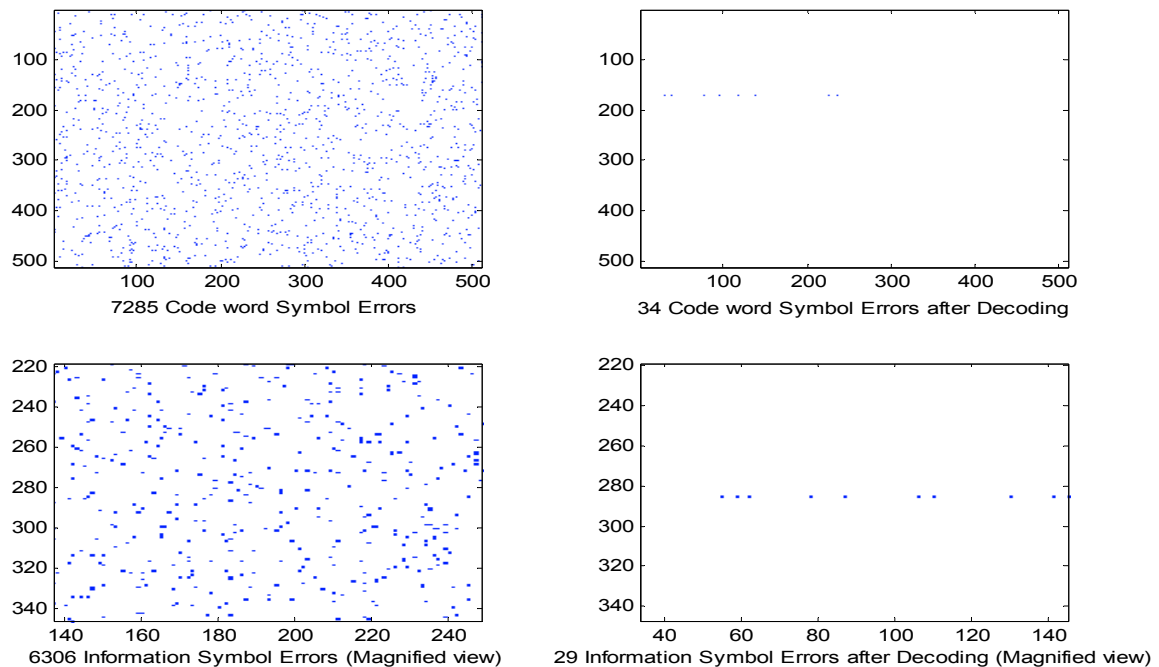
| Candidate RS<br>(n, k) Codes | Redundancy<br>(n-k)/k (%) | Q (dB) | # of Code word<br>Symbol Errors | # of Information<br>Symbol Errors |
|------------------------------|---------------------------|--------|---------------------------------|-----------------------------------|
| RS (255, 247)                | 3.24                      | 10.4   | 20                              | 20                                |
| RS (255, 239)                | 6.69                      | 9.5    | 27                              | 22                                |
| RS (255, 223)                | 14.35                     | 8.6    | 34                              | 29                                |

**Table 8.4 SNR Comparison at Output BER of  $10^{-5}$**

Performance of RS (255, 223) codec @ 8.6 dB



Performance of RS (255, 223) codec @ 8.6 dB



**Figure 8.10 RS (255, 223) Codec**

From table 8.4 it can be noted that in order to achieve the same output BER, the required SNR or Q decreases as the redundancy in the code increases. The number of information symbol errors after correction in the encoded pictures of the respective RS codes transmitted over AWGN is roughly the same, which gives an output BER of  $10^{-5}$ . However, approximately 1 dB more SNR is required by the RS codec to achieve the same BER performance as the redundancy decreases from 14.35 % to 6.69 % to 3.24 % respectively.

## 8.2 Performance of Serially Concatenated RS Codes

Concatenated codes provide an increased coding gain but with only a linear increase in hardware cost [16]. Good design of the concatenated code lies in proper selection of the inner and the outer code to meet the low overhead requirement. We extend the assumptions used to compute the output BER of RS codes to evaluate the approximate analytical output BER of the concatenated RS codes. The BER at the input of the inner decoder is the transition probability of the channel and is given by 6.21

$$P_{\text{(inner\_input)}} = Q\left(\sqrt{\frac{2 * R_c * E_b}{N_o}}\right); \quad (\text{BPSK})$$

The probability of code word symbol error at the input of the inner decoder is

$$P_{\text{se(inner)}} = 1 - (1 - P_{\text{(inner\_input)}})^{m/b}; \quad b = 1, (\text{BPSK})$$

The probability of Uncorrectable word error [36] for the inner decoder is given by 6.22

$$P_{\text{u(inner)}}(E) \approx \sum_{i=t+1}^n \frac{i}{n} \binom{n}{i} P_{\text{se(inner)}}^i (1 - P_{\text{se(inner)}})^{n-i} \quad ; n = q^m - 1$$

The BER at the output of the inner decoder is given by 6.23

$$P_{\text{b(inner\_output)}} \approx 1 - (1 - P_{\text{u(inner)}}(E))^{1/m}$$

The output BER of the inner decoder will in turn be the input BER for the outer decoder. Therefore, the probability of code word symbol error at the input of the outer decoder will be

$$P_{\text{se(outer)}} \approx 1 - (1 - P_{\text{b(inner\_output)}})^{m/b}; \quad b = 1, (\text{BPSK})$$

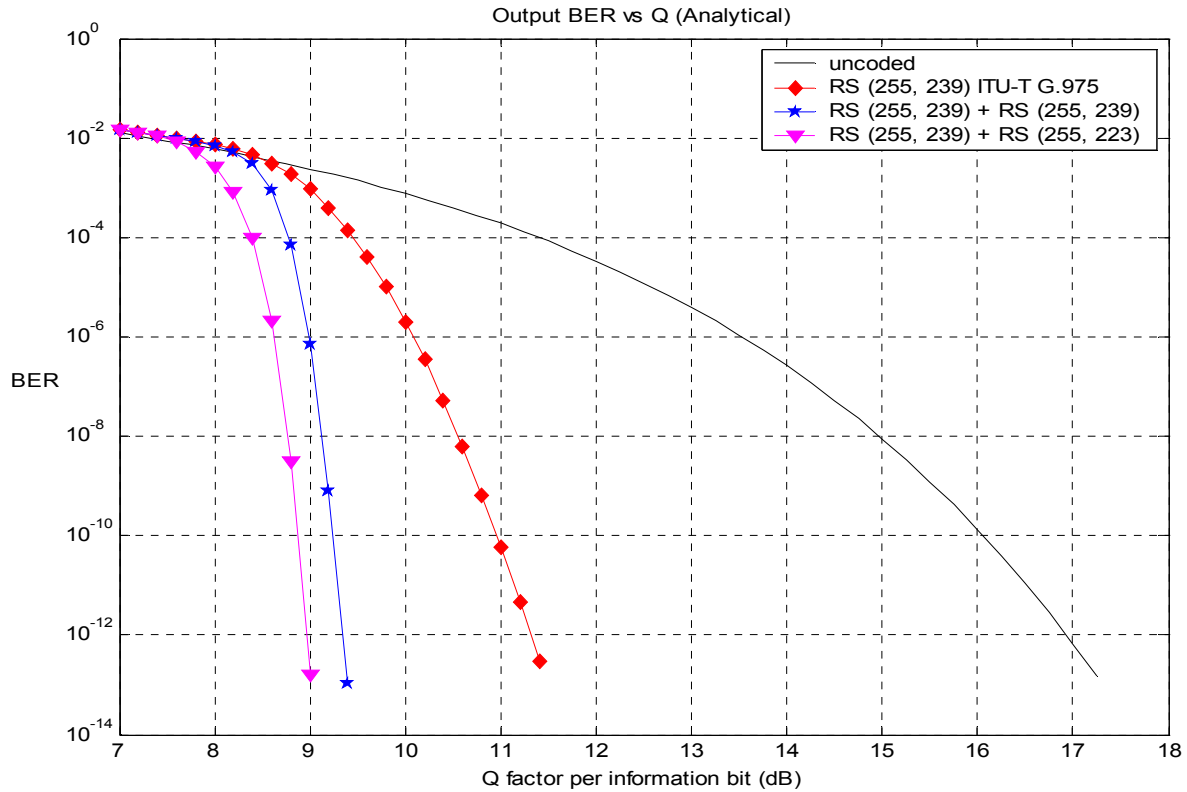
Hence, the probability of Uncorrectable word error for the outer decoder will be

$$P_{\text{u(outer)}}(E) \approx \sum_{i=t+1}^n \frac{i}{n} \binom{n}{i} P_{\text{se(outer)}}^i (1 - P_{\text{se(outer)}})^{n-i} \quad ; n = q^m - 1$$

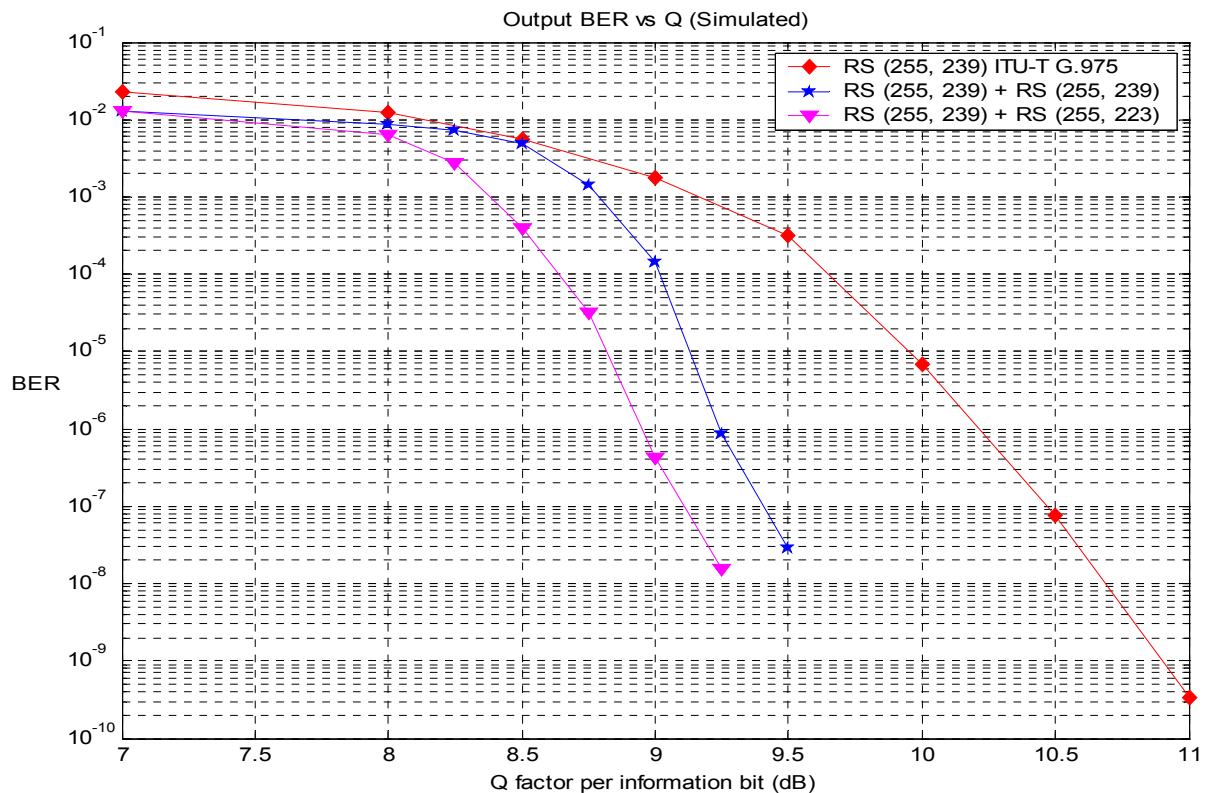
The BER at the output of the outer decoder is then given by

$$P_{\text{b(outer\_output)}} \approx 1 - (1 - P_{\text{u(outer)}}(E))^{1/m}$$

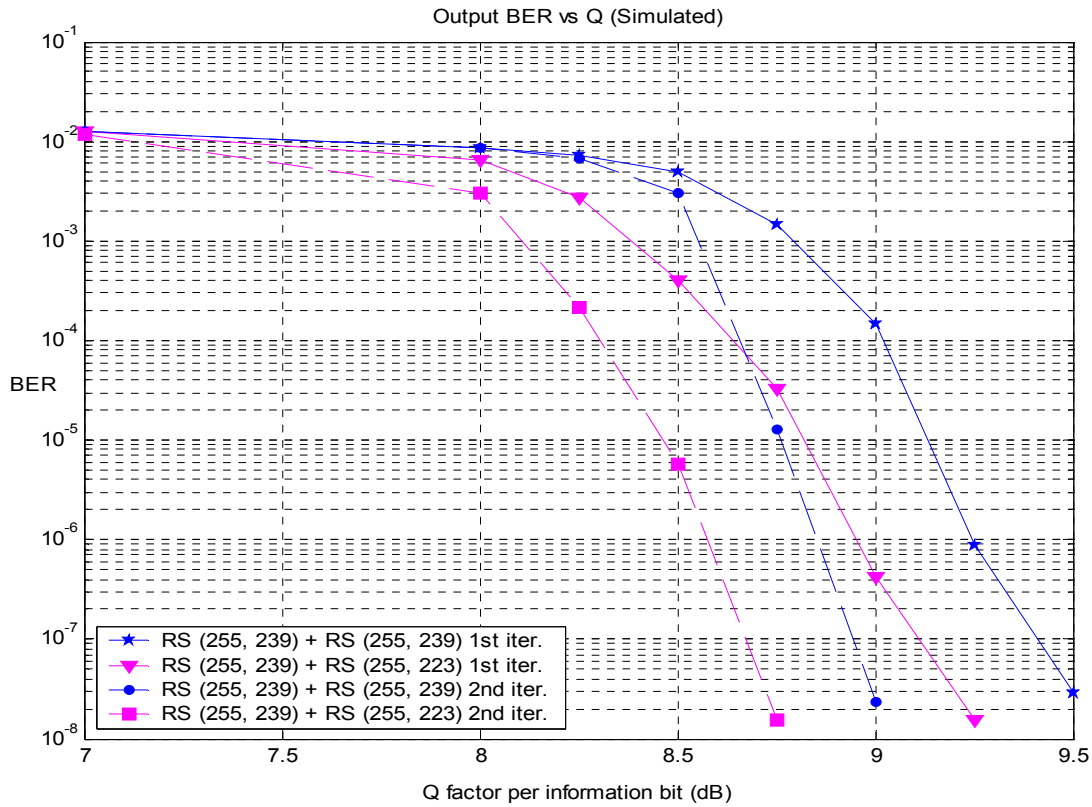
The RS product code parameters are presented in table 8.5. The approximate analytical output BER is presented in figure 8.11. The Monte-Carlo simulations were performed piece-wise for an output BER of  $\leq 10^{-7}$ . The simulated results are presented in figure 8.12. The simulated performance will be definitely poor than the approximate analytical performance since the decoding is a two-step decoding procedure based on HIHO component decoders, which is sub-optimal. Hence, the approximated analytical performance in figure 8.11 can be considered as a lower bound on the output BER performance for the candidate RS product codes. The simulated results are in coherence with [16]. The simulated results with two iterations of the component decoding process of the RS product codes are presented in figure 8.13. It is observed from figure 8.13 that after the second iteration approximately 0.5 dB additional gain is achieved compared to that after the first iteration at an output BER of  $\leq 10^{-7}$ .



**Figure 8.11 Approximate Output BER Performance of RS Product Codes**



**Figure 8.12 Simulated Output BER Performance of RS Product Codes**



**Figure 8.13 Simulated Performance of RS Product Codes after 2 Iterations**

| Candidate RS Product Codes    | Code Rate<br>$R = R_1 * R_2$ | Redundancy (%)<br>$(n_1 * n_2 - k_1 * k_2) / k_1 * k_2$ | $d_{min} = d_{min_1} * d_{min_2}$ |
|-------------------------------|------------------------------|---------------------------------------------------------|-----------------------------------|
| RS (255, 239) + RS (255, 239) | 0.8784                       | 13.83                                                   | 289                               |
| RS (255, 239) + RS (255, 223) | 0.8196                       | 22.00                                                   | 561                               |

**Table 8.5 RS Product Code Parameters**

The comparison of the performance of the candidate RS product codes and ITU-T standard in terms of coding gain at an output BER of  $\leq 10^{-7}$  and  $\leq 10^{-12}$  is presented in table 8.6. These results are obtained from figures 8.11 and 8.12. As mentioned earlier the BER requirement for optical communication systems is  $\leq 10^{-12}$ . It can be observed from table 8.6 the RS product codes have at least twice the redundancy compared to the ITU-T standard. This increased redundancy pays roughly more than 2 dB additional NCG. However, the RS (255, 239) + RS (255, 223) code with roughly 1.5 times more redundancy than the RS (255, 239) + RS (255, 239) code does not offer significant coding gain.

| Candidate Code                | Redundancy | NECG (dB) @ BER $\leq 10^{-7}$<br>(Theoretical) | NECG (dB) @ BER $\leq 10^{-7}$<br>(Simulated) | NECG (dB) @ BER $\leq 10^{-12}$<br>(Theoretical) | NECG (dB) @ BER $\leq 10^{-12}$<br>(Sim.Ext <sup>⊥</sup> ) |
|-------------------------------|------------|-------------------------------------------------|-----------------------------------------------|--------------------------------------------------|------------------------------------------------------------|
| RS (255, 239)                 | 6.69       | 4.42                                            | 4.28                                          | 5.63                                             | 5.3                                                        |
| RS (255, 239) + RS (255, 239) | 13.83      | 5.8                                             | 5.3                                           | 7.9                                              | 7.46                                                       |
| RS (255, 239) + RS (255, 223) | 22.00      | 6.2                                             | 5.7                                           | 8.3                                              | 7.6                                                        |

**Table 8.6 Coding Gain Comparison of the ITU-T standard and RS Product Codes**

<sup>⊥</sup> Sim. Ext.: Extrapolating the simulated BER curves to  $10^{-13}$

### 8.3 Performance of Block Turbo Codes

We have considered the BTC with identical component BCH codes since they are more suitable for implementation [43]. The parameters of the candidate BCH product codes are listed in table 8.7. As the code rate decreases the redundancy in the code increases.

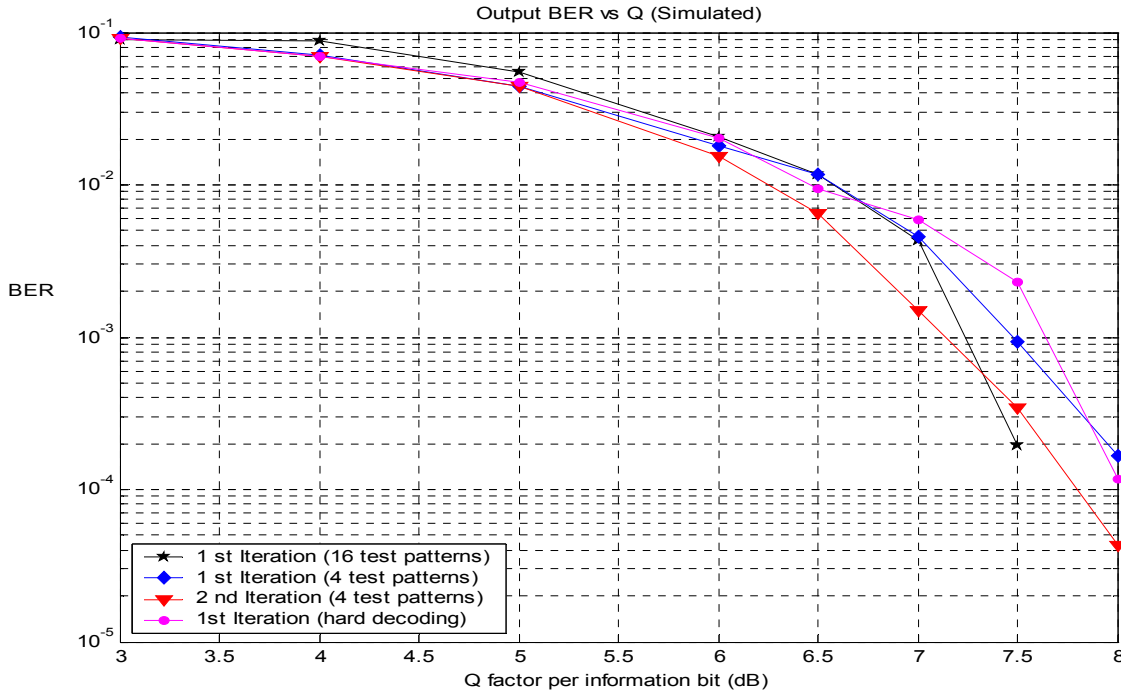
| Candidate BTC               | Code Rate<br>$R = R_1 * R_2$ | Redundancy (%)<br>$(n_1 * n_2 - k_1 * k_2) / k_1 * k_2$ | $d_{min}$ | $G_a$ (dB)<br>Asymptotic |
|-----------------------------|------------------------------|---------------------------------------------------------|-----------|--------------------------|
| BCH (255, 239) <sup>2</sup> | 0.8784                       | 13.83                                                   | 25        | 13.41                    |
| BCH (63, 57) <sup>2</sup>   | 0.8186                       | 22.16                                                   | 9         | 8.673                    |
| BCH (127, 113) <sup>2</sup> | 0.7916                       | 26.31                                                   | 25        | 12.96                    |

**Table 8.7 BCH Product Code Parameters**

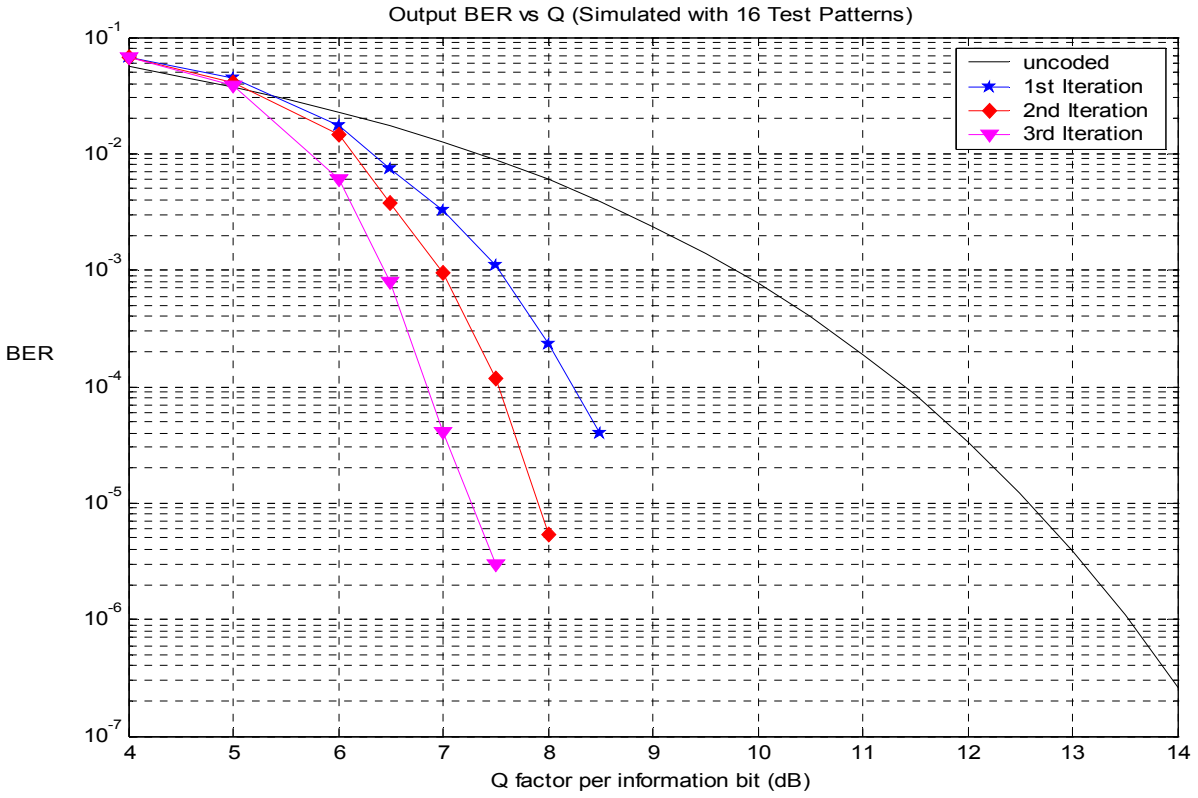
The scaling parameter  $\alpha$  used by the turbo decoding algorithm is summarized in table 8.8. We have used the same value of  $\alpha$  in the iterative decoding of all the BTC's in the computer simulations. The performance of the SISO decoder can be improved by optimizing the scaling parameter  $\alpha$  [42]. From figure 8.14, it becomes evident that the performance of the SISO decoder varies with a single parameter in the decoding process (the number of least reliable bits  $p$ ). Thus, the computational complexity of the SISO decoder increases exponential in  $p$ . As mentioned earlier (refer sub section 7.4.2) iterative HIHO decoding of the component codes of BTC at low values of SNR does not offer substantial performance.

| Test Patterns $T^q$       | P = 2 and 4. # of test patterns is 4 and 16 respectively |          |             |     |             |  |
|---------------------------|----------------------------------------------------------|----------|-------------|-----|-------------|--|
|                           | Iteration 1                                              |          | Iteration 2 |     | Iteration 3 |  |
| Weighting factor $\alpha$ | 0.0                                                      | 0.2 0.25 | 0.3         | 0.3 | 0..35       |  |

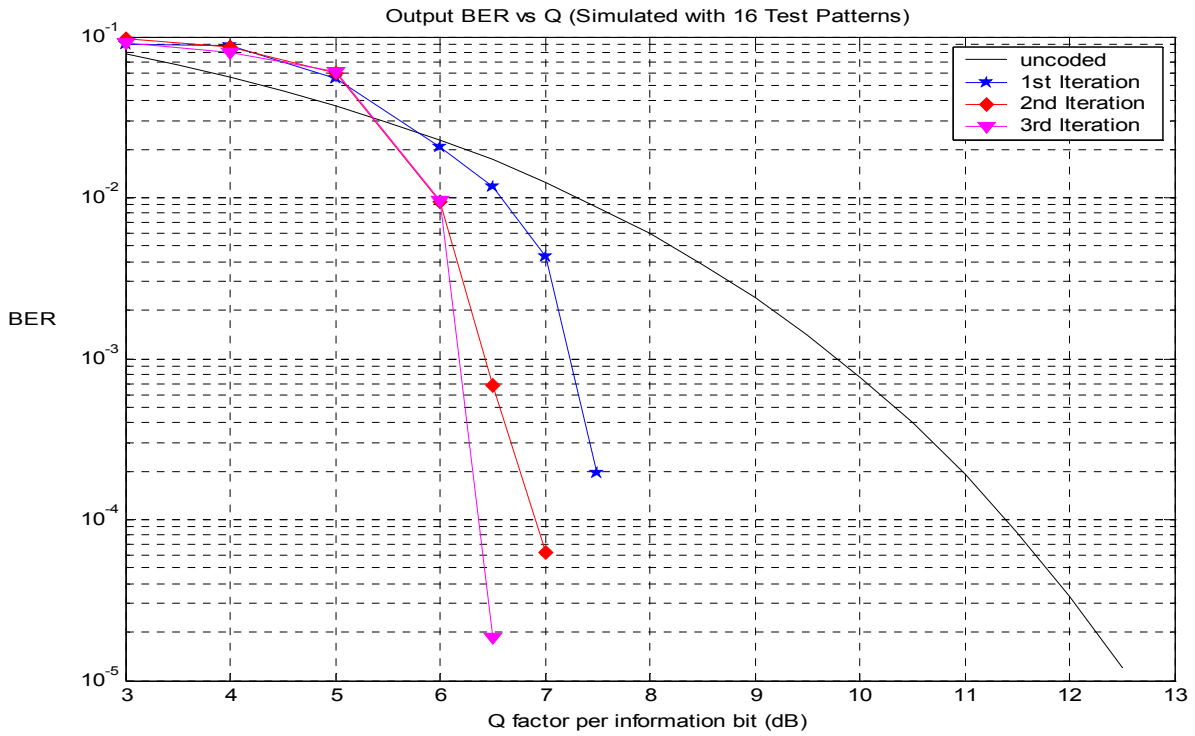
**Table 8.8 Iterative Decoding Scaling Parameter**



**Figure 8.14 Simulated Performance of BTC (127, 113)<sup>2</sup>**

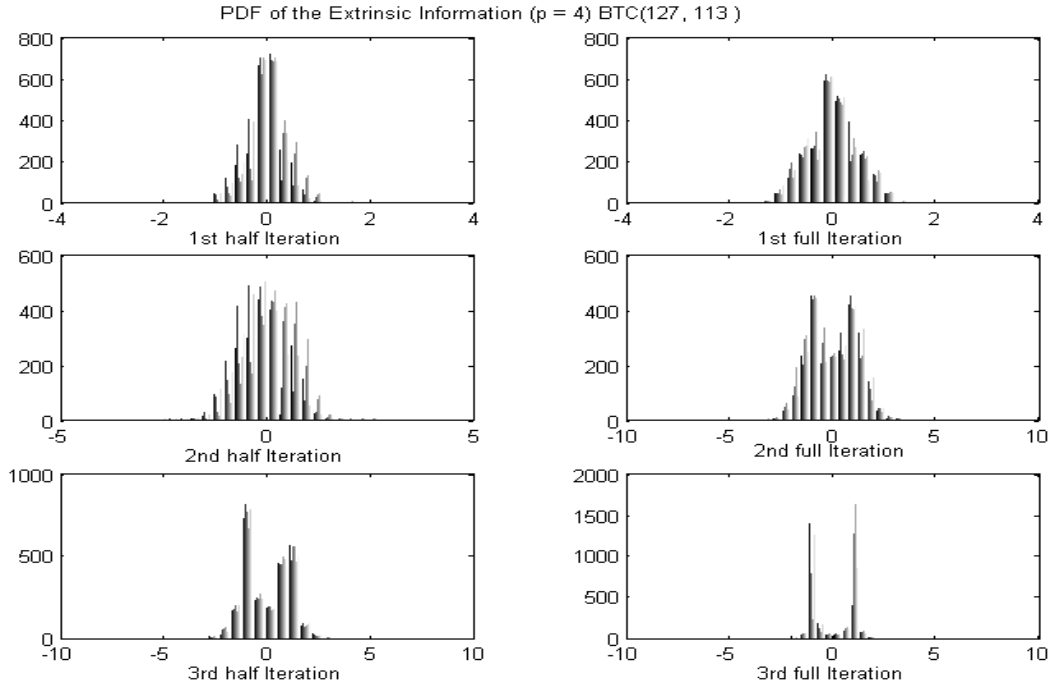


**Figure 8.15 Simulated Performance of BTC  $(63, 57)^2$  after 3 Iterations**

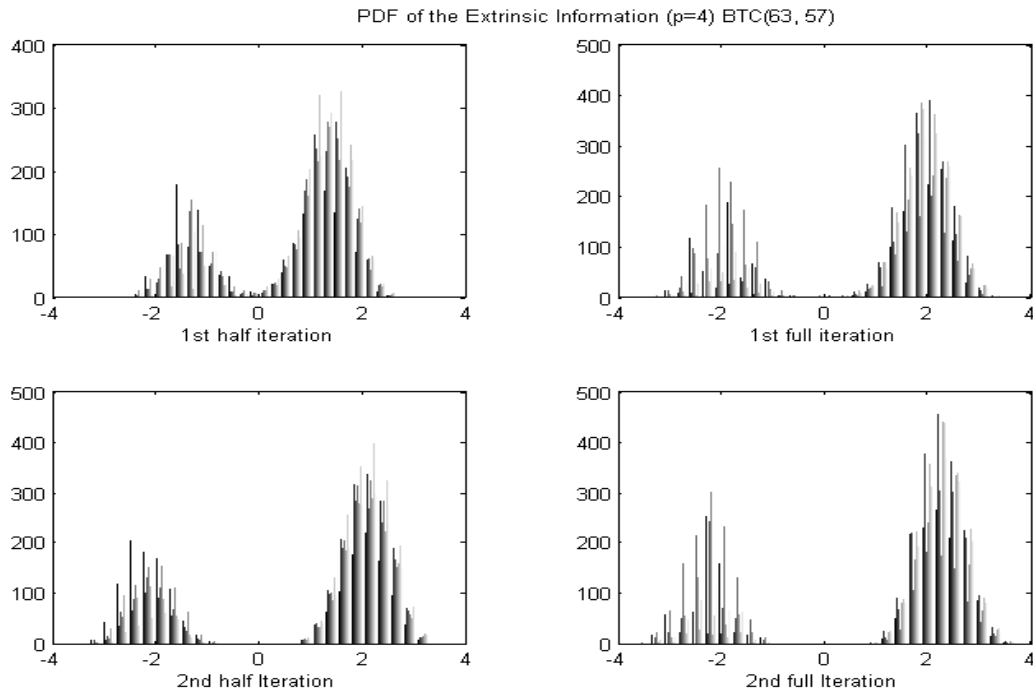


**Figure 8.16 Simulated Performance of BTC  $(127, 113)^2$  after 3 Iterations**

The results presented in figures 8.15 and 8.16 for the BTC's  $(63, 57)^2$  and  $(127, 113)^2$  are in coherence with [42]. Better performance can be achieved by using extended BCH codes [43], [17].

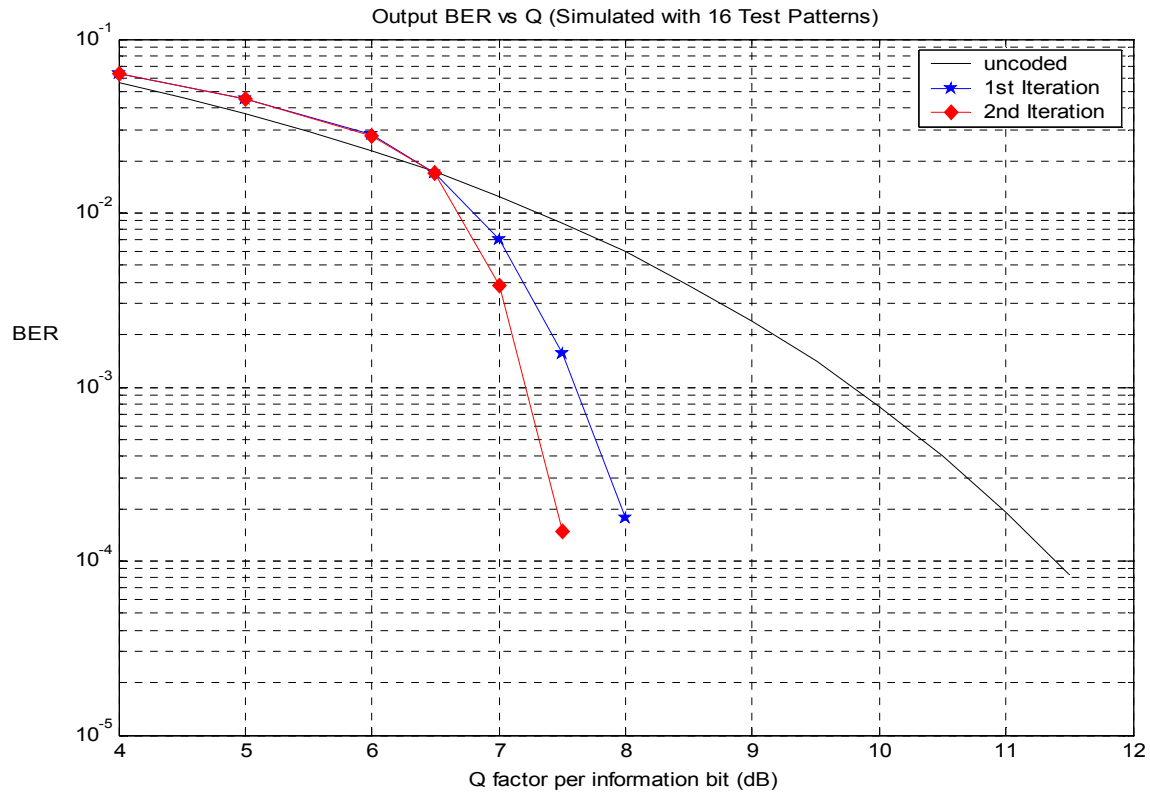


**Figure 8.17 PDF of Extrinsic Information BTC (127, 113)<sup>2</sup>**

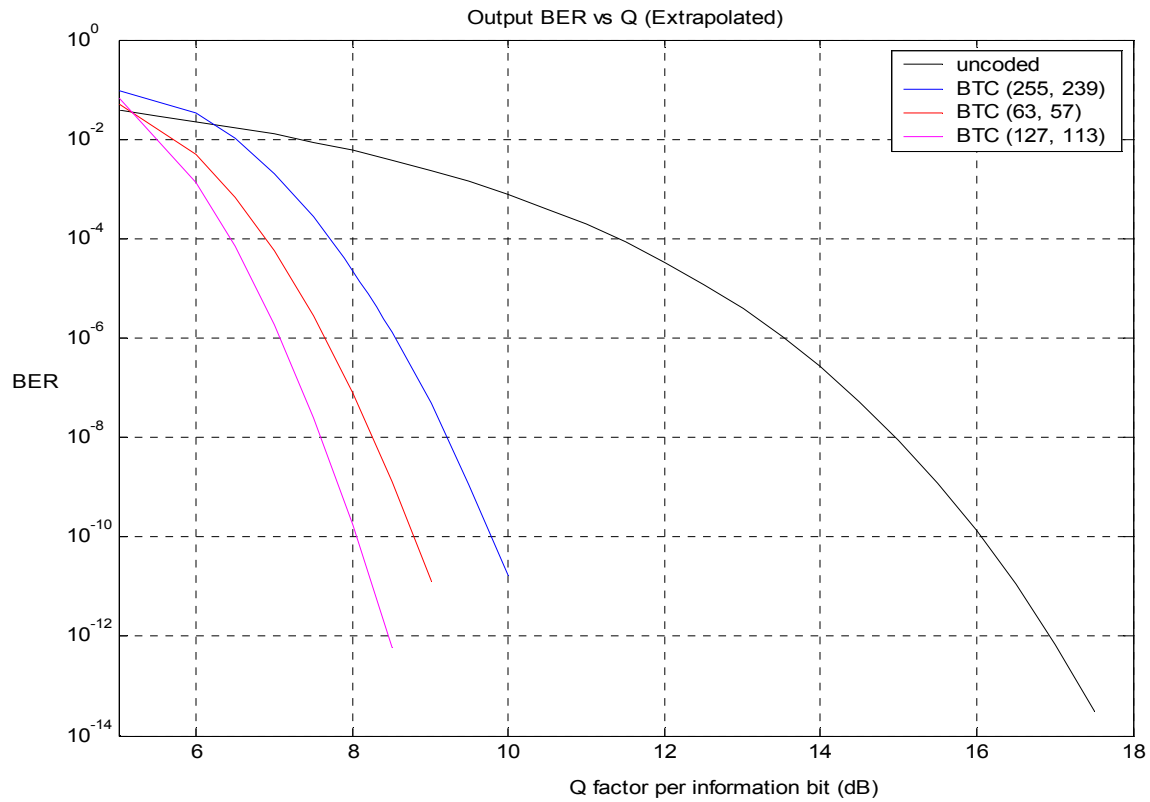


**Figure 8.18 PDF of Extrinsic Information BTC (63, 57)<sup>2</sup>**

The extrinsic information has a Gaussian distribution as can be observed from figures 8.17 and 8.18. The standard deviation of the extrinsic information is high in the first iteration and decreases with the successive iterations. This is evident from the width and height of the distribution with the normalization maintained. The standard deviation decreases as we iterate the decoding, the width of the distribution decreases while the distribution becomes more peaky.



**Figure 8.19 Simulated Performance of BTC (255, 239)<sup>2</sup> after 2 Iterations**



**Figure 8.20 Performance of all BTC's (Extrapolated)**

Figure 8.20 is obtained by extrapolating the simulated BER curves for BTC (255, 239)<sup>2</sup> after 2 iteration while for other BTCs after 3 iterations.

The comparison of the coding gain performance of the different BTC's is presented in table 8.9. As expected, the NECG increases as the redundancy in the code increases. The numerical values of the NECG are obtained from the BER curve after the third iteration.

| Candidate Code              | Redundancy (%) | Code rate | NECG (dB) @ $10^{-5}$ (Simulated) | NECG (dB) @ $10^{-10}$ (Sim. Ext. <sup>1</sup> ) |
|-----------------------------|----------------|-----------|-----------------------------------|--------------------------------------------------|
| BTC (255, 239) <sup>2</sup> | 13.83          | 0.8784    | -                                 | 6.4                                              |
| BTC (63, 57) <sup>2</sup>   | 22.16          | 0.8185    | 5.25                              | 7.3                                              |
| BTC (127, 113) <sup>2</sup> | 26.31          | 0.7916    | 6                                 | 8.0                                              |

**Table 8.9 Coding Gain Comparison of the Block Turbo Codes**

Next, we present the comparison in terms of net coding gain performance offered by the different coding schemes with comparable redundancy and code rate.

| Candidate Code              | Redundancy | Code rate | NECG (dB) @ $10^{-8}$ | NECG (dB) @ $10^{-12}$ |
|-----------------------------|------------|-----------|-----------------------|------------------------|
| RS (255, 223)               | 14.35      | 0.8745    | 5.15                  | 6.33                   |
| RS (255, 239)               | 13.83      | 0.8784    | 5.3                   | 7.46                   |
| + RS (255, 239)             |            |           |                       |                        |
| BTC (255, 239) <sup>2</sup> | 13.83      | 0.8784    | 5.8                   | -                      |

**Table 8.10 Coding Gain Comparison of Codes with 14% Redundancy**

| Candidate Code              | Redundancy | Code rate | NECG (dB) @ $10^{-8}$ | NECG @ $10^{-12}$ |
|-----------------------------|------------|-----------|-----------------------|-------------------|
| RS (255, 239)               | 22.00      | 0.8196    | 5.7                   | 7.6               |
| + RS (255, 223)             |            |           |                       |                   |
| BTC (63, 57) <sup>2</sup>   | 22.16      | 0.8185    | 6.6                   | -                 |
| BTC (127, 113) <sup>2</sup> | 26.31      | 0.7916    | 7.3                   | 8.5               |

**Table 8.11 Coding Gain Comparison of Codes with  $\geq 22\%$  Redundancy**

It can be observed from table 8.10, even though the candidate codes offer comparable performance for the same amount of redundancy, the computational complexity of the encoder and decoder for these candidate codes is not the same. The RS (255, 223) encoder has slightly less computational complexity compared to RS (255, 239) + RS (255, 239) encoder (refer figure 6.11). Even though, the RS (255, 223) decoder has higher computational complexity compared to the RS (255, 239) decoder (refer figure 6.12), the decoder for RS (255, 239) + RS (255, 239) code involves two component RS (255, 239) decoders and a memory element for the deinterleaver. However, the overall encoder/decoder complexity for these two codes is comparable. The BTC (255, 239)<sup>2</sup> encoder will be less complex than the RS encoders since finite-field computations are from GF (2). However, the decoder complexity of the component SISO decoder for the BTC will be much higher than that of the RS HIHO decoders. In addition, the comparable coding gain performance will be achieved after at least 3 iterations with 16 test patterns.

From table 8.11, we notice that the candidate codes have  $\geq 22\%$  redundancy. BTC's do offer additional coding gain in comparison with the RS product code. Again, decoding of the BTC's involve SISO decoder which is too complex. From the results presented in sections 8.1, 8.2 and 8.3 we conclude that while designing an error correcting scheme for reliable communication we have to tradeoff not only redundancy against NECG but also complexity against NECG.

<sup>1</sup> Sim. Ext.: Simulated BER after 3 iterations is extrapolated

## 8.4 Remarks and Conclusion

From the review of the ITU-T G.975 recommendation and performance of potential candidate coding schemes presented in the earlier sections, it has been proved that FEC relaxes the requirement on the SNR at the receiver. The net coding gain offered by the FEC solution can be translated in improving the distance-capacity performance of an optical link. With soft decision iterated decoding of BTC's promising results in terms of coding gain performance are achievable, but hardware implementation of SISO decoders at data rate of 10 Gbits/s does not seem to be a feasible FEC solution. Concatenated RS codes with two-step hard decoding provide significant improvement in coding gain performance compared to the ITU-T standard for bit-error rate of  $\leq 10^{-12}$ . Nevertheless, this improved performance is achieved at the cost of increased computational complexity. However, the parameters like redundancy, computational complexity and coding gain performance of the RS (255, 223) and RS (255, 239) + RS (255, 239) codes are comparable. In addition, concatenated RS codes provide better random as well as burst error correction capability. If sufficient buffer memory is available, further improved performance can be achieved by 2 iterations of the two-step decoding process. Such iterative decoding does not increase the code overhead, and does not change the encoder. More than 2 iterations do not offer further performance improvement [16].

The main objective of this thesis project was to investigate the advantages and limitations of different FEC solutions regarding performance and feasible ways of implementation. The work presented in the report was more theoretical based on analytical and computer simulation results. The presented work could be further continued with a hardware implementation of a encoder/decoder in a fiber-optic transceiver. A scalable RS encoder/decoder operating at low data rate could be initially implemented with a FPGA for single channel as well as multichannel systems. Laboratory experiments at low data rate and variable span of fiber link could be conducted on the transceiver with FEC by introducing and varying the critical channel impairments. With positive outcome of these experiments, an ASIC implementation with parallel operations may be realized considering cost and power consumption. RS decoder designs and architectures implemented in VLSI circuits for the long RS codes are found in [13], [47] and [55]. Efficient implementation of finite-field arithmetic operations is presented in [34], [35], [39] and [48].

Optical networks have evolved significantly over the past few years, moving from data rates of 2.5 Gbits/s to 10 Gbits/s and looking forward to 40 Gbits/s and beyond. As the transmission speed increases on one hand, the transmission impairments also increase on the other. In many cases, the impairments increase in a nonlinear fashion, even though the transmission rates are increasing linearly. The same phenomenon is also true for WDM systems. Hence, FEC becomes vital in single channel as well as multi-channel WDM systems for delivering error free transmission between 10-100 Gbits/s for next generation optical systems.

## 9 APPENDICES

### 9.1 Appendix A

Galois Field m-tuple representation for GF ( $q = 2^m$ ) for  $3 \leq m \leq 8$

GF (8)

$$p(X) = X^3 + X + 1$$

|   |   |   |   |
|---|---|---|---|
| 0 | 1 | 0 | 0 |
| 1 | 0 | 1 | 0 |
| 2 | 0 | 0 | 1 |
| 3 | 1 | 1 | 0 |
| 4 | 0 | 1 | 1 |
| 5 | 1 | 1 | 1 |
| 6 | 1 | 0 | 1 |

GF (16)

$$p(X) = X^4 + X + 1$$

|    |   |   |   |   |
|----|---|---|---|---|
| 0  | 1 | 0 | 0 | 0 |
| 1  | 0 | 1 | 0 | 0 |
| 2  | 0 | 0 | 1 | 0 |
| 3  | 0 | 0 | 0 | 1 |
| 4  | 1 | 1 | 0 | 0 |
| 5  | 0 | 1 | 1 | 0 |
| 6  | 0 | 0 | 1 | 1 |
| 7  | 1 | 1 | 0 | 1 |
| 8  | 1 | 0 | 1 | 0 |
| 9  | 0 | 1 | 0 | 1 |
| 10 | 1 | 1 | 1 | 0 |
| 11 | 0 | 1 | 1 | 1 |
| 12 | 1 | 1 | 1 | 1 |
| 13 | 1 | 0 | 1 | 1 |
| 14 | 1 | 0 | 0 | 1 |

GF (64)

$$p(X) = X^6 + X + 1$$

|    |   |   |   |   |   |   |
|----|---|---|---|---|---|---|
| 0  | 1 | 0 | 0 | 0 | 0 | 0 |
| 1  | 0 | 1 | 0 | 0 | 0 | 0 |
| 2  | 0 | 0 | 1 | 0 | 0 | 0 |
| 3  | 0 | 0 | 0 | 1 | 0 | 0 |
| 4  | 0 | 0 | 0 | 0 | 1 | 0 |
| 5  | 0 | 0 | 0 | 0 | 0 | 1 |
| 6  | 1 | 1 | 0 | 0 | 0 | 0 |
| 7  | 0 | 1 | 1 | 0 | 0 | 0 |
| 8  | 0 | 0 | 1 | 1 | 0 | 0 |
| 9  | 0 | 0 | 0 | 1 | 1 | 0 |
| 10 | 0 | 0 | 0 | 0 | 1 | 1 |
| 11 | 1 | 1 | 0 | 0 | 0 | 1 |
| 12 | 1 | 0 | 1 | 0 | 0 | 0 |
| 13 | 0 | 1 | 0 | 1 | 0 | 0 |
| 14 | 0 | 0 | 1 | 0 | 1 | 0 |
| 15 | 0 | 0 | 0 | 1 | 0 | 1 |
| 16 | 1 | 1 | 0 | 0 | 1 | 0 |

GF (32)

$$p(X) = X^5 + X^2 + 1$$

|    |   |   |   |   |   |
|----|---|---|---|---|---|
| 0  | 1 | 0 | 0 | 0 | 0 |
| 1  | 0 | 1 | 0 | 0 | 0 |
| 2  | 0 | 0 | 1 | 0 | 0 |
| 3  | 0 | 0 | 0 | 1 | 0 |
| 4  | 0 | 0 | 0 | 0 | 1 |
| 5  | 1 | 0 | 1 | 0 | 0 |
| 6  | 0 | 1 | 0 | 1 | 0 |
| 7  | 0 | 0 | 1 | 0 | 1 |
| 8  | 1 | 0 | 1 | 1 | 0 |
| 9  | 0 | 1 | 0 | 1 | 1 |
| 10 | 1 | 0 | 0 | 0 | 1 |
| 11 | 1 | 1 | 1 | 0 | 0 |
| 12 | 0 | 1 | 1 | 1 | 0 |
| 13 | 0 | 0 | 1 | 1 | 1 |
| 14 | 1 | 0 | 1 | 1 | 1 |
| 15 | 1 | 1 | 1 | 1 | 1 |

|    |   |   |   |   |   |
|----|---|---|---|---|---|
| 16 | 1 | 1 | 0 | 1 | 1 |
| 17 | 1 | 1 | 0 | 0 | 1 |
| 18 | 1 | 1 | 0 | 0 | 0 |
| 19 | 0 | 1 | 1 | 0 | 0 |
| 20 | 0 | 0 | 1 | 1 | 0 |
| 21 | 0 | 0 | 0 | 1 | 1 |
| 22 | 1 | 0 | 1 | 0 | 1 |
| 23 | 1 | 1 | 1 | 1 | 0 |
| 24 | 0 | 1 | 1 | 1 | 1 |
| 25 | 1 | 0 | 0 | 1 | 1 |
| 26 | 1 | 1 | 1 | 0 | 1 |
| 27 | 1 | 1 | 0 | 1 | 0 |
| 28 | 0 | 1 | 1 | 0 | 1 |
| 29 | 1 | 0 | 0 | 1 | 0 |
| 30 | 0 | 1 | 0 | 0 | 1 |

|    |   |   |   |   |   |   |    |   |   |   |   |   |   |    |   |   |   |   |   |   |
|----|---|---|---|---|---|---|----|---|---|---|---|---|---|----|---|---|---|---|---|---|
| 17 | 0 | 1 | 1 | 0 | 0 | 1 | 39 | 0 | 1 | 1 | 0 | 1 | 1 | 61 | 1 | 0 | 0 | 0 | 1 | 1 |
| 18 | 1 | 1 | 1 | 1 | 0 | 0 | 40 | 1 | 1 | 1 | 1 | 0 | 1 | 62 | 1 | 0 | 0 | 0 | 0 | 1 |
| 19 | 0 | 1 | 1 | 1 | 1 | 0 | 41 | 1 | 0 | 1 | 1 | 1 | 0 |    |   |   |   |   |   |   |
| 20 | 0 | 0 | 1 | 1 | 1 | 1 | 42 | 0 | 1 | 0 | 1 | 1 | 1 |    |   |   |   |   |   |   |
| 21 | 1 | 1 | 0 | 1 | 1 | 1 | 43 | 1 | 1 | 1 | 0 | 1 | 1 |    |   |   |   |   |   |   |

GF (128)

$$p(X) = X^7 + X^3 + 1$$

|    |   |   |   |   |   |   |   |    |   |   |   |   |   |   |   |     |   |   |   |   |   |   |   |
|----|---|---|---|---|---|---|---|----|---|---|---|---|---|---|---|-----|---|---|---|---|---|---|---|
| 0  | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 43 | 0 | 1 | 1 | 0 | 1 | 0 | 1 | 86  | 0 | 0 | 1 | 0 | 1 | 1 | 0 |
| 1  | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 44 | 1 | 0 | 1 | 0 | 0 | 1 | 0 | 87  | 0 | 0 | 0 | 1 | 0 | 1 | 1 |
| 2  | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 45 | 0 | 1 | 0 | 1 | 0 | 0 | 1 | 88  | 1 | 0 | 0 | 1 | 1 | 0 | 1 |
| 3  | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 46 | 1 | 0 | 1 | 1 | 1 | 0 | 0 | 89  | 1 | 1 | 0 | 1 | 1 | 1 | 0 |
| 4  | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 47 | 0 | 1 | 0 | 1 | 1 | 1 | 0 | 90  | 0 | 1 | 1 | 0 | 1 | 1 | 1 |
| 5  | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 48 | 0 | 0 | 1 | 0 | 1 | 1 | 1 | 91  | 1 | 0 | 1 | 0 | 0 | 1 | 1 |
| 6  | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 49 | 1 | 0 | 0 | 0 | 0 | 1 | 1 | 92  | 1 | 1 | 0 | 0 | 0 | 0 | 1 |
| 7  | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 50 | 1 | 1 | 0 | 1 | 0 | 0 | 1 | 93  | 1 | 1 | 1 | 1 | 0 | 0 | 0 |
| 8  | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 51 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 94  | 0 | 1 | 1 | 1 | 1 | 0 | 0 |
| 9  | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 52 | 0 | 1 | 1 | 1 | 1 | 1 | 0 | 95  | 0 | 0 | 1 | 1 | 1 | 1 | 0 |
| 10 | 0 | 0 | 0 | 1 | 0 | 0 | 1 | 53 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 96  | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 11 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 54 | 1 | 0 | 0 | 0 | 1 | 1 | 1 | 97  | 1 | 0 | 0 | 1 | 1 | 1 | 1 |
| 12 | 0 | 1 | 0 | 0 | 1 | 1 | 0 | 55 | 1 | 1 | 0 | 1 | 0 | 1 | 1 | 98  | 1 | 1 | 0 | 1 | 1 | 1 | 1 |
| 13 | 0 | 0 | 1 | 0 | 0 | 1 | 1 | 56 | 1 | 1 | 1 | 1 | 1 | 0 | 1 | 99  | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 14 | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 57 | 1 | 1 | 1 | 0 | 1 | 1 | 0 | 100 | 1 | 1 | 1 | 0 | 1 | 1 | 1 |
| 15 | 1 | 1 | 0 | 1 | 0 | 0 | 0 | 58 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 101 | 1 | 1 | 1 | 0 | 0 | 1 | 1 |
| 16 | 0 | 1 | 1 | 0 | 1 | 0 | 0 | 59 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 102 | 1 | 1 | 1 | 0 | 0 | 0 | 1 |
| 17 | 0 | 0 | 1 | 1 | 0 | 1 | 0 | 60 | 1 | 1 | 0 | 0 | 0 | 1 | 0 | 103 | 1 | 1 | 1 | 0 | 0 | 0 | 0 |
| 18 | 0 | 0 | 0 | 1 | 1 | 0 | 1 | 61 | 0 | 1 | 1 | 0 | 0 | 0 | 1 | 104 | 0 | 1 | 1 | 1 | 0 | 0 | 0 |
| 19 | 1 | 0 | 0 | 1 | 1 | 1 | 0 | 62 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 105 | 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| 20 | 0 | 1 | 0 | 0 | 1 | 1 | 1 | 63 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 106 | 0 | 0 | 0 | 1 | 1 | 1 | 0 |
| 21 | 1 | 0 | 1 | 1 | 0 | 1 | 1 | 64 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 107 | 0 | 0 | 0 | 0 | 1 | 1 | 1 |
| 22 | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 65 | 0 | 0 | 0 | 1 | 0 | 1 | 0 | 108 | 1 | 0 | 0 | 1 | 0 | 1 | 1 |
| 23 | 1 | 1 | 1 | 1 | 0 | 1 | 0 | 66 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 109 | 1 | 1 | 0 | 1 | 1 | 0 | 1 |
| 24 | 0 | 1 | 1 | 1 | 1 | 0 | 1 | 67 | 1 | 0 | 0 | 1 | 0 | 1 | 0 | 110 | 1 | 1 | 1 | 1 | 1 | 1 | 0 |
| 25 | 1 | 0 | 1 | 0 | 1 | 1 | 0 | 68 | 0 | 1 | 0 | 0 | 1 | 0 | 1 | 111 | 0 | 1 | 1 | 1 | 1 | 1 | 1 |
| 26 | 0 | 1 | 0 | 1 | 0 | 1 | 1 | 69 | 1 | 0 | 1 | 1 | 0 | 1 | 0 | 112 | 1 | 0 | 1 | 0 | 1 | 1 | 1 |
| 27 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 70 | 0 | 1 | 0 | 1 | 1 | 0 | 1 | 113 | 1 | 1 | 0 | 0 | 0 | 1 | 1 |
| 28 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 71 | 1 | 0 | 1 | 1 | 1 | 1 | 0 | 114 | 1 | 1 | 1 | 1 | 0 | 0 | 1 |
| 29 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 72 | 0 | 1 | 0 | 1 | 1 | 1 | 1 | 115 | 1 | 1 | 1 | 0 | 1 | 0 | 0 |
| 30 | 1 | 0 | 1 | 0 | 0 | 0 | 1 | 73 | 1 | 0 | 1 | 1 | 1 | 1 | 1 | 116 | 0 | 1 | 1 | 1 | 0 | 1 | 0 |
| 31 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 74 | 1 | 1 | 0 | 0 | 1 | 1 | 1 | 117 | 0 | 0 | 1 | 1 | 1 | 0 | 1 |
| 32 | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 75 | 1 | 1 | 1 | 1 | 0 | 1 | 1 | 118 | 1 | 0 | 0 | 0 | 1 | 1 | 0 |
| 33 | 0 | 0 | 1 | 1 | 0 | 0 | 0 | 76 | 1 | 1 | 1 | 0 | 1 | 0 | 1 | 119 | 0 | 1 | 0 | 0 | 0 | 1 | 1 |
| 34 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 77 | 1 | 1 | 1 | 0 | 0 | 1 | 0 | 120 | 1 | 0 | 1 | 1 | 0 | 0 | 1 |
| 35 | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 78 | 0 | 1 | 1 | 1 | 0 | 0 | 1 | 121 | 1 | 1 | 0 | 0 | 1 | 0 | 0 |
| 36 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 79 | 1 | 0 | 1 | 0 | 1 | 0 | 0 | 122 | 0 | 1 | 1 | 0 | 0 | 1 | 0 |
| 37 | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 80 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 123 | 0 | 0 | 1 | 1 | 0 | 0 | 1 |
| 38 | 1 | 1 | 0 | 1 | 1 | 0 | 0 | 81 | 0 | 0 | 1 | 0 | 1 | 0 | 1 | 124 | 1 | 0 | 0 | 0 | 1 | 0 | 0 |
| 39 | 0 | 1 | 1 | 0 | 1 | 1 | 0 | 82 | 1 | 0 | 0 | 0 | 0 | 1 | 0 | 125 | 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| 40 | 0 | 0 | 1 | 1 | 0 | 1 | 1 | 83 | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 126 | 0 | 0 | 1 | 0 | 0 | 0 | 1 |
| 41 | 1 | 0 | 0 | 0 | 1 | 0 | 1 | 84 | 1 | 0 | 1 | 1 | 0 | 0 | 0 |     |   |   |   |   |   |   |   |
| 42 | 1 | 1 | 0 | 1 | 0 | 1 | 0 | 85 | 0 | 1 | 0 | 1 | 1 | 0 | 0 |     |   |   |   |   |   |   |   |

GF (256)

$$p(X) = X^8 + X^4 + X^3 + X^2 + 1$$

|   |   |   |   |   |   |   |   |   |     |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|-----|---|---|---|---|---|---|---|---|
| 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 127 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 |
| 1 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 128 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 1 |
| 2 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 129 | 1 | 1 | 1 | 0 | 1 | 0 | 0 | 0 |
| 3 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 130 | 0 | 1 | 1 | 1 | 0 | 1 | 0 | 0 |
| 4 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 131 | 0 | 0 | 1 | 1 | 1 | 0 | 1 | 0 |
| 5 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 132 | 0 | 0 | 0 | 1 | 1 | 1 | 0 | 1 |
| 6 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 133 | 1 | 0 | 1 | 1 | 0 | 1 | 1 | 0 |
| 7 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 134 | 0 | 1 | 0 | 1 | 1 | 0 | 1 | 1 |
|   |   |   |   |   |   |   |   |   | 135 | 1 | 0 | 0 | 1 | 0 | 1 | 0 | 1 |

|    |   |   |   |   |   |   |   |   |     |   |   |   |   |   |   |   |   |
|----|---|---|---|---|---|---|---|---|-----|---|---|---|---|---|---|---|---|
| 8  | 1 | 0 | 1 | 1 | 1 | 0 | 0 | 0 | 136 | 1 | 1 | 1 | 1 | 0 | 0 | 1 | 0 |
| 9  | 0 | 1 | 0 | 1 | 1 | 1 | 0 | 0 | 137 | 0 | 1 | 1 | 1 | 1 | 0 | 0 | 1 |
| 10 | 0 | 0 | 1 | 0 | 1 | 1 | 1 | 0 | 138 | 1 | 0 | 0 | 0 | 0 | 1 | 0 | 0 |
| 11 | 0 | 0 | 0 | 1 | 0 | 1 | 1 | 1 | 139 | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 0 |
| 12 | 1 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 140 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 1 |
| 13 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 1 | 141 | 1 | 0 | 1 | 0 | 1 | 0 | 0 | 0 |
| 14 | 1 | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 142 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 0 |
| 15 | 0 | 1 | 1 | 0 | 0 | 1 | 0 | 0 | 143 | 0 | 0 | 1 | 0 | 1 | 0 | 1 | 0 |
| 16 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 0 | 144 | 0 | 0 | 0 | 1 | 0 | 1 | 0 | 1 |
| 17 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 145 | 1 | 0 | 1 | 1 | 0 | 0 | 1 | 0 |
| 18 | 1 | 0 | 1 | 1 | 0 | 1 | 0 | 0 | 146 | 0 | 1 | 0 | 1 | 1 | 0 | 0 | 1 |
| 19 | 0 | 1 | 0 | 1 | 1 | 0 | 1 | 0 | 147 | 1 | 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| 20 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 1 | 148 | 0 | 1 | 0 | 0 | 1 | 0 | 1 | 0 |
| 21 | 1 | 0 | 1 | 0 | 1 | 1 | 1 | 0 | 149 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 1 |
| 22 | 0 | 1 | 0 | 1 | 0 | 1 | 1 | 1 | 150 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 |
| 23 | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 1 | 151 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 |
| 24 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 1 | 152 | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 0 |
| 25 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 153 | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 1 |
| 26 | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 154 | 1 | 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| 27 | 0 | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 155 | 0 | 1 | 0 | 0 | 1 | 1 | 1 | 0 |
| 28 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 0 | 156 | 0 | 0 | 1 | 0 | 0 | 1 | 1 | 1 |
| 29 | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 157 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 1 |
| 30 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 158 | 1 | 1 | 1 | 0 | 1 | 1 | 0 | 1 |
| 31 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 159 | 1 | 1 | 0 | 0 | 1 | 1 | 1 | 0 |
| 32 | 1 | 0 | 1 | 1 | 1 | 0 | 0 | 1 | 160 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 1 |
| 33 | 1 | 1 | 1 | 0 | 0 | 1 | 0 | 0 | 161 | 1 | 0 | 0 | 0 | 1 | 0 | 1 | 1 |
| 34 | 0 | 1 | 1 | 1 | 0 | 0 | 1 | 0 | 162 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 1 |
| 35 | 0 | 0 | 1 | 1 | 1 | 0 | 0 | 1 | 163 | 1 | 1 | 0 | 0 | 0 | 1 | 1 | 0 |
| 36 | 1 | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 164 | 0 | 1 | 1 | 0 | 0 | 0 | 1 | 1 |
| 37 | 0 | 1 | 0 | 1 | 0 | 0 | 1 | 0 | 165 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 1 |
| 38 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 1 | 166 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0 |
| 39 | 1 | 0 | 1 | 0 | 1 | 1 | 0 | 0 | 167 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 0 |
| 40 | 0 | 1 | 0 | 1 | 0 | 1 | 1 | 0 | 168 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 |
| 41 | 0 | 0 | 1 | 0 | 1 | 0 | 1 | 1 | 169 | 1 | 0 | 1 | 0 | 0 | 1 | 1 | 1 |
| 42 | 1 | 0 | 1 | 0 | 1 | 1 | 0 | 1 | 170 | 1 | 1 | 1 | 0 | 1 | 0 | 1 | 1 |
| 43 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 0 | 171 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 1 |
| 44 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 172 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 0 |
| 45 | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 173 | 0 | 1 | 1 | 0 | 1 | 1 | 1 | 1 |
| 46 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 1 | 174 | 1 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 47 | 1 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 175 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 48 | 0 | 1 | 1 | 0 | 0 | 0 | 1 | 0 | 176 | 1 | 1 | 0 | 0 | 0 | 1 | 1 | 1 |
| 49 | 0 | 0 | 1 | 1 | 0 | 0 | 0 | 1 | 177 | 1 | 1 | 0 | 1 | 1 | 0 | 1 | 1 |
| 50 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 178 | 1 | 1 | 0 | 1 | 0 | 1 | 0 | 1 |
| 51 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 189 | 1 | 1 | 0 | 1 | 0 | 0 | 1 | 0 |
| 52 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 180 | 0 | 1 | 1 | 0 | 1 | 0 | 0 | 1 |
| 53 | 0 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 181 | 1 | 0 | 0 | 0 | 1 | 1 | 0 | 0 |
| 54 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 0 | 182 | 0 | 1 | 0 | 0 | 0 | 1 | 1 | 0 |
| 55 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 183 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 1 |
| 56 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 0 | 184 | 1 | 0 | 1 | 0 | 1 | 0 | 0 | 1 |
| 57 | 0 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 185 | 1 | 1 | 1 | 0 | 1 | 1 | 0 | 0 |
| 58 | 1 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 186 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 0 |
| 59 | 0 | 1 | 0 | 0 | 1 | 0 | 1 | 1 | 187 | 0 | 0 | 1 | 1 | 1 | 0 | 1 | 1 |
| 60 | 1 | 0 | 0 | 1 | 1 | 1 | 0 | 1 | 188 | 1 | 0 | 1 | 0 | 0 | 1 | 0 | 1 |
| 61 | 1 | 1 | 1 | 1 | 0 | 1 | 1 | 0 | 189 | 1 | 1 | 1 | 0 | 1 | 0 | 1 | 0 |
| 62 | 0 | 1 | 1 | 1 | 1 | 0 | 1 | 1 | 190 | 0 | 1 | 1 | 1 | 0 | 1 | 0 | 1 |
| 63 | 1 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 191 | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |
| 64 | 1 | 1 | 1 | 1 | 1 | 0 | 1 | 0 | 192 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 1 |
| 65 | 0 | 1 | 1 | 1 | 1 | 1 | 0 | 1 | 193 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 0 |
| 66 | 1 | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 194 | 0 | 1 | 0 | 0 | 1 | 1 | 0 | 0 |
| 67 | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 1 | 195 | 0 | 0 | 1 | 0 | 0 | 1 | 1 | 0 |
| 68 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 196 | 0 | 0 | 0 | 1 | 0 | 0 | 1 | 1 |
| 69 | 1 | 1 | 1 | 1 | 0 | 1 | 0 | 0 | 197 | 1 | 0 | 1 | 1 | 0 | 0 | 0 | 1 |

|     |   |   |   |   |   |   |   |   |     |   |   |   |   |   |   |   |   |
|-----|---|---|---|---|---|---|---|---|-----|---|---|---|---|---|---|---|---|
| 70  | 0 | 1 | 1 | 1 | 1 | 0 | 1 | 0 | 198 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0 |
| 71  | 0 | 0 | 1 | 1 | 1 | 1 | 0 | 1 | 199 | 0 | 1 | 1 | 1 | 0 | 0 | 0 | 0 |
| 72  | 1 | 0 | 1 | 0 | 0 | 1 | 1 | 0 | 200 | 0 | 0 | 1 | 1 | 1 | 0 | 0 | 0 |
| 73  | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 1 | 201 | 0 | 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| 74  | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 202 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 0 |
| 75  | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 203 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 |
| 76  | 0 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 204 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 |
| 77  | 0 | 0 | 1 | 1 | 1 | 1 | 0 | 0 | 205 | 1 | 1 | 1 | 0 | 0 | 1 | 0 | 1 |
| 78  | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 0 | 206 | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 0 |
| 79  | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 207 | 0 | 1 | 1 | 0 | 0 | 1 | 0 | 1 |
| 80  | 1 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 208 | 1 | 0 | 0 | 0 | 1 | 0 | 1 | 0 |
| 81  | 1 | 1 | 1 | 0 | 0 | 1 | 1 | 1 | 209 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 1 |
| 82  | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 1 | 210 | 1 | 0 | 0 | 1 | 1 | 0 | 1 | 0 |
| 83  | 1 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 211 | 0 | 1 | 0 | 0 | 1 | 1 | 0 | 1 |
| 84  | 1 | 1 | 0 | 1 | 0 | 1 | 1 | 0 | 212 | 1 | 0 | 0 | 1 | 1 | 1 | 1 | 0 |
| 85  | 0 | 1 | 1 | 0 | 1 | 0 | 1 | 1 | 213 | 0 | 1 | 0 | 0 | 1 | 1 | 1 | 1 |
| 86  | 1 | 0 | 0 | 0 | 1 | 1 | 0 | 1 | 214 | 1 | 0 | 0 | 1 | 1 | 1 | 1 | 1 |
| 87  | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 215 | 1 | 1 | 1 | 1 | 0 | 1 | 1 | 1 |
| 88  | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 216 | 1 | 1 | 0 | 0 | 0 | 0 | 1 | 1 |
| 89  | 1 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 217 | 1 | 1 | 0 | 1 | 1 | 0 | 0 | 1 |
| 90  | 1 | 1 | 1 | 1 | 1 | 0 | 1 | 1 | 218 | 1 | 1 | 0 | 1 | 0 | 1 | 0 | 0 |
| 91  | 1 | 1 | 0 | 0 | 0 | 1 | 0 | 1 | 219 | 0 | 1 | 1 | 0 | 1 | 0 | 1 | 0 |
| 92  | 1 | 1 | 0 | 1 | 1 | 0 | 1 | 0 | 220 | 0 | 0 | 1 | 1 | 0 | 1 | 0 | 1 |
| 93  | 0 | 1 | 1 | 0 | 1 | 1 | 0 | 1 | 221 | 1 | 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| 94  | 1 | 0 | 0 | 0 | 1 | 1 | 1 | 0 | 222 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 1 |
| 95  | 0 | 1 | 0 | 0 | 0 | 1 | 1 | 1 | 223 | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |
| 96  | 1 | 0 | 0 | 1 | 1 | 0 | 1 | 1 | 224 | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 0 |
| 97  | 1 | 1 | 1 | 1 | 0 | 1 | 0 | 1 | 225 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 |
| 98  | 1 | 1 | 0 | 0 | 0 | 0 | 1 | 0 | 226 | 0 | 0 | 0 | 1 | 0 | 0 | 1 | 0 |
| 99  | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 1 | 227 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 1 |
| 100 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 228 | 1 | 0 | 1 | 1 | 1 | 1 | 0 | 0 |
| 101 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 229 | 0 | 1 | 0 | 1 | 1 | 1 | 1 | 0 |
| 102 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 230 | 0 | 0 | 1 | 0 | 1 | 1 | 1 | 1 |
| 103 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 231 | 1 | 0 | 1 | 0 | 1 | 1 | 1 | 1 |
| 104 | 1 | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 232 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 1 |
| 105 | 0 | 1 | 0 | 1 | 1 | 0 | 0 | 0 | 233 | 1 | 1 | 0 | 0 | 1 | 1 | 1 | 1 |
| 106 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 0 | 234 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 1 |
| 107 | 0 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 235 | 1 | 1 | 0 | 1 | 0 | 1 | 1 | 1 |
| 108 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 1 | 236 | 1 | 1 | 0 | 1 | 0 | 0 | 1 | 1 |
| 109 | 1 | 0 | 1 | 1 | 1 | 1 | 0 | 1 | 237 | 1 | 1 | 0 | 1 | 0 | 0 | 0 | 1 |
| 110 | 1 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 238 | 1 | 1 | 0 | 1 | 0 | 0 | 0 | 0 |
| 111 | 0 | 1 | 1 | 1 | 0 | 0 | 1 | 1 | 239 | 0 | 1 | 1 | 0 | 1 | 0 | 0 | 0 |
| 112 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 240 | 0 | 0 | 1 | 1 | 0 | 1 | 0 | 0 |
| 113 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 241 | 0 | 0 | 0 | 1 | 1 | 0 | 1 | 0 |
| 114 | 0 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 242 | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 1 |
| 115 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 0 | 243 | 1 | 0 | 1 | 1 | 1 | 1 | 1 | 0 |
| 116 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 244 | 0 | 1 | 0 | 1 | 1 | 1 | 1 | 1 |
| 117 | 1 | 0 | 1 | 1 | 0 | 1 | 1 | 1 | 245 | 1 | 0 | 0 | 1 | 0 | 1 | 1 | 1 |
| 118 | 1 | 1 | 1 | 0 | 0 | 0 | 1 | 1 | 246 | 1 | 1 | 1 | 1 | 0 | 0 | 1 | 1 |
| 119 | 1 | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 247 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 1 |
| 120 | 1 | 1 | 0 | 1 | 1 | 1 | 0 | 0 | 248 | 1 | 1 | 0 | 1 | 1 | 0 | 0 | 0 |
| 121 | 0 | 1 | 1 | 0 | 1 | 1 | 1 | 0 | 249 | 0 | 1 | 1 | 0 | 1 | 1 | 0 | 0 |
| 122 | 0 | 0 | 1 | 1 | 0 | 1 | 1 | 1 | 250 | 0 | 0 | 1 | 1 | 0 | 1 | 1 | 0 |
| 123 | 1 | 0 | 1 | 0 | 0 | 0 | 1 | 1 | 251 | 0 | 0 | 0 | 1 | 1 | 0 | 1 | 1 |
| 124 | 1 | 1 | 1 | 0 | 1 | 0 | 0 | 1 | 252 | 1 | 0 | 1 | 1 | 0 | 1 | 0 | 1 |
| 125 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 253 | 1 | 1 | 1 | 0 | 0 | 0 | 1 | 0 |
| 126 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 254 | 0 | 1 | 1 | 1 | 0 | 0 | 0 | 1 |

## Appendix B

|        |                                            |
|--------|--------------------------------------------|
| ARQ:   | Automatic Repeat Request                   |
| ASE:   | Amplified Spontaneous Emission             |
| ASIC:  | Application Specific Integrated Circuit    |
| ASK:   | Amplitude Shift Keying                     |
| AWGN:  | Additive White Gaussian Noise              |
| ECC:   | Error-correcting (Control) Codes           |
| BCH:   | Bose-Chaudhuri-Hocqueaghem                 |
| BER:   | Bit-error rate                             |
| BPSK:  | Binary Phase Shift Keying                  |
| BTC:   | Block Turbo Code                           |
| CTC:   | Convolutional Turbo Code                   |
| DCF:   | Dispersion Compensating Fiber              |
| DFE:   | Decision Feedback Equalizer                |
| DFT:   | Discrete Fourier Transform                 |
| EDFA:  | Erbium-doped Fiber Amplifier               |
| FDM:   | Frequency-Division Multiplexing            |
| FEC:   | Forward Error Correction                   |
| FFE:   | Feed Forward Equalizer                     |
| FPGA:  | Field Programmable Gate Array              |
| FIR:   | Finite Impulse Response                    |
| FWM:   | Four-wave mixing                           |
| GCD:   | Greatest Common Divisor                    |
| GF:    | Galois Field                               |
| GFFT:  | Galois Field Fourier Transform             |
| GVD:   | Group Velocity Dispersion                  |
| HIHO:  | Hard-Input Hard-Output                     |
| IM/DD: | Intensity Modulation with Direct detection |
| ISI:   | Intersymbol Interference                   |
| LED:   | Light Emitting Diode                       |
| LFSR:  | Linear Feedback Shift Register             |
| LLR:   | Log-likelihood Ratio                       |
| LTI:   | Linear Time Invariant                      |
| LMS:   | Least Mean Square                          |
| MAP:   | Maximum a Posteriori Probability           |
| ML:    | Maximum Likelihood                         |
| MLSD:  | Maximum Likelihood Sequence Detection      |
| MSE:   | Mean Square Error                          |
| OEO:   | Optical-Electrical-Optical                 |
| OOK:   | On-Off Keying                              |
| OTDM:  | Optical Time-Division Multiplexing         |
| PCBC:  | Parallel Concatenated Block Code           |
| PCCC:  | Parallel Concatenated Convolutional Code   |
| PDM:   | Polarization-Division Multiplexing         |
| PMD:   | Polarization Mode Dispersion               |
| RIN:   | Relative Intensity Noise                   |
| RS:    | Reed-Solomon                               |
| SBS:   | Stimulated Brillouin Scattering            |
| SCBC:  | Serially Concatenated Block Code           |
| SCCC:  | Serially Concatenated Convolutional Code   |
| SISO:  | Soft-Input Soft-Output                     |
| SNR:   | Signal-to-Noise Ratio                      |
| SPM:   | Self Phase Modulation                      |
| SRS:   | Stimulated Raman Scattering                |
| TDM:   | Time-Division Multiplexing                 |
| WDM:   | Wavelength Division Multiplexing           |
| WEF:   | Weight Enumerating Function                |
| WGR:   | Wave guide-grating Router                  |
| XPM:   | Cross phase modulation                     |

## 10 BIBLIOGRAPHY

### Books:

- [1] G.P.Agrawal, Fiber-Optic Communication Systems, 2<sup>nd</sup> Edition, John Wiley & Sons, Inc. 1997.
- [2] E.R.Berlekamp, Algebraic Coding Theory, New York, McGraw-Hill, 1968.
- [3] R.E.Blahut, Theory and Practice of Error Control Codes, Reading, Mass. Addison Wesley, 1984.
- [4] G. Clark, J.Cain, Error-Correcting Coding for Digital Communications, New York, Plenum Press, 1981.
- [5] G.D.Forney Jr. Concatenated Codes, Cambridge, MIT Press, 1966.
- [6] R.G.Gallager, Information Theory and Reliable Communication, New York, John Wiley & Sons, 1968.
- [7] S. Lin, D.J. Costello, Jr., Error Control Coding, Englewood Cliffs, NJ, Prentice Hall Inc., 1983.
- [8] F.J.MacWilliams and N.J.A.Sloane, The Theory of Error Correcting Codes, Amsterdam: North Holland, 1977.
- [9] J.G.Proakis, M. Salehi, Communications Systems Engineering, 1<sup>st</sup> edition, Upper Saddle River, NJ, Prentice Hall Inc., 1994.
- [10] I.S. Reed, X.Chen, Error Control Coding For Data Networks, Kluwer Academic, cop. 1999.
- [11] B.Vucetic, J.Yuan, Turbo Codes: Principles and Applications, Kluwer Academic, 2000.
- [12] S.B.Wicker, Error Control Systems for Digital Communication and Storage, Upper Saddle River, NJ, Prentice Hall Inc., 1995.
- [13] S.B.Wicker, V.K.Bhargava, Reed-Solomon Codes and Their Applications, NJ, IEEE Press, 1994.
- [14] J. M. Wozencraft and I. M. Jacobs, Principles of Communication Engineering, New York, John Wiley & Sons, 1965.

### Conference papers, journals and articles

- [15] O.A.Sab, R.Pyndiah, Performance of Reed-Solomon Block Turbo Code, Proceeding of IEEE GLOBECOM Conference, vol 1/3, pp. 121-125, Nov. 1996.
- [16] O.A.Sab, Concatenated Forward Error Correction Schemes for Long-haul DWDM Optical Transmission Systems, proceedings of ECOC, 1999.
- [17] O.A.Sab, V.Lemaire, Block Turbo Code Performances for Long-haul DWDM Optical Transmission Systems, Optical Fiber Communication Conference, Baltimore USA, March 2000.
- [18] K.Azadet, E.Haratsch, H.Kim, F.Saibi, J.Saunders, M.Shaffer, L.Song, M.L.Yu, DSP techniques for Optical Transceivers, Custom Integrated Circuits Conference, IEEE, pp 281-287, 2001.
- [19] R.Bahl, J.Cocke, F.Jelinek, J.Raviv, Optimal Decoding of Linear Codes for Minimizing Symbol Error Rate, IEEE Transactions on Information Theory, March 1974.
- [20] S.Benedetto, G.Montorsi, Unveiling turbo-codes: Some results on parallel concatenated coding schemes, IEEE Transactions on Information Theory, vol. 42, no. 2, pp. 409-429, Mar. 1996.
- [21] E.R.Berlekamp, Bit Serial Reed-Solomon Encoder, IEEE Transactions on Information Theory, vol IT-28, Number 6, pp 869-874, November 1982.
- [22] E.R.Berlekamp, L.R.Welch, Error Correction for Algebraic Block Codes, U.S. Patent No. 4,633,470, issued Dec. 30, 1986.
- [23] C.Berrou, A.Glavieux, P.Thitimajshima, Near Shannon limit error-correcting coding and decoding: Turbo codes (1), International Conference on Communications, IEEE, vol. 2/3, pp. 1064-1070, May 1993.
- [24] D.Chase, A Class of Algorithms for Decoding Block Codes With Channel Measurement Information, IEEE Transaction on Information Theory, vol IT-18, No.1, pp. 170-182, January 1972.
- [25] R.T.Chien, Cyclic Decoding Procedure for the Bose-Chaudhuri-Hocquenghem Codes, IRE Transactions on Information Theory, vol IT 6, pp. 357-363, October 1964.
- [26] S.Dave, J.Kim, S.C.Kwatra, An Efficient Decoding Algorithm for Block Turbo Codes, IEEE Transactions on Communications, vol. 49, No.1, January 2001.
- [27] P.Elias, Error-Free Coding, IRE Transaction on Information Theory, vol IT-4, pp. 29-37, Sept. 1954.
- [28] S.T.J. Fenn, N.Benaissa, D.Taylor, Rapid Prototyping of Reed-Solomon Codes using Field Programmable Gate Arrays, Proceedings of the 2<sup>nd</sup> International Conference on Concurrent Engineering and Electronic Design Automation, pp. 309-312, April 1994.
- [29] G.D.Forney, On Decoding BCH Codes, IEEE Transactions on Information Theory, vol IT-11, pp. 549-557, October 1965.
- [30] D.Gorenstein, N.Zieler, A Class of Error Correcting Code in  $p^m$  symbols, Journal of the Society of Industrial and Applied Mathematics, vol 9, pp. 207-214, June 1961.
- [31] B.Green, G.Drolet, A Universal Reed-Solomon Decoder Chip, Proceedings of the 16<sup>th</sup> Biennial Symposium on Communications, Kingston, Ontario, pp. 327-330, May 1992.

- [32] J.Hagenauer, E.Offer, L.Papke, Iterative decoding of binary block and convolutional codes, IEEE Transactions on Information Theory, vol 42, No. 2, pp. 429-445, March 1996.
- [33] R.W.Hamming, Error Detecting and Error Correcting Codes, Bell System Technical Journal, vol. 29, pp. 147-160, April 1950.
- [34] M.A.Hasan and V.K.Bhargava, A VLSI Architecture for a Low Complexity Rate Adaptive Reed-Solomon Encoder, Proceedings of the 16<sup>th</sup> Biennial Symposium on Communications, Kingston, Ontario, pp 331-334 May 1992.
- [35] M.A.Hasan and V.K.Bhargava, Bit Serial Systolic Divider and Multiplier for GF ( $2^m$ ), IEEE Transactions on Computers, vol 41, pp 972-980 August 1992.
- [36] ITU-T G.975, Forward Error Correction for Submarine Applications, October 2000.
- [37] H.T.Kung, Why Systolic Architectures?, IEEE Computer Magazine, vol. 15, pp. 37 – 45, 1982.
- [38] J.L.Massey, Shift Register Synthesis and BCH Decoding, IEEE Transactions on Information Theory, vol IT-15, No.1. pp. 122-127, January 1969.
- [39] J.L.Massey, J.K.Omura, Apparatus for finite field computation, U.S. Patent Application, pp. 21-40, 1984.
- [40] H.Nicki, J.Hageneauer, F.Burkert, Approaching Shannon's capacity limit by 0.27 dB using simple Hamming codes, IEEE Communications Letters, Vol. 1, No.5, pp. 130-132, Sept. 1997.
- [41] W.W.Peterson, Encoding and Error-Correction Procedures for the Bose-Chaudhuri Codes, IRE Transactions on Information Theory, vol IT-6, pp. 459-470, September 1960.
- [42] R. Pyndiah, A.Glavieux, A. Picart, S.Jacq, Near Optimum Decoding of Products Codes, Proceeding of IEEE GLOBECOM Conference, vol. 1/3, pp.339-343, Nov.- Dec. 1994.
- [43] R. Pyndiah, Near Optimum Decoding of Products Codes: Block Turbo Codes, IEEE Transactions on Communications, vol. 46, No. 8, pp. 1003-1010, August 1998.
- [44] R.Pyndiah, Iterative Decoding of Product Codes: Block Turbo Codes, International Symposium on Turbo Codes and related topics, Brest, September 1997.
- [45] N.Ramanujam, A.B.Puc, G.Lenner, H.D. Kidorf, C.R. Davidson, I.Hayee, J-X, Cai, M.Nissov, A.Pilipetskii, C.Rivers, N.S. Bergano, Forward Error Correction (FEC) Techniques in Long-haul Optical Transmission Systems, Lasers and Electro-Optics Society 2000 Annual Meeting. LEOS 2000. 13th Annual Meeting.IEEE, vol. 2, pp 405-406 2000.
- [46] A.Schmitt, Forward Error Correction advances optical-network performance, Lightwave August 1999.
- [47] K.Seki, K.Mikami, M.Baba, N.Shinohara, S.Suzuki, H.Tezuka, S.Uchino, N.Okada, Y.Kakinuma, A.Katayama, Single – chip 10.7 Gb/s FEC CODEC LSI Using Time – Multiplexed RS decoder, IEEE Custom Integrated Circuits Conference, pp. 289 – 292, 2001.
- [48] G.Seroussi, A Systolic Reed-Solomon Encoder, IEEE Transactions on Information Theory, vol IT-37, pp 1217-1220, July 1991.
- [49] C.E.Shannon, A Mathematical Theory of Communication, Bell System Technical Journal, vol. 27, pp. 379-423, July (1948a).
- [50] C.E.Shannon, A Mathematical Theory of Communication, Bell System Technical Journal, vol. 27, pp. 623-656, October (1948b).
- [51] R.C.Singleton, Maximum Distance Q-Nary Codes, IEEE Transactions on Information Theory, vol IT-10, pp. 116-118, 1964.
- [52] J.Sjölander, Equalization of Fiber Optic Channels, Master thesis, Royal Institute of Technology, IR-SB-EX-0118 (1032157), December 2001.
- [53] B.Sklar, A Primer on Turbo Code Concepts, IEEE Communications Magazine, pp. 94-102, December 1997.
- [54] Y.Sugiyama, M. Kasahara, S.Hirasawa, T.Namekawa, A Method for Solving Key Equations for Decoding Goppa Codes, Information and Control, vol. 27, pp. 87 – 89, January 1975.
- [55] H.Tezuka, I.Matsuoka, T.Asahi, N.Arai, T.Suzaki, Y.Aoki, K.Emura, 2.677 Gbit/s – Throughput Forward Error Correction LSI For Long – Haul Optical Transmission Systems, ECOC, pp. 561 – 562, September 1998.

